DEVOPS ENGINEERING MASTERY

CLOUD ENGINEERING MICROSOFT AZURE

Azure Architecture, Entra ID, Virtual Networks,
Virtual Machines, Storage, Databases, AKS,
Azure DevOps, Security & Production Cloud Operations

VOLUME 12 • FIRST EDITION

```
nahid@technova:~$ az account show
✓ Subscription: technova-prod  •  tenant: technova.onmicrosoft.com
nahid@technova:~$ az vm list --resource-group technova-prod-rg --output table
✓ 4 VMs running  •  2 zones  •  Managed Identity on
nahid@technova:~$ az storage blob list --account-name technovastore
```

Entra ID • VNet • VM • Storage • SQL • AKS • Functions • MERN

First Edition • Published 9 November 2026 • Dhaka, Bangladesh

24 Chapters • 24 Guided Labs • 3 Major Projects • 120 MCQs

Cloud Engineering — Microsoft Azure

DEVOPS ENGINEERING MASTERY — Volume 12

Azure Architecture, Entra ID, Virtual Networks, Virtual Machines, Storage, Databases, AKS, Azure DevOps, Security ও Production Cloud Operations

First Edition — Published 9 November 2026.

Hands-on environment: Azure CLI, Azure PowerShell, Azure Portal, Bicep and Terraform, with Node.js 20, Docker, and a MERN (MongoDB, Express, React, Node.js) example application called TechNova. All domains, IP addresses, subscription IDs and credentials in examples are fictional (technova.example, 203.0.113.x, 00000000-0000-0000-0000-000000000000, CHANGE-ME-...). Sample terminal outputs are illustrative. Azure service availability, features and portal behaviour change over time — official Microsoft Learn documentation is the authority, and no price is stated as a timeless fact in this book: always check the official Azure Pricing Calculator and pricing pages before running a lab. Resources that can generate charges — NAT Gateway, AKS, VMs, Application Gateway, managed databases, Log Analytics ingestion, data transfer — are flagged in every lab, and every lab ends with explicit cleanup steps.

How to use this book. Read the parts in order — each part builds on the previous one. Keep an Azure subscription (with MFA on privileged accounts and a budget alert switched on) and the Azure CLI beside you, and do every hands-on exercise and every lab in the companion Lab Workbook before moving on. Do not copy a CLI command or a policy document blindly; read the explanation under each field first. Destructive operations are always flagged with a WARNING box and a safe alternative. Never practise on a production subscription: use a dedicated test subscription, and never paste a real secret or access key into an example — use the fictional placeholders shown.

A note on honesty. Reading a book alone does not make anyone an expert. This book teaches the foundations and the practical judgment to progress toward professional engineering — but the judgment itself comes from doing the labs, breaking a test environment, finding the root cause in your own logs and metrics, fixing it, and repeating. There is no shortcut for that practice.

Conventions. Bangla is the explanatory language; technical terms, command names, APIs and configuration keys stay in English. `monospace` marks commands, file paths and code. **CONCEPT** explains an idea, **REAL WORLD** shows an industry scenario, **WARNING** marks risk (including cost risk), **PRO TIP** gives professional advice, and **LAB** marks hands-on work.

© 2026 DevOps Engineering Mastery series. All rights reserved. This book is for personal learning. Cloud changes should first be practised in an isolated test account, never directly on a production system. The author and publisher accept no liability for data loss, service disruption or unexpected cloud charges caused by running commands without a backup, an approval gate, or a billing alert.

Who this book is for

A learner who knows JavaScript, React, Next.js, Node.js, Express and MongoDB, understands basic Git and Linux, has studied Linux server administration, CI/CD, Docker, Kubernetes, Terraform and Ansible, and

wants to design, build, secure, operate and troubleshoot real cloud systems on Microsoft Azure — from a first Entra ID role assignment to a production multi-zone MERN platform with load balancing, auto scaling, managed databases, containers, CI/CD, observability, backup and disaster recovery.

Learning outcomes

By the end of this book you will be able to: explain cloud computing models and Azure global infrastructure and design for Regions and Availability Zones; govern Azure subscriptions with Management Groups, Azure Policy and landing zone concepts; design least-privilege Azure RBAC role assignments, Managed Identities and Conditional Access and troubleshoot access failures; use the Azure CLI, PowerShell, SDKs and workload identity federation safely; apply the Shared Responsibility Model and the Well-Architected Framework and investigate an unexpected bill; design VNets with correct CIDR planning, public and private subnets, NAT Gateway and Private Endpoints; secure networks with NSGs and ASGs and connect hybrid networks with VPN Gateway and ExpressRoute; run hardened VMs with cloud-init, extensions and Azure Bastion; operate Azure Load Balancer, Application Gateway and Front Door with VM Scale Sets; engineer Blob Storage, Managed Disks, Azure SQL, PostgreSQL, Cosmos DB and Managed Redis for production data; deliver content with Azure DNS and Front Door; run containers on ACR, Container Apps and AKS; build serverless, event-driven systems with Functions, Service Bus and Event Grid; build CI/CD with Azure Pipelines and GitHub Actions workload identity federation; provision infrastructure with ARM templates, Bicep and Terraform; monitor with Azure Monitor, Log Analytics and Application Insights, secure with Key Vault, Defender for Cloud and Microsoft Sentinel; and design reliable, recoverable, cost-controlled production architectures with runbooks.

সূচিপত্র — Contents

PART I — Azure Foundations & Architecture

Chapter 01 — Cloud Computing and Microsoft Azure Global Infrastructure 7

Chapter 02 — Azure Accounts, Subscriptions and Governance 12

Chapter 03 — Microsoft Entra ID and Identity Engineering 16

Chapter 04 — Azure CLI, PowerShell and SDK Automation 20

Chapter 05 — Azure Resource Manager, Policy and Well-Architected 23

PART II — Networking & Compute Engineering

Chapter 06 — Azure Virtual Network Architecture 28

Chapter 07 — Network Security and Private Connectivity 32

Chapter 08 — Azure DNS and Load Balancing 36

Chapter 09 — Azure Virtual Machines 39

Chapter 10 — Hybrid Connectivity 43

Chapter 11 — Availability and Compute Scaling 46

PART III — Storage, Database & Application Services

Chapter 12 — Azure Storage Engineering 50

Chapter 13 — Disks, Files and Backup 54

Chapter 14 — Azure Relational Database Engineering 57

Chapter 15 — Cosmos DB and Caching 60

Chapter 16 — Application Hosting and Edge Delivery 63

PART IV — Containers, Serverless & Delivery

Chapter 17 — Azure Container Engineering 67

Chapter 18 — Serverless and Messaging 71

Chapter 19 — Azure DevOps and CI/CD 74

Chapter 20 — Infrastructure as Code on Azure 78

PART V — Production Azure Engineering

Chapter 21 — Observability and Monitoring 83

Chapter 22 — Azure Security Engineering 86

Chapter 23 — Reliability and Disaster Recovery 89

Chapter 24 — Production Architecture and FinOps 92

Companion Volumes

Lab Workbook — 24 Guided Labs • 15 Assignments • 3 Major Projects • Capstone

Assessment and Answer Guide — 120 MCQs • Short Answers • Scenarios • Case Studies • Final Exam

Quick Reference Handbook — Glossary (150 Terms) • Azure CLI / PowerShell / Entra ID / VNet References • Architecture Patterns • Checklists • Decision Trees

Guided labs (24), major projects (3) and the capstone are in the separate *Azure Lab Workbook*. MCQs, assignments, examinations and the answer key are in the separate *Assessment and Answer Guide*. Glossary, Azure CLI, PowerShell, Entra ID and VNet references, architecture patterns, checklists and decision trees are in the separate *Quick Reference Handbook*.

PART I • DEVOPS ENGINEERING MASTERY

Azure Foundations & Architecture

ক্লাউড কম্পিউটিং, Azure গ্লোবাল ইনফ্রাস্ট্রাকচার, অ্যাকাউন্ট ও সাবস্ক্রিপশন গভর্ন্যান্স, Microsoft Entra ID, Azure CLI ও PowerShell, ARM, Policy ও Well-Architected।

অধ্যায় ১ • PART I

Chapter 1 — Cloud Computing and Microsoft Azure Global Infrastructure

১.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি পারবেন — (১) ক্লাউড কম্পিউটিং কী এবং প্রচলিত অন-প্রিমিসেস অবকাঠামোর তুলনায় এটি কোন সমস্যার সমাধান করে তা প্রথম নীতি থেকে ব্যাখ্যা করতে; (২) IaaS, PaaS ও SaaS-এর মধ্যে পার্থক্য করতে এবং কোনো কাজের জন্য কোন মডেলটি উপযুক্ত তা সিদ্ধান্ত নিতে; (৩) পাবলিক, প্রাইভেট ও হাইব্রিড ক্লাউডের তুলনা করতে; (৪) Azure Region, Availability Zone (AZ) ও Edge Location-এর সংজ্ঞা, সম্পর্ক ও ব্যবহার ব্যাখ্যা করতে; (৫) রিজিওনাল সার্ভিস বনাম গ্লোবাল সার্ভিসের পার্থক্য বুঝতে; (৬) ফল্ট ডোমেইন ও ফেইলিওর ডোমেইন ধারণা দিয়ে উচ্চ-উপলভ্যতা নকশা করতে; এবং (৭) TechNova-র MERN অ্যাপ্লিকেশনের জন্য একটি মাল্টি-AZ আর্কিটেকচারের রূপরেখা আঁকতে।

১.২ পূর্বশর্ত জ্ঞান

এই অধ্যায় পড়ার আগে আপনার কম্পিউটার নেটওয়ার্কের মৌলিক ধারণা থাকা ভালো — IP ঠিকানা, DNS, HTTP/HTTPS এবং ক্লায়েন্ট-সার্ভার মডেল। সার্ভার বলতে কী বোঝায়, ডেটাবেস কীভাবে কাজ করে এবং লিনাক্স টার্মিনালে সাধারণ কমান্ড চালানোর অভিজ্ঞতা থাকলে ভালো হয়। Azure সম্পর্কে আগের কোনো অভিজ্ঞতা দরকার নেই। আপনি যদি Volume 1–11 পড়ে এসে থাকেন, তাহলে Linux, নেটওয়ার্কিং, Docker ও CI/CD-র ধারণাগুলো এখানে বারবার কাজে লাগবে।

১.৩ কেন এই বিষয়টি গুরুত্বপূর্ণ

আপনি যদি একজন ডেভঅপস ইঞ্জিনিয়ার হতে চান, তাহলে ক্লাউড আপনার কর্মক্ষেত্র। আজকের দিনে প্রায় প্রতিটি আধুনিক অ্যাপ্লিকেশন — ব্যাংকিং অ্যাপ, ই-কমার্স সাইট, স্ট্রিমিং সেবা, সরকারি পোর্টাল — কোনো না কোনো ক্লাউডে চলে। কিন্তু ক্লাউড শুধু “অন্যের কম্পিউটার ভাড়া নেওয়া” নয়। ক্লাউডের আসল শক্তি হলো এটি ব্যর্থতাকে (failure) একটি স্বাভাবিক ঘটনা হিসেবে ধরে নিয়ে নকশা করে। একটি সার্ভার নষ্ট হবে, একটি ডেটা সেন্টারে বিদ্যুৎ যাবে, একটি নেটওয়ার্ক লিংক ছিঁড়বে — ক্লাউড আর্কিটেকচারের কাজ হলো এসব ঘটনা সত্ত্বেও ব্যবহারকারী যেন কিছু টের না পায়, তা নিশ্চিত করা। এই অধ্যায়ে আপনি সেই চিন্তাভাবনার ভিত্তি শিখবেন। Region ও Availability Zone না বুঝে আপনি কখনোই একটি নির্ভরযোগ্য সিস্টেম বানাতে পারবেন না।

১.৪ প্রথম নীতি থেকে: ক্লাউড কম্পিউটিং কী

ক্লাউড কম্পিউটিং (Cloud Computing) হলো ইন্টারনেটের মাধ্যমে চাহিদা অনুযায়ী কম্পিউটিং রিসোর্স — সার্ভার, স্টোরেজ, ডেটাবেস, নেটওয়ার্কিং, সফটওয়্যার — ভাড়া নেওয়ার মডেল। প্রচলিত মডেলে একটি কোম্পানিকে নিজের ডেটা সেন্টার কিনতে হতো, সার্ভার কিনতে হতো, বিদ্যুৎ ও কুলিংয়ের ব্যবস্থা করতে হতো, এবং সর্বোচ্চ লোডের কথা ভেবে প্রয়োজনের চেয়ে বেশি ক্ষমতা কিনে রাখতে হতো। এর ফলে (১) শুরুতে বিশাল মূলধন ব্যয় (capital expenditure), (২) ক্ষমতা বাড়াতে সপ্তাহ বা মাসের অপেক্ষা, (৩) কম ব্যবহারের সময়েও পুরো খরচ বহন — এই তিনটি সমস্যা তৈরি হতো। ক্লাউড এই তিনটি সমস্যারই সমাধান করে: আপনি শুধু যা ব্যবহার করেন তার জন্য অর্থ দেন (pay-as-you-go), কয়েক মিনিটে নতুন সার্ভার পান, এবং লোড বাড়লে স্বয়ংক্রিয়ভাবে ক্ষমতা বাড়াতে পারেন।

Microsoft Azure হলো Microsoft-এর ক্লাউড প্ল্যাটফর্ম — ২০১০ সালে চালু হওয়া এই প্ল্যাটফর্ম আজ বিশ্বের দ্বিতীয় বৃহত্তম পাবলিক ক্লাউড। Azure-এ ২০০-র বেশি সার্ভিস আছে: কম্পিউট, স্টোরেজ, ডেটাবেস, নেটওয়ার্কিং, AI, IoT, নিরাপত্তা। একজন DevOps ইঞ্জিনিয়ার হিসেবে আপনার কাজ হলো এই সার্ভিসগুলোকে একত্র করে এমন একটি সিস্টেম বানানো যা নির্ভরযোগ্য, নিরাপদ, স্কেলযোগ্য এবং খরচ-সাশ্রয়ী।

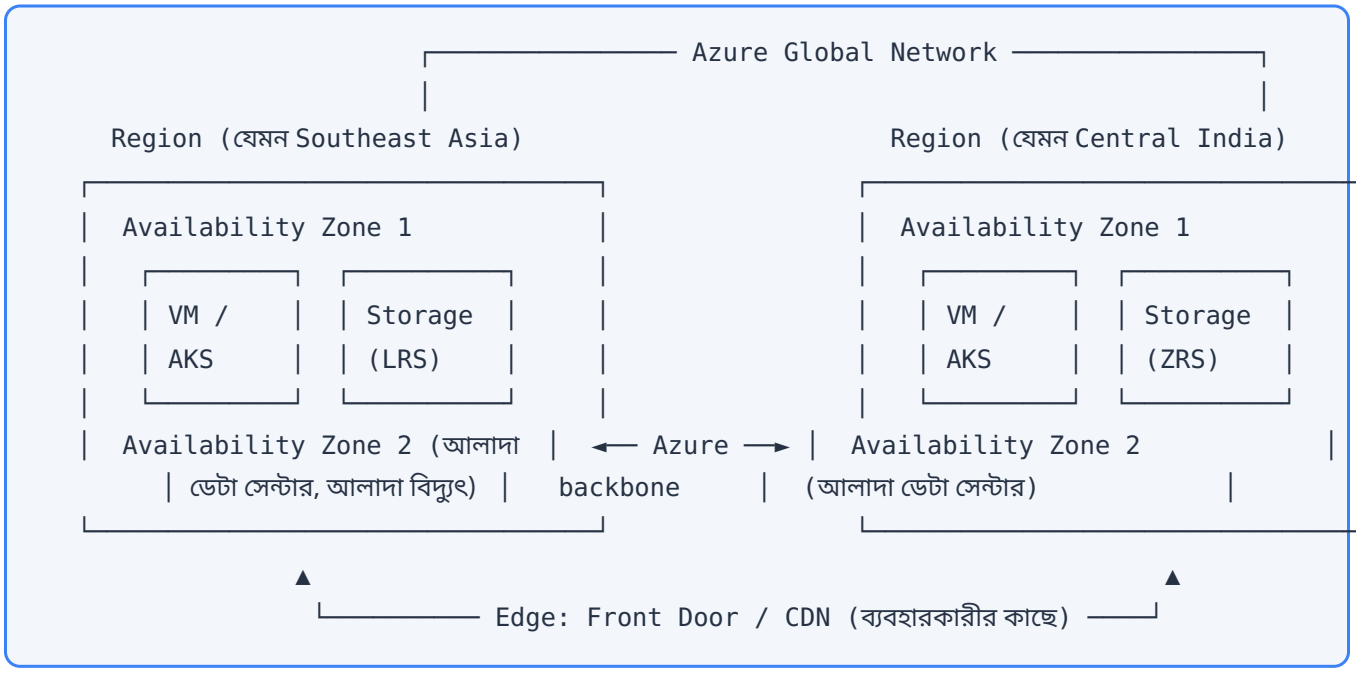
১.৫ IaaS, PaaS ও SaaS — দায়িত্বের তিনটি স্তর

ক্লাউড মডেলগুলোর পার্থক্য বোঝার সবচেয়ে ভালো উপায় হলো প্রশ্ন করা: “কে কোন স্তরের দায়িত্ব নেয়?” একটি অ্যাপ্লিকেশন চালাতে পাঁচটি স্তর লাগে — হার্ডওয়্যার ও ডেটা সেন্টার, ভার্চুয়ালাইজেশন, অপারেটিং সিস্টেম, রানটাইম ও মিডলওয়্যার, এবং অ্যাপ্লিকেশন কোড ও ডেটা।

মডেল	প্রদানকারী যা দেয়	আপনি যা করেন	Azure উদাহরণ
IaaS	হার্ডওয়্যার, ভার্চুয়ালাইজেশন, নেটওয়ার্ক	OS, রানটাইম, অ্যাপ, ডেটা	Virtual Machines, Virtual Network
PaaS	IaaS + OS + রানটাইম	শুধু অ্যাপ কোড ও ডেটা	App Service, Azure SQL Database
SaaS	সবকিছু — সম্পূর্ণ সফটওয়্যার	শুধু ব্যবহার ও কনফিগারেশন	Microsoft 365, Dynamics 365

একটি ভুল ধারণা হলো “PaaS সবসময় ভালো”। আসলে সিদ্ধান্ত নির্ভর করে নিয়ন্ত্রণের প্রয়োজনে। আপনার যদি OS-স্তরে বিশেষ কনফিগারেশন লাগে (যেমন একটি নির্দিষ্ট কার্নেল মডিউল), তাহলে IaaS (VM) দরকার। আপনার যদি শুধু Node.js অ্যাপ চালাতে হয়, তাহলে App Service (PaaS) অনেক কম পরিচালনা-ভার দেয়। TechNova-র MERN অ্যাপের জন্য আমরা এই বইয়ে দুটো পথই দেখব: VM-ভিত্তিক এবং PaaS/কনটেইনার-ভিত্তিক।

১.৬ Azure গ্লোবাল ইনফ্রাস্ট্রাকচার: Region, Availability Zone, Edge



Region হলো একটি ভৌগোলিক এলাকায় অবস্থিত ডেটা সেন্টারের সমষ্টি — যেমন Southeast Asia (সিঙ্গাপুর), Central India, West Europe। প্রতিটি Region-এ এক বা একাধিক **Availability Zone (AZ)** থাকে। প্রতিটি AZ হলো এক বা

একাধিক আলাদা ডেটা সেন্টার — আলাদা বিদ্যুৎ সরবরাহ, আলাদা কুলিং, আলাদা নেটওয়ার্ক। একই Region-এর AZ-গুলোর মধ্যে খুব কম ল্যাটেন্সির (সাধারণত ২ মিলিসেকেন্ডের কম) নেটওয়ার্ক সংযোগ থাকে।

ফল্ট ডোমেইন (Fault Domain) হলো এমন একটি একক যেখানে একটি একক ব্যর্থতা — যেমন একটি র‍্যাকের বিদ্যুৎ বিভ্রাট — একসাথে সব রিসোর্সকে প্রভাবিত করতে পারে। AZ হলো সবচেয়ে বড় ফল্ট ডোমেইন: একটি AZ-এর পুরো ডেটা সেন্টার বন্ধ হয়ে গেলেও অন্য AZ-এর রিসোর্স চলতে থাকে। **ফেইলিওর ডোমেইন (Failure Domain)** একই ধারণার বিস্তৃত রূপ — Region-স্তরেও ফেইলিওর ডোমেইন আছে: একটি Region-জুড়ে বিপর্যয় হলে অন্য Region-এ চলা রেপ্লিকা দিয়ে সেবা চালু রাখা যায়। এজন্যই প্রোডাকশন আর্কিটেকচারে আমরা অন্তত দুটি AZ ব্যবহার করি, এবং গুরুত্বপূর্ণ সিস্টেমে দুটি Region-এর পরিকল্পনা করি।

> **CONCEPT:** কিছু Azure সার্ভিস **রিজিওনাল** (যেমন Virtual Machines, Azure SQL — প্রতিটি Region-এ আলাদা করে তৈরি করতে হয়) আর কিছু **গ্লোবাল** (যেমন Azure DNS, Front Door, Microsoft Entra ID — একটি টেন্যান্ট সব Region-এ কাজ করে)। কোন সার্ভিসটি কোন ধরনের, তা না জানলে বিপর্যয়-পুনরুদ্ধারের (disaster recovery) পরিকল্পনা ভুল হবে।

১.৭ পাবলিক, প্রাইভেট ও হাইব্রিড ক্লাউড

- **পাবলিক ক্লাউড:** Microsoft-এর মালিকানাধীন অবকাঠামোতে আপনার রিসোর্স চলে; অন্য গ্রাহকদের সাথে হার্ডওয়্যার ভাগ হয়, কিন্তু ভার্চুয়ালাইজেশন ও আইসোলেশন দিয়ে আলাদা থাকে। সবচেয়ে সাশ্রয়ী ও দ্রুত।
- **প্রাইভেট ক্লাউড:** শুধু আপনার সংস্থার জন্য আলাদা অবকাঠামো (অন-প্রিমিসেস বা Azure Stack HCI)। নিয়ন্ত্রণ বেশি, কিন্তু খরচ ও পরিচালনা-ভার বেশি।
- **হাইব্রিড ক্লাউড:** পাবলিক ক্লাউড + অন-প্রিমিসেস/প্রাইভেট ক্লাউড একসাথে — যেমন সংবেদনশীল ডেটা নিজের ডেটা সেন্টারে, আর পাবলিক ওয়েবসাইট Azure-এ। Azure Arc দিয়ে অন-প্রিমিসেস সার্ভারকেও Azure থেকে পরিচালনা করা যায়।

১.৮ TechNova উদাহরণ: একটি মাল্টি-AZ MERN আর্কিটেকচারের রূপরেখা

TechNova একটি MERN অ্যাপ (React ফ্রন্টএন্ড, Express API, MongoDB-সদৃশ ডেটা) চালায়। Southeast Asia Region-এ আমরা দুটি AZ ব্যবহার করব: প্রতিটি AZ-তে একটি করে API VM (বা AKS নোড), একটি Application Gateway, এবং ডেটাবেসের জন্য Zone-redundant সেটআপ। একটি AZ বন্ধ হয়ে গেলে ট্র্যাফিক স্বয়ংক্রিয়ভাবে অন্য AZ-এ যাবে। ফ্রন্টএন্ড থাকবে Static Web Apps বা Front Door-এর পেছনে। এই বইয়ের শেষে (অধ্যায় ২৪) এই আর্কিটেকচারটি সম্পূর্ণরূপে বানানো হবে।

> **WARNING:** Region বাছাইয়ের সময় তিনটি বিষয় দেখুন: (১) আপনার ব্যবহারকারীর কাছাকাছি Region (ল্যাটেন্সি), (২) প্রয়োজনীয় সার্ভিসটি সেই Region-এ আছে কি না (সব সার্ভিস সব Region-এ নেই), (৩) ডেটা রেসিডেন্সি নিয়ম (কিছু দেশে ডেটা দেশের ভেতরে রাখা বাধ্যতামূলক)। একবার ডেটা তৈরি হয়ে গেলে Region বদলানো কঠিন ও ব্যয়বহুল।

১.৯ সাধারণ ভুল ও ট্রাবলশুটিং

- **ভুল:** “একটি Region মানেই উচ্চ-উপলভ্যতা।” — না; একটি Region-এর ভেতরে একটি মাত্র AZ ব্যবহার করলে AZ-স্তরের ব্যর্থতায় সেবা বন্ধ হবে। কমপক্ষে দুটি AZ ব্যবহার করুন।
- **ভুল:** “Azure-এ ডেটা রাখলেই ব্যাকআপ হয়ে গেল।” — না; রিডান্ড্যান্সি (ZRS/GRS) হার্ডওয়্যার ব্যর্থতা থেকে রক্ষা করে, কিন্তু ভুল করে ডিলিট করা ডেটা বা র‍্যানসমওয়্যার থেকে নয়। ব্যাকআপ আলাদা বিষয় (অধ্যায় ১৩ ও ২৩)।
- **ট্রাবলশুটিং:** কোনো সার্ভিস তৈরি করতে গিয়ে “not available in this region” দেখলে Azure Portal-এর Region ফিল্টার দেখুন বা Microsoft Learn-এর “Products available by region” পাতা যাচাই করুন।

১.১০ হাতে-কলমে: Region ও Zone যাচাই

```
az account list-locations --query "[?contains(name, 'asia')].{Region:name,
Display:displayName}" --output table
az account list-locations --query "[?name=='southeastasia'].{Region:name,
Zones:availabilityZoneMappings}" --output json
```

প্রথম কমান্ডটি এশিয়ার Region-গুলোর তালিকা দেখায়, দ্বিতীয়টি Southeast Asia Region-এ কোন Availability Zone ম্যাপিং আছে তা দেখায়। Azure CLI না থাকলে Portal-এ “All services → Resource groups → Create” পাতায় Region ড্রপডাউনেও একই তথ্য পাবেন।

১.১২ সারসংক্ষেপ

ক্লাউড কম্পিউটিং মানে চাহিদা অনুযায়ী কম্পিউটিং রিসোর্স ভাড়া নেওয়া — মূলধন ব্যয়ের বদলে পরিচালন ব্যয়, এবং মিনিটের মধ্যে স্কেল। IaaS দায়িত্ব কম নেয়, PaaS বেশি, SaaS সবচেয়ে বেশি। Azure-এর গ্লোবাল ইনফ্রাস্ট্রাকচারের ভিত্তি হলো Region ও Availability Zone: AZ হলো আলাদা ডেটা সেন্টার, এবং দুটি AZ ব্যবহার করাই উচ্চ-উপলভ্যতার প্রথম ধাপ। রিজিওনাল বনাম গ্লোবাল সার্ভিসের পার্থক্য, ফল্ট ডোমেইন ও ফেইলিওর ডোমেইনের ধারণা — এই ভিত্তির ওপরই বাকি ২৩ অধ্যায় দাঁড়িয়ে আছে।

১.১৩ পরিভাষা

Region — একটি ভৌগোলিক এলাকায় অবস্থিত Azure ডেটা সেন্টারের সমষ্টি; যেমন Southeast Asia।

Availability Zone (AZ) — একটি Region-এর ভেতরে আলাদা বিদ্যুৎ, কুলিং ও নেটওয়ার্কসহ এক বা একাধিক ডেটা সেন্টার।

IaaS — Infrastructure as a Service — প্রদানকারী হার্ডওয়্যার ও ভার্চুয়ালাইজেশন দেয়, আপনি OS ও অ্যাপ পরিচালনা করেন।

PaaS — Platform as a Service — প্রদানকারী OS ও রানটাইমও দেয়, আপনি শুধু অ্যাপ কোড লেখেন।

SaaS — Software as a Service — সম্পূর্ণ তৈরি সফটওয়্যার ব্যবহারের মডেল।

Fault Domain — একটি একক ব্যর্থতায় একসাথে প্রভাবিত হতে পারে এমন রিসোর্সের গোষ্ঠী।

Edge Location — ব্যবহারকারীর কাছাকাছি অবস্থিত ক্যাশিং ও ডেলিভারি পয়েন্ট (Front Door / CDN)।

১.১৪ জ্ঞান যাচাই

1. IaaS, PaaS ও SaaS-এর মধ্যে দায়িত্বের পার্থক্য কী? একটি উদাহরণ দিয়ে বোঝান।
2. একটি Region-এর ভেতরে দুটি Availability Zone ব্যবহার করলে কী সুবিধা হয়? একটি AZ বন্ধ হলে কী ঘটে?
3. রিজিওনাল সার্ভিস ও গ্লোবাল সার্ভিসের পার্থক্য কী? দুটি করে উদাহরণ দিন।
4. ফল্ট ডোমেইন ও ফেইলিওর ডোমেইনের মধ্যে পার্থক্য কী?
5. হাইব্রিড ক্লাউড কখন প্রয়োজন হয়? একটি বাস্তব দৃশ্যকল্প লিখুন।

১.১৫ অনুশীলন

Azure Portal-এ (বা CLI-তে) Southeast Asia ও Central India Region-এ কোন কোন Availability Zone আছে তা দেখুন। TechNova-র MERN অ্যাপের জন্য একটি এক পাতার আর্কিটেকচার নোট লিখুন: কোন Region, কয়টি AZ, কেন — এবং একটি AZ বন্ধ হলে কী হবে।

অধ্যায় ২ • PART I

Chapter 2 — Azure Accounts, Subscriptions and Governance

২.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি পারবেন — (১) Microsoft Entra টেন্যান্ট, Azure সাবস্ক্রিপশন, Management Group ও Resource Group-এর স্তরবিন্যাস (hierarchy) ব্যাখ্যা করতে; (২) dev, staging ও production-এর জন্য সাবস্ক্রিপশন আলাদা করার যুক্তি দিতে; (৩) Azure Policy, লক ও ট্যাগ দিয়ে গভর্ন্যান্স প্রয়োগ করতে; (৪) এন্টারপ্রাইজ ল্যান্ডিং জোনের ধারণা ব্যাখ্যা করতে; এবং (৫) TechNova-র জন্য একটি মাল্টি-সাবস্ক্রিপশন নকশা আঁকতে।

২.২ পূর্বশর্ত জ্ঞান

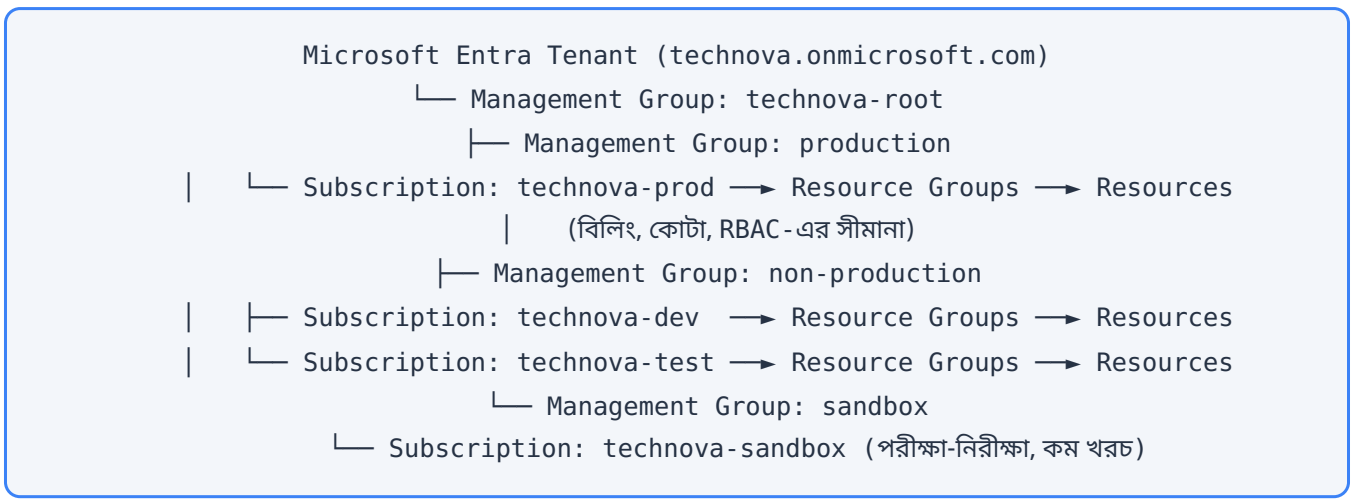
অধ্যায় ১-এর Region ও Availability Zone-এর ধারণা। Azure Portal-এ লগইন করার অভিজ্ঞতা থাকলে ভালো হয়, না থাকলেও চলবে — এই অধ্যায়েই অ্যাকাউন্টের কাঠামো শেখানো হবে।

২.৩ কেন এই বিষয়টি গুরুত্বপূর্ণ

একটি ক্লাউড অ্যাকাউন্ট শুধু বিল দেওয়ার মাধ্যম নয় — এটি নিরাপত্তা, খরচ ও দায়িত্বের সীমানা। যে কোম্পানি dev ও production একই সাবস্ক্রিপশনে চালায়, সেখানে একদিন কোনো ডেভেলপার ভুল করে প্রোডাকশন ডেটাবেস মুছে ফেলে। যে কোম্পানি root-সদৃশ অ্যাক্সেস সবাইকে দেয়, সেখানে একদিন একটি চুরি যাওয়া পাসওয়ার্ড পুরো কোম্পানির বিল হাজার ডলার বাড়িয়ে দেয়। গভর্ন্যান্স মানে এই সীমানাগুলো আগেই ঠিক করে রাখা — যাতে ভুলের সুযোগই না থাকে।

২.৪ প্রথম নীতি থেকে: Azure-এর স্তরবিন্যাস

Azure-এর সম্পদ (resource) সংগঠিত হয় চার স্তরে। সবচেয়ে উপরে **Microsoft Entra টেন্যান্ট (Tenant)** — এটি আপনার সংস্থার পরিচয়ের সীমানা (যেমন technova.onmicrosoft.com)। একটি টেন্যান্টের ভেতরে একাধিক **সাবস্ক্রিপশন (Subscription)** থাকতে পারে — প্রতিটি সাবস্ক্রিপশন একটি বিলিং ও অ্যাক্সেসের একক। সাবস্ক্রিপশনের ভেতরে থাকে **Resource Group** — একসাথে পরিচালিত রিসোর্সের গোষ্ঠী (যেমন technova-prod-rg)। আর সবশেষে **রিসোর্স** নিজে — VM, Storage Account, SQL Database ইত্যাদি।



Management Group হলো সাবস্ক্রিপশনগুলোকে গোষ্ঠীবদ্ধ করার স্তর। একটি Management Group-এ Azure Policy বা RBAC প্রয়োগ করলে তার অধীন সব সাবস্ক্রিপশনে তা কার্যকর হয়। এভাবে "production-এর সব সাবস্ক্রিপশনে শুধু নির্দিষ্ট Region-এ রিসোর্স তৈরি করা যাবে" — এমন নিয়ম এক জায়গা থেকে প্রয়োগ করা যায়।

২.৫ সাবস্ক্রিপশন আলাদা করা: কেন dev ও prod একসাথে নয়

- **ব্লাস্ট রেডিয়াস (Blast Radius):** dev-এ একটি ভুল স্ক্রিপ্ট চললে production-এর কিছু হবে না — কারণ সাবস্ক্রিপশন আলাদা, অ্যাক্সেসও আলাদা।
- **খরচের স্বচ্ছতা:** প্রতিটি সাবস্ক্রিপশনের আলাদা বিল হয় — কোন পরিবেশে কত খরচ হচ্ছে তা স্পষ্ট থাকে।
- **কোটা (Quota):** প্রতিটি সাবস্ক্রিপশনে আলাদা কোটা থাকে — dev-এর অতিরিক্ত VM production-এর কোটা খেয়ে ফেলতে পারে না।
- **নীতি (Policy):** production সাবস্ক্রিপশনে কঠোর নীতি (যেমন শুধু নির্দিষ্ট VM সাইজ), dev-এ শিথিল নীতি।

> **WARNING:** Azure-এ কোনো **root user** ধারণা নেই যেমন AWS-এ আছে — কিন্তু টেন্যান্টের **Global Administrator** প্রায় সমতুল্য ক্ষমতা রাখে। এই অ্যাকাউন্টে MFA বাধ্যতামূলক করুন, দৈনন্দিন কাজে এটি ব্যবহার করবেন না, এবং কমপক্ষে দুটি "break-glass" অ্যাডমিন অ্যাকাউন্ট (আলাদা ফোন নম্বরসহ) রাখুন — যাতে একজন অ্যাডমিনের ফোন হারালেও টেন্যান্টে ঢোকা যায়।

২.৬ Azure Policy, লক ও ট্যাগ

Azure Policy হলো নিয়ম প্রয়োগের ব্যবস্থা: "সব Storage Account-এ HTTPS-only থাকতে হবে", "শুধু Southeast Asia Region-এ রিসোর্স তৈরি করা যাবে", "সব রিসোর্সে CostCenter ট্যাগ থাকতে হবে"। Policy অমান্য করে রিসোর্স তৈরি করতে গেলে Azure তা ব্লক করে (Deny effect) অথবা অডিট লগে চিহ্নিত করে (Audit effect)। **Resource Lock** দুই ধরনের: ReadOnly (কোনো পরিবর্তন নয়) ও CanNotDelete (মোছা যাবে না) — প্রোডাকশন ডেটাবেসের Resource Group-এ CanNotDelete লক লাগানো একটি মৌলিক সুরক্ষা। **ট্যাগ (Tag)** হলো key-value লেবেল (যেমন Env=prod, CostCenter=platform) — খরচ ভাগ করা, মালিকানা চিহ্নিত করা ও অটোমেশনে ব্যবহৃত হয়।

```
az group create --name technova-prod-rg --location southeastasia --tags Env=prod
CostCenter=platform Owner=nahid
az lock create --name prod-rg-lock --lock-type CanNotDelete --resource-group technova-
prod-rg
az policy assignment create --name deny-non-sea --policy "/providers/
Microsoft.Authorization/policyDefinitions/e56962a6-4747-49cd-b67b-bf8b01975c4c" --params
'{ "listOfAllowedLocations": { "value": ["southeastasia"] } }' --scope "/subscriptions/
00000000-0000-0000-0000-000000000000"
```

তৃতীয় কমান্ডের Policy Definition ID-টি "Allowed locations" বিল্ট-ইন পলিসির — আপনার টেন্যান্টে `az policy definition list --query '[?displayName=='Allowed locations']'` দিয়ে সঠিক ID যাচাই করে নিন।

২.৭ এন্টারপ্রাইজ ল্যান্ডিং জোন (Landing Zone)

ল্যান্ডিং জোন হলো একটি প্রস্তুতকৃত ভিত্তি — Management Group কাঠামো, নেটওয়ার্কিং (hub-and-spoke), পরিচয় (Entra ID), লগিং ও নীতি আগে থেকে সাজানো থাকে, যাতে নতুন দল এসে নিরাপদভাবে কাজ শুরু করতে পারে।

Microsoft-এর **Cloud Adoption Framework (CAF)** ল্যান্ডিং জোনের নকশা দেয়। ছোট দলের জন্য পুরো CAF দরকার নেই — কিন্তু এর নীতিগুলো (সাবস্ক্রিপশন আলাদা করা, কেন্দ্রীয় লগিং, নীতি দিয়ে সুরক্ষা) ছোট থেকেই প্রয়োগ করা উচিত।

২.৮ সাধারণ ভুল ও ট্রাবলশুটিং

- **ভুল:** সব রিসোর্স একটি Resource Group-এ রাখা। Resource Group মুছলে তার সব রিসোর্স মুছে যায় — পরিবেশ অনুযায়ী আলাদা করুন।
- **ভুল:** সাবস্ক্রিপশন তৈরি করে বাজেট অ্যালার্ট না লাগানো। একটি ভুল কনফিগারেশন এক রাতেই বড় বিল তৈরি করতে পারে (অধ্যায় ৫)।
- **ট্রাবলশুটিং:** "Subscription not found" বা অ্যাক্সেস না পেলে প্রথমে `az account show` দিয়ে দেখুন আপনি কোন সাবস্ক্রিপশনে আছেন — ভুল সাবস্ক্রিপশন কনটেক্সট সবচেয়ে সাধারণ কারণ।

২.১২ সারসংক্ষেপ

Azure-এর কাঠামো: Entra টেন্যান্ট → Management Group → সাবস্ক্রিপশন → Resource Group → রিসোর্স। সাবস্ক্রিপশন হলো বিলিং, কোটা ও অ্যাক্সেসের সীমানা — dev, staging ও production আলাদা সাবস্ক্রিপশনে রাখুন। Management Group দিয়ে নীতি এক জায়গা থেকে প্রয়োগ করুন, Resource Group-এ লক লাগান, ট্যাগ দিয়ে খরচ ও মালিকানা চিহ্নিত করুন। ল্যান্ডিং জোন হলো এই সবকিছুর প্রস্তুতকৃত ভিত্তি।

২.১৩ পরিভাষা

Tenant — Microsoft Entra পরিচয়ের সীমানা — একটি সংস্থার Azure পরিচয়ের মূল একক।

Subscription — বিলিং, কোটা ও অ্যাক্সেসের একক; একটি টেন্যান্টে একাধিক সাবস্ক্রিপশন থাকতে পারে।

Management Group — সাবস্ক্রিপশন গোষ্ঠীবদ্ধ করার স্তর; নীতি ও RBAC এখান থেকে উত্তরাধিকারসূত্রে প্রয়োগ হয়।

Resource Group — একসাথে পরিচালিত ও একসাথে মোছা যায় এমন রিসোর্সের গোষ্ঠী।

Azure Policy — নিয়ম প্রয়োগের ব্যবস্থা — অমান্যকারী রিসোর্স তৈরি ব্লক বা অডিট করে।

Resource Lock — CanNotDelete বা ReadOnly লক — ভুল মোছা বা পরিবর্তন থেকে রক্ষা করে।

২.১৪ জ্ঞান যাচাই

1. টেন্যান্ট, সাবস্ক্রিপশন ও Resource Group-এর মধ্যে সম্পর্ক ব্যাখ্যা করুন।
2. dev ও production আলাদা সাবস্ক্রিপশনে রাখার তিনটি কারণ লিখুন।
3. Management Group কীভাবে নীতি প্রয়োগ সহজ করে?
4. CanNotDelete লক কখন ব্যবহার করবেন? একটি উদাহরণ দিন।

5. ল্যান্ডিং জোন কী এবং ছোট দলের জন্য এর কোন অংশগুলো সবচেয়ে দরকারি?

২.১৫ অনুশীলন

TechNova-র জন্য একটি সাবস্ক্রিপশন নকশা আঁকুন: prod, staging, dev ও sandbox — প্রতিটির Management Group, ট্যাগিং নিয়ম ও লকের পরিকল্পনাসহ। Azure Portal-এ একটি Resource Group তৈরি করে তাতে CanNotDelete লক লাগান, তারপর মুছতে চেষ্টা করে কী বার্তা আসে তা লিখে রাখুন।

অধ্যায় ৩ • PART I

Chapter 3 — Microsoft Entra ID and Identity Engineering

৩.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি পারবেন — (১) Microsoft Entra ID-এর ভূমিকা ও Azure RBAC-এর কাজ ব্যাখ্যা করতে; (২) ব্যবহারকারী, গ্রুপ, Service Principal ও Managed Identity-র পার্থক্য করতে; (৩) Role Assignment-এর scope (Management Group / Subscription / Resource Group / Resource) বুঝতে; (৪) least-privilege নীতিতে ভূমিকা বাছাই করতে; (৫) MFA, Conditional Access ও PIM-এর ধারণা ব্যাখ্যা করতে; এবং (৬) "Access denied" সমস্যার ধাপে ধাপে সমাধান করতে।

৩.২ পূর্বশর্ত জ্ঞান

অধ্যায় ২-এর সাবস্ক্রিপশন ও Resource Group কাঠামো। পরিচয় (identity) ও অনুমতি (permission) সম্পর্কে সাধারণ ধারণা থাকলেই চলবে।

৩.৩ কেন এই বিষয়টি গুরুত্বপূর্ণ

ক্লাউডে প্রায় প্রতিটি বড় নিরাপত্তা-ঘটনার মূলে থাকে পরিচয়: একটি চুরি যাওয়া পাসওয়ার্ড, একটি অতিরিক্ত ক্ষমতাসম্পন্ন Service Principal, একটি ভুলে খোলা Storage Account। Azure-এ নিরাপত্তার প্রথম লাইন হলো পরিচয় — কে, কী করতে পারবে, কোথায় করতে পারবে। এই অধ্যায়টি পুরো বইয়ের সবচেয়ে গুরুত্বপূর্ণ অধ্যায়গুলোর একটি: এখানকার নীতি না বুঝলে পরের প্রতিটি অধ্যায়ের নিরাপত্তা-অংশ অসম্পূর্ণ থাকবে।

৩.৪ প্রথম নীতি থেকে: পরিচয় কেন সবকিছুর কেন্দ্রে

অন-প্রিমিসেস যুগে নিরাপত্তার সীমানা ছিল নেটওয়ার্ক — ফায়ারওয়ালের ভেতরে থাকলেই বিশ্বাস করা হতো। ক্লাউডে সেই সীমানা ভেঙে গেছে: আপনার VM ইন্টারনেট থেকে অ্যাক্সেসযোগ্য হতে পারে, আপনার ডেভেলপার বাড়ি থেকে কাজ করে, আপনার CI/CD পাইপলাইন GitHub থেকে Azure-এ রিসোর্স তৈরি করে। তাই আধুনিক নিরাপত্তা মডেল হলো **Zero Trust** — "কখনোই অন্ধভাবে বিশ্বাস করো না, প্রতিবার যাচাই করো"। প্রতিটি অনুরোধে Azure যাচাই করে: এই পরিচয়টি কে, এটি প্রমাণিত (authenticated) কি না, এবং এই কাজের অনুমতি (authorized) আছে কি না। এই দুটি ধাপ — **Authentication** (আপনি কে) ও **Authorization** (আপনি কী করতে পারেন) — আলাদা জিনিস, এবং দুটোতেই ভুল হতে পারে।

Microsoft Entra ID (আগের নাম Azure Active Directory) হলো Azure-এর পরিচয় সেবা। এটি ব্যবহারকারী, গ্রুপ, অ্যাপ্লিকেশন ও ডিভাইসের পরিচয় সংরক্ষণ করে এবং টোকেন দেয়। Azure-এর প্রায় প্রতিটি সার্ভিস Entra ID-এর ওপর নির্ভর করে।

৩.৫ পরিচয়ের চারটি রূপ

পরিচয়ের রূপ	কী	কখন ব্যবহার করবেন
User		দৈনন্দিন কাজ, Portal ও CLI ব্যবহার

	একজন মানুষের অ্যাকাউন্ট (nahid@technova.onmicrosoft.com)	
Group	ব্যবহারকারীদের গোষ্ঠী	ভূমিকা একসাথে দেওয়ার জন্য — ব্যক্তিকে নয়, গ্রুপকে ভূমিকা দিন
Service Principal	অ্যাপ্লিকেশন/অটোমেশনের পরিচয় (অ্যাপ রেজিস্ট্রেশন)	CI/CD, বাইরের অ্যাপ — নিজের client secret থাকে
Managed Identity	Azure-স পরিচালিত পরিচয় — কোনো গোপন তথ্য সংরক্ষণ করতে হয় না	VM, App Service, Function থেকে Azure রিসোর্সে অ্যাক্সেস — সেরা পদ্ধতি

Managed Identity দুই ধরনের: **System-assigned** (রিসোর্সের সাথে জন্ম নেয় ও মরে — যেমন একটি VM-এর নিজস্ব পরিচয়) ও **User-assigned** (আলাদাভাবে তৈরি, একাধিক রিসোর্সে ব্যবহারযোগ্য)। নিয়মটি মনে রাখুন: যেখানে সম্ভব, Managed Identity ব্যবহার করুন — কারণ এতে কোনো পাসওয়ার্ড বা secret সংরক্ষণ করতে হয় না, Azure নিজেই টোকেন ঘোরায় (rotate করে)।

৩.৬ Azure RBAC: ভূমিকা, Scope ও Assignment

Azure RBAC (Role-Based Access Control) তিনটি উপাদানে কাজ করে: **Security Principal** (কে — ব্যবহারকারী/গ্রুপ/Service Principal/Managed Identity), **Role Definition** (কী করতে পারবে — যেমন Virtual Machine Contributor, Reader, Storage Blob Data Reader), এবং **Scope** (কোথায় — Management Group, Subscription, Resource Group বা একটি নির্দিষ্ট রিসোর্স)। এই তিনটির সমন্বয়কে বলে **Role Assignment**।

Scope-এর স্তরবিন্যাস (উপরের স্তরের অনুমতি নিচের সব স্তরে প্রযোজ্য):

Management Group ⟶ Subscription ⟶ Resource Group ⟶ Resource
(সবচেয়ে বিস্তৃত) (সবচেয়ে সংকীর্ণ)

উদাহরণ: `technova-prod-rg`-তে `"Reader"` ভূমিকা = শুধু সেই গ্রুপের রিসোর্স দেখা যাবে; `Subscription`-স্তরে `"Contributor"` = পুরো সাবস্ক্রিপশনে রিসোর্স তৈরি/পরিবর্তন করা যাবে।

কয়েকটি বিল্ট-ইন ভূমিকা মনে রাখুন: **Owner** (সবকিছু + অ্যাক্সেস দেওয়ার অধিকার), **Contributor** (সবকিছু তৈরি/পরিবর্তন/মোছা, কিন্তু অ্যাক্সেস দিতে পারে না), **Reader** (শুধু দেখা), **User Access Administrator** (শুধু অ্যাক্সেস পরিচালনা)। এছাড়া প্রতিটি সার্ভিসের নিজস্ব ডেটা-স্তরের ভূমিকা আছে — যেমন Storage Blob Data Reader: শুধু Blob পড়া যায়, Storage Account-এর সেটিং বদলানো যায় না। এই পার্থক্যটি গুরুত্বপূর্ণ: “Storage Account দেখতে পারা” আর “Blob-এর ভেতরের ডেটা পড়তে পারা” এক জিনিস নয়।

> **CONCEPT:** নীতি মূল্যায়নের নিয়ম: **Explicit Deny সবকিছুর ওপরে** — কোনো Deny Assignment বা Azure Policy (Deny) থাকলে Allow যতই থাকুক, কাজটি হবে না। তারপর Allow খোঁজা হয় — কোনো Allow না থাকলে স্বয়ংক্রিয়ভাবে অস্বীকৃত হয় (implicit deny)। মনে রাখুন: Azure RBAC-এ “Deny” ভূমিকা সাধারণ Role Assignment দিয়ে দেওয়া যায় না — Deny Assignment শুধু বিশেষ ক্ষেত্রে (যেমন Azure Blueprints) ব্যবহৃত হয়।

৩.৭ Least Privilege: কম ক্ষমতা, বেশি নিরাপত্তা

Least Privilege মানে: প্রত্যেককে শুধু তার কাজের জন্য যতটুকু দরকার, ঠিক ততটুকু ক্ষমতা দিন — তার বেশি নয়, এবং যতদিন দরকার ততদিনের জন্য। বাস্তবে এর অর্থ: (১) কাউকে Owner দেবেন না যদি Contributor-এ কাজ হয়; (২) ভূমিকা দিন গ্রুপকে, ব্যক্তিকে নয়; (৩) Scope যত সংকীর্ণ সম্ভব রাখুন — পুরো Subscription নয়, নির্দিষ্ট Resource Group; (৪) স্থায়ী অ্যাক্সেসের বদলে **Privileged Identity Management (PIM)** দিয়ে সময়সীমাবদ্ধ, অনুমোদন-সাপেক্ষ অ্যাক্সেস দিন — যেমন একজন ইঞ্জিনিয়ার শুধু ২ ঘণ্টার জন্য production-এ Contributor হতে পারবেন, এবং প্রতিটি অ্যাক্টিভেশন লগ হবে।

৩.৮ MFA ও Conditional Access

Multi-Factor Authentication (MFA) মানে পাসওয়ার্ডের পাশাপাশি দ্বিতীয় প্রমাণ — ফোনের অ্যাপে অনুমোদন, SMS কোড, বা FIDO2 নিরাপত্তা-কী। পাসওয়ার্ড চুরি গেলেও MFA থাকলে আক্রমণকারী ঢুকতে পারে না। **Conditional Access** হলো নীতিভিত্তিক নিয়ম: “অপরিচিত দেশ থেকে লগইন করলে MFA বাধ্যতামূলক”, “অনিরাপদ ডিভাইস থেকে production সাবস্ক্রিপশনে ঢোকা যাবে না”, “ঝুঁকিপূর্ণ সাইন-ইন ব্লক করো”। Entra ID-এর **Identity Protection** সাইন-ইনের ঝুঁকি (অস্বাভাবিক স্থান, পরিচিত আক্রমণের প্যাটার্ন) শনাক্ত করে।

```
# বর্তমান পরিচয় ও সাবস্ক্রিপশন দেখুন
az account show --query "{user:user.name, subscription:name, tenant:tenantId}" --output
table
# একটি গ্রুপকে Resource Group-স্তরে Reader ভূমিকা দিন (least privilege)
az role assignment create --assignee-object-id <GROUP_OBJECT_ID> --assignee-principal-
type Group \
  --role Reader --resource-group technova-prod-rg
# কার কী ভূমিকা আছে তা যাচাই করুন
az role assignment list --resource-group technova-prod-rg --output table
```

৩.৯ সাধারণ ভুল ও ট্রাবলশুটিং

- **ভুল:** Service Principal-এর client secret কোডে বা Git-এ লেখা। বদলে Managed Identity বা Workload Identity Federation ব্যবহার করুন (অধ্যায় ১৯)।
- **ভুল:** সবাইকে Owner দেওয়া। Owner অ্যাক্সেস দিতে পারে — একটি আপস হওয়া Owner অ্যাকাউন্ট পুরো সাবস্ক্রিপশন দখল করতে পারে।
- **ট্রাবলশুটিং — “AuthorizationFailed”:** ধাপ ১: `az account show` দিয়ে সঠিক সাবস্ক্রিপশনে আছেন কি না দেখুন। ধাপ ২: `az role assignment list --assignee <আপনার object ID>` দিয়ে ভূমিকা আছে কি না দেখুন। ধাপ ৩: Scope দেখুন — ভূমিকা হয়তো অন্য Resource Group-এ দেওয়া। ধাপ ৪: Entra ID-তে “Sign-in logs” দেখুন — Conditional Access কি ব্লক করছে? মনে রাখুন: নতুন Role Assignment কার্যকর হতে কয়েক মিনিট লাগতে পারে।

৩.১২ সারসংক্ষেপ

পরিচয়ই ক্লাউড নিরাপত্তার কেন্দ্র — Zero Trust মডেলে প্রতিটি অনুরোধ যাচাই করা হয়। Entra ID পরিচয় দেয়; Azure RBAC নির্ধারণ করে কে, কোন Scope-এ, কোন ভূমিকায় কী করতে পারবে। Managed Identity ব্যবহার করুন যেখানে সম্ভব — এতে কোনো secret সংরক্ষণ করতে হয় না। Least privilege প্রয়োগ করুন: গ্রুপকে ভূমিকা দিন, Scope সংকীর্ণ রাখুন, PIM দিয়ে সময়সীমাবদ্ধ অ্যাক্সেস দিন, এবং সব privileged অ্যাকাউন্টে MFA বাধ্যতামূলক করুন।

৩.১৩ পরিভাষা

Microsoft Entra ID — Azure-এর পরিচয় সেবা — ব্যবহারকারী, গ্রুপ, অ্যাপ ও ডিভাইসের পরিচয় পরিচালনা করে।

Authentication — আপনি কে — তা প্রমাণ করা (পাসওয়ার্ড, MFA, টোকেন)।

Authorization — আপনি কী করতে পারেন — তা নির্ধারণ করা (RBAC ভূমিকা)।

Service Principal — অ্যাপ্লিকেশন বা অটোমেশনের পরিচয় — নিজস্ব credential থাকে।

Managed Identity — Azure-পরিচালিত পরিচয় — কোনো secret সংরক্ষণ ছাড়াই Azure রিসোর্সে অ্যাক্সেস দেয়।

Role Assignment — Principal + Role Definition + Scope — এই তিনটির সমন্বয়।

PIM — Privileged Identity Management — সময়সীমাবদ্ধ, অনুমোদন-সাপেক্ষ privileged অ্যাক্সেস।

Conditional Access — শর্তভিত্তিক অ্যাক্সেস নীতি — স্থান, ডিভাইস, ঝুঁকি অনুযায়ী MFA বা ব্লক।

৩.১৪ জ্ঞান যাচাই

1. Authentication ও Authorization-এর পার্থক্য কী? একটি উদাহরণ দিন।
2. Managed Identity কেন Service Principal-এর client secret-এর চেয়ে নিরাপদ?
3. Role Assignment-এর তিনটি উপাদান কী কী? Scope সংকীর্ণ রাখা কেন গুরুত্বপূর্ণ?
4. Owner ও Contributor ভূমিকার মধ্যে পার্থক্য কী?
5. PIM কী সমস্যার সমাধান করে? একটি বাস্তব দৃশ্যকল্প লিখুন।

৩.১৫ অনুশীলন

একটি test Resource Group তৈরি করুন। একটি সহকর্মীর (বা নিজের দ্বিতীয়) অ্যাকাউন্টকে শুধু সেই গ্রুপে Reader ভূমিকা দিন। তারপর সেই অ্যাকাউন্ট দিয়ে (১) রিসোর্স দেখার চেষ্টা করুন, (২) একটি VM তৈরি করার চেষ্টা করুন — কী ঘটে তা লিখুন। তারপর ভূমিকাটি মুছে ফেলুন।

অধ্যায় ৪ • PART I

Chapter 4 — Azure CLI, PowerShell and SDK Automation

৪.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি পারবেন — (১) Azure CLI ও Azure PowerShell ইনস্টল, লগইন ও কনফিগার করতে; (২) সাবস্ক্রিপশন কনটেক্সট বদলাতে ও যাচাই করতে; (৩) Resource Group, VM ও Storage Account তৈরি/তালিকা/মোছার মৌলিক কমান্ড চালাতে; (৪) Service Principal ও Managed Identity দিয়ে নিরাপদ অটোমেশন সাজাতে; (৫) CI/CD-তে Workload Identity Federation-এর ধারণা ব্যাখ্যা করতে; এবং (৬) credential সংক্রান্ত সাধারণ ভুল এড়াতে।

৪.২ পূর্বশর্ত জ্ঞান

অধ্যায় ২ ও ৩-এর সাবস্ক্রিপশন ও RBAC ধারণা। টার্মিনাল/কমান্ড প্রম্পটে কাজের অভিজ্ঞতা এবং Bash বা PowerShell-এর মৌলিক জ্ঞান।

৪.৩ কেন এই বিষয়টি গুরুত্বপূর্ণ

Portal-এ ক্লিক করে আপনি একটি VM বানাতে পারেন — কিন্তু একশোটি VM, দশটি পরিবেশ, প্রতিদিনের ডিপ্লয়মেন্ট — এসব ক্লিক করে করা যায় না। পেশাদার DevOps কাজের ভিত্তি হলো কমান্ড-লাইন ও স্ক্রিপ্ট: একই কমান্ড বারবার, একইভাবে, ভুল ছাড়া চালানো যায়; স্ক্রিপ্ট Git-এ রাখা যায়; CI/CD পাইপলাইন থেকে চালানো যায়। এই অধ্যায়ে আপনি Azure-এর সাথে কথা বলার ভাষা শিখবেন।

৪.৪ প্রথম নীতি থেকে: CLI কীভাবে Azure-এর সাথে কথা বলে

আপনি যখন `az vm create` চালান, তখন Azure CLI আসলে একটি HTTPS অনুরোধ পাঠায় Azure Resource Manager (ARM)-এর REST API-তে। ARM অনুরোধটি যাচাই করে (আপনার টোকেন বৈধ কি না, আপনার ভূমিকা অনুমতি দেয় কি না, নীতি মানা হচ্ছে কি না), তারপর সংশ্লিষ্ট Resource Provider-কে (যেমন Microsoft.Compute) নির্দেশ দেয়। Portal-ও একই API ব্যবহার করে — Portal শুধু একটি গ্রাফিকাল মোড়ক। এই কারণে CLI-তে যা করা যায়, Portal-এও করা যায়, এবং উল্টোটাও সত্য। একজন ইঞ্জিনিয়ার হিসেবে এই বোঝাপড়াটি গুরুত্বপূর্ণ: আপনি যখন কোনো কমান্ড চালান, আপনি সরাসরি Azure-এর নিয়ন্ত্রণ-স্তরের (control plane) সাথে কথা বলছেন।

৪.৫ Azure CLI: ইনস্টল, লগইন, কনটেক্সট

```
# লগইন (ব্রাউজারে Entra ID প্রমাণীকরণ খুলবে)
az login
# কোন সাবস্ক্রিপশনে আছেন তা দেখুন
az account show --output table
# সব সাবস্ক্রিপশন তালিকা ও সক্রিয়টি বদলান
az account list --output table
az account set --subscription "technova-prod"
# বর্তমান পরিচয় যাচাই
az ad signed-in-user show --query "{name:displayName, upn:userPrincipalName}"
```

`az login` আপনার টোকেন স্থানীয়ভাবে সংরক্ষণ করে (সাধারণত `~/.azure`-এ)। এই ডিরেক্টরি কখনো অন্যের সাথে ভাগ করবেন না, এবং শেয়ার্ড মেশিনে কাজ শেষে `az logout` চালান। **সাবস্ক্রিপশন কনটেক্সট** সবচেয়ে সাধারণ ভুলের উৎস: ভুল সাবস্ক্রিপশনে রিসোর্স তৈরি করা। প্রতিটি গুরুত্বপূর্ণ কমান্ডের আগে `az account show` দিয়ে যাচাই করার অভ্যাস করুন — অথবা স্ক্রিপ্টের শুরুতে সাবস্ক্রিপশন ID যাচাই করে নিন।

৪.৬ Azure PowerShell: কখন, কেন

Azure PowerShell (Az মডিউল) একই ARM API ব্যবহার করে, কিন্তু PowerShell-এর অবজেক্ট-ভিত্তিক পাইপলাইন দেয় — Windows পরিবেশে বা জটিল স্ক্রিপ্টে এটি সুবিধাজনক। CLI-এর কমান্ডগুলো JSON টেক্সট দেয়; PowerShell অবজেক্ট দেয়, যা ফিল্টার ও রূপান্তরে শক্তিশালী। নিয়ম: Linux/macOS ও CI/CD-তে Azure CLI, Windows-কেন্দ্রিক অটোমেশনে PowerShell — দুটোই শিখে রাখুন, কারণ দলে দুটোই ব্যবহৃত হয়।

```
# PowerShell-এ লগইন ও সাবস্ক্রিপশন বাছাই
Connect-AzAccount
Get-AzSubscription | Format-Table Name, Id, State
Set-AzContext -Subscription "technova-prod"
# একটি Resource Group তৈরি
New-AzResourceGroup -Name technova-prod-rg -Location southeastasia -Tag @{Env="prod"}
```

৪.৭ Credential-এর স্তর ও নিরাপদ অটোমেশন

স্থানীয়ভাবে আপনি ইন্টারঅ্যাক্টিভ লগইন করেন। কিন্তু CI/CD পাইপলাইন বা স্ক্রিপ্টে ইন্টারঅ্যাক্টিভ লগইন সম্ভব নয় — সেখানে তিনটি পদ্ধতি আছে। (১) **Service Principal + client secret:** কাজ করে, কিন্তু secret সংরক্ষণ ও ঘোরানোর (rotation) দায়িত্ব আপনার। (২) **Managed Identity:** Azure-এর ভেতরে চলা কাজের জন্য সেরা — কোনো secret নেই। (৩) **Workload Identity Federation (OIDC):** GitHub Actions-এর মতো বাইরের প্ল্যাটফর্ম থেকে Azure-এ লগইন করার আধুনিক পদ্ধতি — কোনো দীর্ঘমেয়াদি secret সংরক্ষণ করতে হয় না; GitHub একটি স্বল্পমেয়াদি OIDC টোকেন দেয়, Azure তা যাচাই করে Entra ID টোকেন দেয়। অধ্যায় ১৯-এ আমরা GitHub Actions-এ এটি বাস্তবে সাজাব।

> **WARNING:** কখনোই অ্যাক্সেস কী, client secret বা connection string Git-এ, চ্যাটে বা স্ক্রিনশটে দেবেন না। একটি secret একবার প্রকাশ পেলেই তাকে আপস হয়েছে ধরে নিন — সাথে সাথে ঘোরান (rotate) বা বাতিল (revoke) করুন। Azure Key Vault-এ secret রাখুন এবং Managed Identity দিয়ে সেখান থেকে পড়ুন (অধ্যায় ২২)।

৪.৮ সাধারণ ভুল ও ট্রাবলশুটিং

- **ভুল:** স্ক্রিপ্টে `--subscription` না দিয়ে ভুল সাবস্ক্রিপশনে রিসোর্স তৈরি করা। স্ক্রিপ্টের শুরুতে `az account show --query id` যাচাই করুন।
- **ভুল:** `az login`-এর টোকেনের মেয়াদ শেষ হয়ে গেছে ভেবে বারবার লগইন করা — সাধারণত `az account get-access-token` দিয়ে যাচাই করুন।
- **ট্রাবলশুটিং — "Insufficient privileges":** আপনার ভূমিকা (অধ্যায় ৩) যাচাই করুন; অনেক কমান্ডের জন্য নির্দিষ্ট ভূমিকা লাগে (যেমন Role Assignment দিতে User Access Administrator)।
- **ট্রাবলশুটিং — "ResourceGroupNotFound":** Resource Group-এর নাম ও সাবস্ক্রিপশন কনটেক্সট দুটোই যাচাই করুন — নামের বানান ভুল সবচেয়ে সাধারণ কারণ।

৪.১২ সারসংক্ষেপ

Azure CLI ও PowerShell দুটোই ARM REST API-এর ক্লায়েন্ট — Portal-ও একই API ব্যবহার করে। দৈনন্দিন কাজে CLI, Windows-কেন্দ্রিক অটোমেশনে PowerShell। সবসময় সাবস্ক্রিপশন কনটেক্সট যাচাই করুন। অটোমেশনে Managed Identity প্রথম পছন্দ, বাইরের CI/CD-তে Workload Identity Federation — দীর্ঘমেয়াদি secret সংরক্ষণ শেষ উপায় হিসেবে রাখুন।

৪.১৩ পরিভাষা

Azure CLI (az) — ক্রস-প্ল্যাটফর্ম কমান্ড-লাইন টুল — ARM API-তে কমান্ড পাঠায়।

Azure PowerShell (Az) — PowerShell মডিউল — অবজেক্ট-ভিত্তিক Azure অটোমেশন।

ARM (Azure Resource Manager) — Azure-এর নিয়ন্ত্রণ স্তর — সব অনুরোধ যাচাই করে Resource Provider-কে পাঠায়।

Subscription Context — বর্তমানে কোন সাবস্ক্রিপশনে কমান্ড চলছে — তা নির্ধারণ করে।

Service Principal — অটোমেশনের পরিচয় — অ্যাপ রেজিস্ট্রেশনের মাধ্যমে তৈরি।

Workload Identity Federation — OIDC টোকেন দিয়ে বাইরের প্ল্যাটফর্ম (যেমন GitHub Actions) থেকে secret ছাড়া Azure-এ প্রমাণীকরণ।

৪.১৪ জ্ঞান যাচাই

1. Azure CLI আসলে Azure-এর সাথে কীভাবে যোগাযোগ করে? Portal-এর সাথে এর সম্পর্ক কী?
2. ভুল সাবস্ক্রিপশনে রিসোর্স তৈরি হওয়া কীভাবে প্রতিরোধ করবেন?
3. Managed Identity, Service Principal ও Workload Identity Federation-এর মধ্যে পার্থক্য কী?
4. Azure PowerShell কখন CLI-এর চেয়ে সুবিধাজনক?
5. একটি client secret Git-এ ভুলে প্রকাশ পেলে কী করতে হবে?

৪.১৫ অনুশীলন

Azure CLI ইনস্টল করে `az login` করুন, `az account list` দিয়ে সাবস্ক্রিপশন দেখুন, একটি test Resource Group তৈরি করুন, তারপর `az group delete` দিয়ে মুছে ফেলুন। একটি ছোট Bash স্ক্রিপ্ট লিখুন যা প্রথমে সাবস্ক্রিপশন ID যাচাই করে, তারপরই রিসোর্স তৈরি করে।

অধ্যায় ৫ • PART I

Chapter 5 — Azure Resource Manager, Policy and Well-Architected

৫.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি পারবেন — (১) ARM-এর ভূমিকা ও Resource Provider-এর ধারণা ব্যাখ্যা করতে; (২) ARM টেমপ্লেটের কাঠামো পড়তে ও লিখতে; (৩) Azure Policy ও RBAC-এর পার্থক্য ব্যাখ্যা করতে; (৪) Resource Lock ও ট্যাগ দিয়ে গভর্ন্যান্স প্রয়োগ করতে; (৫) Well-Architected Framework-এর পাঁচটি স্তম্ভ প্রয়োগ করতে; এবং (৬) Cost Management দিয়ে বিল বিশ্লেষণ ও বাজেট অ্যালার্ট সাজাতে।

৫.২ পূর্বশর্ত জ্ঞান

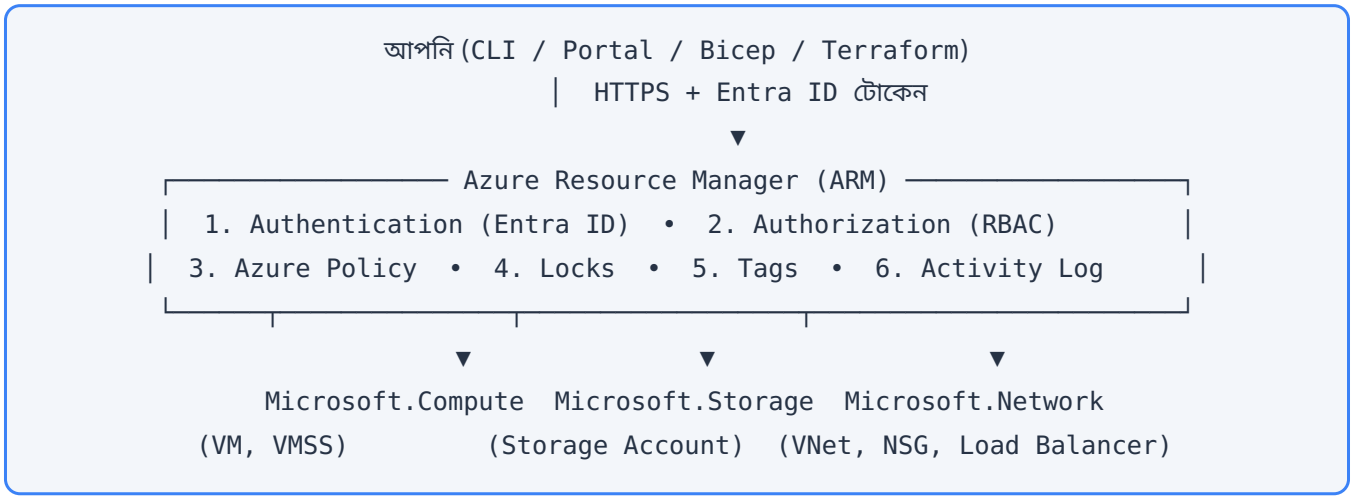
অধ্যায় ২–৪: সাবস্ক্রিপশন, RBAC, CLI। JSON-এর মৌলিক ধারণা থাকলে ARM টেমপ্লেট বুঝতে সুবিধা হবে।

৫.৩ কেন এই বিষয়টি গুরুত্বপূর্ণ

আপনি একটি VM তৈরি করলেন — কিন্তু সেটি কোথায় গেল, কে তা অনুমোদন দিল, কোন নীতি তা পরীক্ষা করল, বিলে তা কীভাবে দেখাবে? এই প্রশ্নগুলোর উত্তর দেয় Azure Resource Manager। আর “ভালো আর্কিটেকচার” মানে শুধু “চলে” নয় — নির্ভরযোগ্য, নিরাপদ, কার্যকর ও সাশ্রয়ী হওয়া। এই অধ্যায়ে আপনি Azure-এর নিয়ন্ত্রণ-স্তর ও ভালো নকশার মানদণ্ড শিখবেন।

৫.৪ প্রথম নীতি থেকে: ARM কী করে

Azure Resource Manager (ARM) হলো Azure-এর **নিয়ন্ত্রণ স্তর (control plane)** — আপনার প্রতিটি অনুরোধ (Portal, CLI, PowerShell, SDK — যেভাবেই আসুক) প্রথমে ARM-এর কাছে যায়। ARM পাঁচটি কাজ করে: (১) **প্রমাণীকরণ ও অনুমোদন** — Entra ID টোকেন যাচাই ও RBAC পরীক্ষা; (২) **নীতি প্রয়োগ** — Azure Policy অমান্য করলে অনুরোধ ব্লক; (৩) **রাউটিং** — অনুরোধটি সঠিক **Resource Provider**-এর কাছে পাঠানো (যেমন Microsoft.Compute VM-এর জন্য, Microsoft.Storage স্টোরেজের জন্য); (৪) **নির্ভরতা ব্যবস্থাপনা** — টেমপ্লেটে ঘোষিত ক্রমে রিসোর্স তৈরি; (৫) **অবস্থা সংরক্ষণ** — প্রতিটি রিসোর্সের অবস্থা ARM-এর ডাটাবেসে থাকে, যা Portal ও CLI পড়ে।



৫.৫ ARM টেমপ্লেট: অবকাঠামোকে কোড হিসেবে লেখা

ARM টেমপ্লেট হলো JSON ফাইল — এতে আপনি ঘোষণা (declare) করেন কোন রিসোর্স চান, ARM তা বাস্তবায়ন করে। এটি **declarative** মডেল: আপনি "কী চাই" বলেন, "কীভাবে করতে হবে" বলেন না। টেমপ্লেটের মূল অংশ: `$schema`, `parameters` (ইনপুট), `variables`, `resources` (রিসোর্সের তালিকা), `outputs`। Bicep হলো ARM টেমপ্লেটের জন্য একটি সহজ ভাষা — Bicep ফাইল কম্পাইল হয়ে ARM JSON হয় (অধ্যায় ২০-এ বিস্তারিত)।

```
{
  "$schema": "https://schema.management.azure.com/schemas/2019-04-01/
deploymentTemplate.json#",
  "contentVersion": "1.0.0.0",
  "parameters": { "location": { "type": "string", "defaultValue": "southeastasia" } },
  "resources": [
    {
      "type": "Microsoft.Storage/storageAccounts",
      "apiVersion": "2023-01-01",
      "name": "technovastore001",
      "location": "[parameters('location')]",
      "sku": { "name": "Standard_LRS" },
      "kind": "StorageV2",
      "properties": { "supportsHttpsTrafficOnly": true, "minimumTlsVersion": "TLS1_2" }
    }
  ]
}
```

লক্ষ্য করুন: `supportsHttpsTrafficOnly: true` ও `minimumTlsVersion: TLS1_2` — এগুলো নিরাপত্তার মৌলিক সেটিং, যা প্রতিটি Storage Account-এ থাকা উচিত। টেমপ্লেট দিয়ে ডিপ্লয় করলে এই সেটিং কখনো ভুলে যাওয়া হয় না — এটাই Infrastructure as Code-এর শক্তি।

৫.৬ Azure Policy বনাম RBAC: দুটি আলাদা প্রশ্ন

বিষয়	Azure RBAC	Azure Policy
প্রশ্ন	কে কী করতে পারবে?	কী অনুমোদিত?
উদাহরণ	নাহিদ VM তৈরি করতে পারবে	VM শুধু নির্দিষ্ট সাইজের হতে পারবে
প্রয়োগের সময়	অনুরোধের সময় (অনুমতি আছে কি না)	রিসোর্স তৈরির সময় (নিয়ম মানা হচ্ছে কি না)
ব্যর্থ হলে	AuthorizationFailed	Policy ব্লক করে / অডিট করে

দুটোই দরকার: RBAC ছাড়া যে কেউ সবকিছু করতে পারবে; Policy ছাড়া অনুমতিপ্রাপ্ত ব্যক্তিও ভুল (যেমন পাবলিক Blob, অনিরাপদ VM সাইজ) তৈরি করতে পারবে। Policy-র **Initiative** হলো একাধিক Policy-র গোষ্ঠী — যেমন "NIST SP 800-53" বা "Azure Security Benchmark" Initiative একসাথে অনেক নিয়ম প্রয়োগ করে।

৫.৭ Well-Architected Framework: ভালো নকশার পাঁচটি স্তম্ভ

- **Reliability (নির্ভরযোগ্যতা):** ব্যর্থতা থেকে পুনরুদ্ধার — মাল্টি-AZ, ব্যাকআপ, স্বাস্থ্য পরীক্ষা।
- **Security (নিরাপত্তা):** Zero Trust, least privilege, এনক্রিপশন, গোপন তথ্যের সুরক্ষা।

- **Cost Optimization (খরচ অপ্টিমাইজেশন):** শুধু প্রয়োজনীয় রিসোর্স, সঠিক সাইজ, অপ্রয়োজনীয় রিসোর্স বন্ধ করা।
- **Operational Excellence (পরিচালন উৎকর্ষ):** অটোমেশন, পর্যবেক্ষণ, রানবুক, পরিবর্তন ব্যবস্থাপনা।
- **Performance Efficiency (কার্যক্ষমতা):** সঠিক সার্ভিস বাছাই, স্কেলিং, ক্যাশিং।

এই পাঁচটি স্তম্ভের মধ্যে টানাপোড়েন (trade-off) আছে: সর্বোচ্চ নির্ভরযোগ্যতা (মাল্টি-Region) খরচ বাড়ায়; সর্বনিম্ন খরচ নির্ভরযোগ্যতা কমায়। একজন আর্কিটেক্টের কাজ হলো ব্যবসার প্রয়োজন অনুযায়ী ভারসাম্য খুঁজে বের করা — এবং সেই সিদ্ধান্ত **Architecture Decision Record (ADR)**-এ লিখে রাখা (অধ্যায় ২৪)।

৫.৮ খরচ ব্যবস্থাপনা: বিল যেন চমক না দেয়

Azure Cost Management + Billing দিয়ে দেখা যায় কোন সাবস্ক্রিপশন, Resource Group বা ট্যাগে কত খরচ হচ্ছে। **Cost Analysis**-এ দৈনিক খরচ, **Budgets**-এ সীমা ও অ্যালার্ট (যেমন মাসে ৫০ ডলার ছাড়ালে ইমেইল), এবং **Cost Alerts** দিয়ে অস্বাভাবিক বৃদ্ধি ধরা যায়। মনে রাখুন: কিছু খরচ "লুকানো" থাকে — NAT Gateway-এর প্রতি ঘণ্টার চার্জ, ডেটা ট্রান্সফার (egress), Log Analytics-এ লগ ইনজেশনের খরচ। ল্যাবে প্রতিটি রিসোর্সের খরচ-ঝুঁকি (Low/Medium/High) আগেই উল্লেখ করা আছে।

```
# একটি মাসিক বাজেট তৈরি করুন (50 USD সীমা, 80% ও 100%-এ অ্যালার্ট)
az consumption budget create --budget-name technova-monthly --amount 50 \
  --time-grain Monthly --start-date 2026-11-01 --end-date 2027-11-01 \
  --resource-group technova-prod-rg
# বর্তমান মাসের খরচ দেখুন
az consumption usage list --start-date 2026-11-01 --end-date 2026-11-09 --output table
```

> **WARNING:** Azure-এর ফ্রি অ্যাকাউন্টের ক্রেডিট ও সীমা সময়ের সাথে বদলায় — ল্যাব শুরুর আগে অফিসিয়াল পাতায় যাচাই করুন। কোনো ল্যাবে "Free Tier" লেখা থাকলেও বাজেট অ্যালার্ট লাগানো বাধ্যতামূলক — একটি ভুল কনফিগারেশন (যেমন ভুল Region-এ বড় VM) ক্রেডিট শেষ করে দিতে পারে।

৫.১২ সারসংক্ষেপ

ARM হলো Azure-এর নিয়ন্ত্রণ স্তর — প্রতিটি অনুরোধ প্রমাণীকরণ, RBAC, Policy, লক ও ট্যাগের মধ্য দিয়ে যায়, তারপর Resource Provider-এর কাছে পৌঁছায়। ARM টেমপ্লেট (ও Bicep) দিয়ে অবকাঠামোকে কোড হিসেবে ঘোষণা করা যায়। RBAC বলে "কে কী করতে পারবে", Policy বলে "কী অনুমোদিত" — দুটোই দরকার। Well-Architected Framework-এর পাঁচটি স্তম্ভ — নির্ভরযোগ্যতা, নিরাপত্তা, খরচ, পরিচালন উৎকর্ষ, কার্যক্ষমতা — ভালো নকশার মানদণ্ড। আর খরচ ব্যবস্থাপনা ছাড়া কোনো প্রোডাকশন আর্কিটেকচার সম্পূর্ণ নয়।

৫.১৩ পরিভাষা

ARM — Azure Resource Manager — Azure-এর নিয়ন্ত্রণ স্তর; সব অনুরোধ যাচাই ও রাউট করে।

Resource Provider — নির্দিষ্ট সার্ভিসের বাস্তবায়নকারী (যেমন Microsoft.Compute) — ARM অনুরোধ এদের কাছে পাঠায়।

ARM Template — JSON ফরম্যাটে অবকাঠামোর ঘোষণামূলক বর্ণনা।

Bicep — ARM টেমপ্লেট লেখার সহজ ভাষা — কম্পাইল হয়ে ARM JSON হয়।

Azure Policy — নিয়ম প্রয়োগের ব্যবস্থা — অমান্যকারী রিসোর্স ব্লক বা অডিট করে।

Initiative — একাধিক Policy-র গোষ্ঠী — যেমন নিরাপত্তা মানদণ্ডের সেট।

Well-Architected Framework — ভালো ক্লাউড নকশার পাঁচ স্তম্ভের নির্দেশিকা।

৫.১৪ জ্ঞান যাচাই

1. ARM-এর পাঁচটি কাজ কী কী? একটি অনুরোধের পথ ব্যাখ্যা করুন।
2. RBAC ও Azure Policy-এর মধ্যে পার্থক্য কী? একটি উদাহরণ দিন যেখানে দুটোই দরকার।
3. ARM টেমপ্লেট কেন declarative মডেল? এর সুবিধা কী?
4. Well-Architected Framework-এর পাঁচটি স্তম্ভ কী কী? দুটি স্তম্ভের মধ্যে একটি trade-off-এর উদাহরণ দিন।
5. বাজেট অ্যালার্ট কেন প্রতিটি সাবস্ক্রিপশনে বাধ্যতামূলক হওয়া উচিত?

৫.১৫ অনুশীলন

Azure Portal-এ Cost Management খুলে বর্তমান সাবস্ক্রিপশনের খরচ দেখুন (খরচ শূন্য হলেও ইন্টারফেসটি ঘুরে দেখুন)। একটি মাসিক বাজেট তৈরি করুন (৫০ ডলার, ৮০% ও ১০০%-এ অ্যালার্ট)। একটি test Storage Account-এ ARM টেমপ্লেট দিয়ে `supportsHttpsTrafficOnly: true` সেট করে ডিপ্লয় করুন।

PART II • DEVOPS ENGINEERING MASTERY

Networking & Compute Engineering

Virtual Network আর্কিটেকচার, নেটওয়ার্ক সিকিউরিটি ও প্রাইভেট কানেক্টিভিটি, DNS ও লোড ব্যালান্সিং, Virtual Machines, হাইব্রিড কানেক্টিভিটি ও স্কেলিং।

অধ্যায় ৬ • PART II

Chapter 6 — Azure Virtual Network Architecture

৬.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি পারবেন — (১) Virtual Network (VNet)-এর ভূমিকা ও Azure-এর সফটওয়্যার-সংজ্ঞায়িত নেটওয়ার্কিং ব্যাখ্যা করতে; (২) CIDR নোটেশন দিয়ে সাবনেট হিসাব করতে; (৩) পাবলিক ও প্রাইভেট সাবনেটের নকশা করতে; (৪) VNet Peering ও hub-and-spoke মডেল ব্যাখ্যা করতে; (৫) Azure DNS সেটিং কনফিগার করতে; এবং (৬) TechNova-র জন্য একটি মাল্টি-AZ VNet নকশা আঁকতে।

৬.২ পূর্বশর্ত জ্ঞান

অধ্যায় ১-এর Region/AZ ধারণা এবং IP ঠিকানা, সাবনেট মাস্ক, CIDR-এর মৌলিক জ্ঞান (Volume 2-এর নেটওয়ার্কিং)। Azure Portal-এ Resource Group তৈরি করার অভিজ্ঞতা থাকলে ভালো হয়।

৬.৩ কেন এই বিষয়টি গুরুত্বপূর্ণ

আপনার অ্যাপ্লিকেশনের প্রতিটি অংশ — VM, ডেটাবেস, কনটেইনার — কোনো না কোনো নেটওয়ার্কে থাকে। নেটওয়ার্ক ভুলভাবে সাজালে দুটি জিনিস ঘটে: হয় অ্যাপ্লিকেশনটি কাজ করে না (ভুল রাউটিং), নয়তো এটি অনিরাপদ হয় (ডেটাবেস ইন্টারনেট থেকে খোলা)। Azure-এ নেটওয়ার্ক হলো সফটওয়্যার-সংজ্ঞায়িত — কোনো ফিজিক্যাল কেবল লাগাতে হয় না, কিন্তু ভুল কনফিগারেশনের পরিণতি ঠিক ততটাই বাস্তব। এই অধ্যায়ে আপনি Azure নেটওয়ার্কিংয়ের ভিত্তি শিখবেন।

৬.৪ প্রথম নীতি থেকে: VNet কী এবং কেন দরকার

একটি **Virtual Network (VNet)** হলো Azure-এর ভেতরে আপনার নিজের বিচ্ছিন্ন নেটওয়ার্ক — একটি ভার্চুয়াল ডেটা সেন্টার নেটওয়ার্ক। VNet ছাড়া Azure রিসোর্সগুলো একে অপরের সাথে ব্যক্তিগতভাবে (privately) কথা বলতে পারে না; প্রতিটি রিসোর্সের জন্য আলাদা করে পাবলিক IP ও ফায়ারওয়াল নিয়ম লাগত। VNet সেই ব্যক্তিগত জায়গাটি দেয়: এর ভেতরের VM-গুলো ব্যক্তিগত IP-তে একে অপরের সাথে কথা বলে, ইন্টারনেট থেকে সরাসরি পৌঁছানো যায় না, এবং আপনি ঠিক করেন কোন ট্র্যাফিক কোথায় যাবে।

VNet তৈরি করার সময় আপনি একটি **ঠিকানা স্থান (address space)** দেন — যেমন 10.0.0.0/16। এই স্থানটি আপনার ব্যক্তিগত IP পরিসর। তার ভেতরে আপনি **সাবনেট (subnet)** ভাগ করেন — যেমন 10.0.1.0/24 ওয়েব সার্ভারের জন্য, 10.0.2.0/24 অ্যাপ সার্ভারের জন্য, 10.0.3.0/24 ডেটাবেসের জন্য। প্রতিটি সাবনেট একটি Availability Zone-এর সাথে যুক্ত হতে পারে, এবং প্রতিটি সাবনেটে আলাদা নিরাপত্তা নিয়ম (NSG) লাগানো যায়।

৬.৫ CIDR ও সাবনেট হিসাব

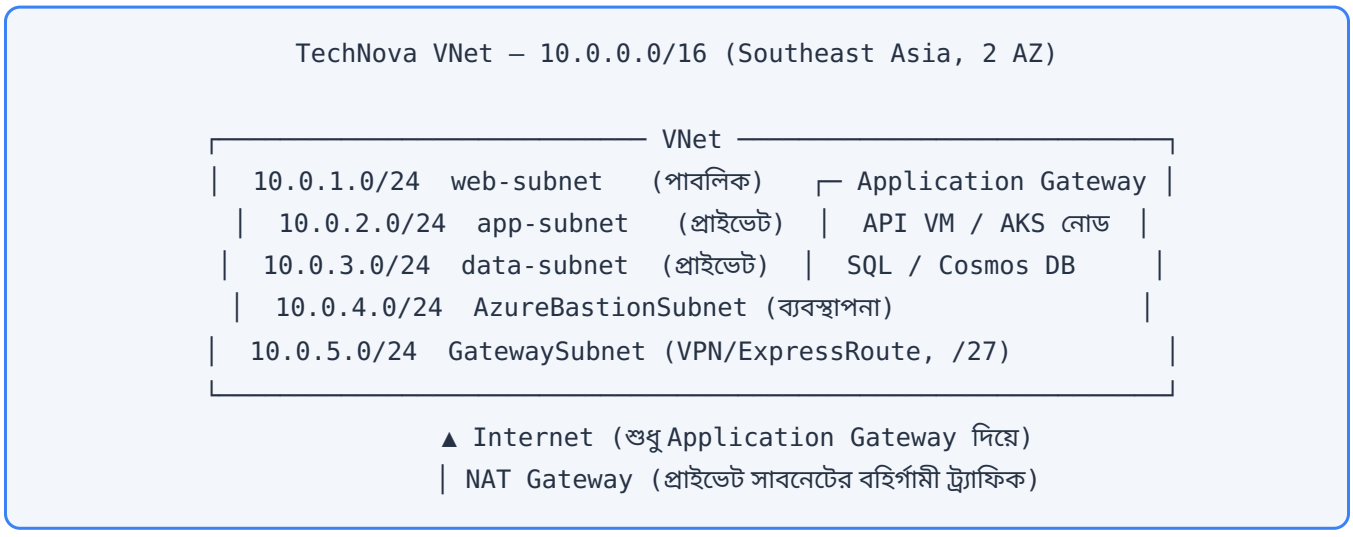
CIDR নোটেশন `10.0.0.0/16`-এর "/16" মানে প্রথম ১৬ বিট নেটওয়ার্ক অংশ, বাকি ১৬ বিট হোস্ট অংশ — মোট ৬৫,৫৩৬টি ঠিকানা। সাবনেট ভাগ করার সময় মনে রাখুন: Azure প্রতিটি সাবনেটে ৫টি ঠিকানা সংরক্ষণ করে (নেটওয়ার্ক ঠিকানা, ব্রডকাস্ট, গেটওয়ে, DNS-এর জন্য ২টি)। তাই /24 সাবনেটে ব্যবহারযোগ্য ঠিকানা ২৫৬ − ৫ = ২৫১টি। ছোট সাবনেট (যেমন /29) ভবিষ্যতে স্কেলিংয়ে সমস্যা তৈরি করে — VM Scale Sets বা AKS নোড বাড়াতে গেলে ঠিকানা ফুরিয়ে যাবে। নিয়ম: সাবনেট প্রয়োজনের চেয়ে বড় রাখুন; ঠিকানা সস্তা, পুনর্নকশা ব্যয়বহুল।

CIDR	মোট ঠিকানা	ব্যবহারযোগ্য (Azure)	সাধারণ ব্যবহার
/16	65,536	65,531	VNet-এর পুরো ঠিকানা স্থান
/24	256	251	একটি সাবনেট (ওয়েব/অ্যাপ/ডেটা স্তর)
/26	64	59	ছোট সাবনেট — যেমন Azure Bastion সাবনেট
/27	32	27	Gateway সাবনেট (VPN/ExpressRoute)

CONCEPT: VNet-এর ঠিকানা স্থান আপনার অন-প্রিমিসেস নেটওয়ার্ক বা অন্য VNet-এর সাথে **ওভারল্যাপ করা যাবে না** — ওভারল্যাপ থাকলে VNet Peering বা VPN সংযোগ কাজ করবে না। হাইব্রিড পরিকল্পনা থাকলে শুরুতেই একটি IP পরিকল্পনা (IPAM) ঠিক করুন: যেমন 10.0.0.0/16 Azure-এর জন্য, 10.1.0.0/16 অন-প্রিমিসেসের জন্য, 10.2.0.0/16 অন্য Region-এর জন্য।

৬.৬ পাবলিক ও প্রাইভেট সাবনেট

পাবলিক সাবনেট বলতে বোঝায় এমন সাবনেট যার রাউট টেবিলে ইন্টারনেট গেটওয়ের দিকে রাউট আছে — এর ভেতরের রিসোর্স পাবলিক IP পেতে পারে (যেমন Load Balancer, NAT Gateway)। **প্রাইভেট সাবনেট**-এ ইন্টারনেটের দিকে সরাসরি রাউট নেই — এর ভেতরের VM ও ডেটাবেস ইন্টারনেট থেকে পৌঁছানো যায় না; বাইরে যেতে হলে NAT Gateway দিয়ে যায়। TechNova-র নকশা: ওয়েব স্তর পাবলিক সাবনেটে (শুধু Load Balancer-এর পেছনে), অ্যাপ ও ডেটা স্তর প্রাইভেট সাবনেটে।



৬.৭ VNet Peering ও Hub-and-Spoke

VNet Peering দুটি VNet-কে ব্যক্তিগতভাবে সংযুক্ত করে — ট্র্যাফিক Azure-এর অভ্যন্তরীণ নেটওয়ার্ক দিয়ে যায়, ইন্টারনেট দিয়ে নয়, এবং কম ল্যাটেন্সি থাকে। Peering **non-transitive**: A peer B, B peer C — তাতে A থেকে C-তে যাওয়া যায় না। বড় সংস্থায় **hub-and-spoke** মডেল ব্যবহৃত হয়: একটি কেন্দ্রীয় hub VNet (ফায়ারওয়াল, VPN Gateway, কেন্দ্রীয় DNS), আর প্রতিটি দলের spoke VNet hub-এর সাথে peer করা। এতে নিরাপত্তা ও নেটওয়ার্ক নীতি এক জায়গায় থাকে।

৬.৮ Azure DNS সেটিং

VNet-এর ভেতরে নাম-রেজোলিউশনের জন্য Azure নিজস্ব DNS দেয় (168.63.129.16 — একটি ভার্চুয়াল ঠিকানা, যা প্রতিটি VNet-এ কাজ করে)। ব্যক্তিগত নামের জন্য **Private DNS Zone** ব্যবহার করুন — যেমন `privatelink.database.windows.net` বা আপনার নিজের `internal.technova.example`। Public DNS-এর জন্য Azure DNS (অধ্যায় ৮) আলাদা সার্ভিস। ভুল DNS সেটিংয়ের লক্ষণ: VM ইন্টারনেটে যেতে পারে কিন্তু অন্য VM-এর নামে পৌঁছাতে পারে না।

```
# VNet ও তিনটি সাবনেট তৈরি
az network vnet create --name technova-vnet --resource-group technova-prod-rg \
  --location southeastasia --address-prefixes 10.0.0.0/16
az network vnet subnet create --name web-subnet --vnet-name technova-vnet \
  --resource-group technova-prod-rg --address-prefixes 10.0.1.0/24
az network vnet subnet create --name app-subnet --vnet-name technova-vnet \
  --resource-group technova-prod-rg --address-prefixes 10.0.2.0/24
az network vnet subnet create --name data-subnet --vnet-name technova-vnet \
  --resource-group technova-prod-rg --address-prefixes 10.0.3.0/24
# VNet যাচাই
az network vnet show --name technova-vnet --resource-group technova-prod-rg --query
"{addressSpace:addressSpace.addressPrefixes, subnets:subnets[].name}"
```

৬.৯ সাধারণ ভুল ও ট্রাবলশুটিং

- **ভুল:** VNet তৈরি করার পর ঠিকানা স্থান বদলাতে চাওয়া — ঠিকানা স্থান বাড়ানো যায়, কিন্তু কমানো বা ওভারল্যাপ করা যায় না। শুরুতেই পরিকল্পনা করুন।
- **ভুল:** সব রিসোর্স একটি সাবনেটে রাখা — সাবনেট-স্তরে NSG লাগানো যায় না ঠিকমতো, এবং blast radius বড় হয়।
- **ট্রাবলশুটিং:** দুটি VM একে অপরকে ping করতে না পারলে প্রথমে দেখুন তারা একই VNet-এ কি না, তারপর NSG নিয়ম, তারপর `az network watcher test-connectivity` (Network Watcher) দিয়ে পথ পরীক্ষা করুন।

৬.১২ সারসংক্ষেপ

VNet হলো Azure-এর ভেতরে আপনার ব্যক্তিগত নেটওয়ার্ক — ঠিকানা স্থান (CIDR) দিয়ে সংজ্ঞায়িত, সাবনেটে ভাগ করা। পাবলিক সাবনেটে শুধু ইন্টারনেট-মুখী উপাদান, প্রাইভেট সাবনেটে অ্যাপ ও ডেটা। VNet Peering দিয়ে VNet সংযুক্ত করুন (non-transitive মনে রাখবেন), বড় সংস্থায় hub-and-spoke মডেল ব্যবহার করুন। ঠিকানা স্থান ওভারল্যাপ করা যাবে না — শুরুতেই IP পরিকল্পনা করুন।

৬.১৩ পরিভাষা

VNet — Virtual Network — Azure-এর ভেতরে বিচ্ছিন্ন ব্যক্তিগত নেটওয়ার্ক।

Address Space — VNet-এর ব্যক্তিগত IP পরিসর, CIDR নোটেশনে (যেমন 10.0.0.0/16)।

Subnet — VNet-এর ভেতরে ভাগ করা IP পরিসর — নিরাপত্তা ও রাউটিংয়ের একক।

VNet Peering — দুটি VNet-এর ব্যক্তিগত সংযোগ — Azure backbone দিয়ে, ইন্টারনেট ছাড়া।

Hub-and-Spoke — কেন্দ্রীয় hub VNet + একাধিক spoke VNet-এর নেটওয়ার্ক নকশা।

Private DNS Zone — VNet-এর ভেতরে ব্যক্তিগত নাম রেজোলিউশনের জন্য DNS জোন।

৬.১৪ জ্ঞান যাচাই

1. VNet কেন দরকার? VNet ছাড়া কী সমস্যা হতো?
2. 10.0.0.0/16 ঠিকানা স্থানে কয়টি ঠিকানা আছে? একটি /24 সাবনেটে ব্যবহারযোগ্য ঠিকানা কয়টি এবং কেন ৫টি কম?
3. পাবলিক ও প্রাইভেট সাবনেটের পার্থক্য কী? ডেটাবেস কোন সাবনেটে রাখবেন এবং কেন?
4. VNet Peering non-transitive মানে কী? একটি উদাহরণ দিন।
5. Hub-and-spoke মডেলের সুবিধা কী?

৬.১৫ অনুশীলন

TechNova-র জন্য একটি VNet নকশা করুন: 10.0.0.0/16, web/app/data/Bastion/Gateway সাবনেটসহ, প্রতিটি সাবনেটের CIDR ও উদ্দেশ্য লিখে। Azure CLI দিয়ে VNet ও তিনটি সাবনেট তৈরি করুন, তারপর `az network vnet show` দিয়ে যাচাই করুন।

অধ্যায় ৭ • PART II

Chapter 7 — Network Security and Private Connectivity

৭.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি পারবেন — (১) Network Security Group (NSG) ও Application Security Group (ASG)-এর নিয়ম লিখতে; (২) NSG ও Route Table-এর মূল্যায়ন ক্রম ব্যাখ্যা করতে; (৩) NAT Gateway দিয়ে প্রাইভেট সাবনেটের বহির্গামী ট্র্যাফিক সাজাতে; (৪) Private Endpoint ও Private Link দিয়ে PaaS সার্ভিসে ব্যক্তিগত সংযোগ করতে; (৫) Service Endpoint ও Private Link-এর পার্থক্য করতে; এবং (৬) নেটওয়ার্ক সমস্যার ধাপে ধাপে সমাধান করতে।

৭.২ পূর্বশর্ত জ্ঞান

অধ্যায় ৬-এর VNet ও সাবনেট। TCP/UDP, পোর্ট নম্বর ও ফায়ারওয়ালের মৌলিক ধারণা।

৭.৩ কেন এই বিষয়টি গুরুত্বপূর্ণ

নেটওয়ার্ক নিরাপত্তা মানে “কে কার সাথে কথা বলতে পারবে” — এই প্রশ্নের উত্তর দেওয়া। একটি ভুল NSG নিয়ম হয় অ্যাপ্লিকেশন বন্ধ করে দেয়, নয়তো ডেটাবেসকে পুরো ইন্টারনেটের সামনে খুলে দেয়। এই অধ্যায়ে আপনি Azure-এর নেটওয়ার্ক নিরাপত্তার সরঞ্জামগুলো শিখবেন — এবং সবচেয়ে গুরুত্বপূর্ণভাবে, PaaS সার্ভিস (SQL, Storage) কীভাবে ব্যক্তিগত নেটওয়ার্কের ভেতর থেকে ব্যবহার করা যায়।

৭.৪ প্রথম নীতি থেকে: NSG কীভাবে ট্র্যাফিক ফিল্টার করে

Network Security Group (NSG) হলো একটি ফায়ারওয়াল নিয়মের সেট — সাবনেটে বা একটি নেটওয়ার্ক ইন্টারফেসে (NIC) লাগানো যায়। প্রতিটি নিয়মে থাকে: অগ্রাধিকার (priority, ছোট সংখ্যা = আগে মূল্যায়ন), উৎস/গন্তব্য (IP, সার্ভিস ট্যাগ, ASG), প্রোটোকল, পোর্ট, এবং Allow বা Deny। Azure নিয়মগুলো অগ্রাধিকার ক্রমে পরীক্ষা করে — প্রথম মিলে যাওয়া নিয়মেই সিদ্ধান্ত হয়; কোনো নিয়ম না মিললে ডিফল্ট Deny হয়।

গুরুত্বপূর্ণ: NSG **stateful** — একটি বহির্গামী সংযোগ অনুমোদিত হলে তার উত্তর-ট্র্যাফিক স্বয়ংক্রিয়ভাবে অনুমোদিত হয়, আলাদা inbound নিয়ম লাগে না। **Application Security Group (ASG)** দিয়ে VM-গুলোকে ভূমিকা অনুযায়ী গোষ্ঠীবদ্ধ করা যায় (যেমন web-asg, app-asg) — নিয়মে IP-এর বদলে ASG-এর নাম লেখা যায়, ফলে VM-এর IP বদলালেও নিয়ম ভাঙে না।

উৎস	গন্তব্য	পোর্ট	কাজ
Internet	web-asg	443	HTTPS ট্র্যাফিক শুধু ওয়েব স্তরে
web-asg	app-asg	8080	ওয়েব থেকে অ্যাপ API-তে
app-asg	data-subnet	1433	অ্যাপ থেকে SQL-এ
Internet	data-subnet	সব	Deny — ডেটা স্তর কখনো ইন্টারনেট থেকে নয়

৭.৫ Route Table ও User-Defined Route (UDR)

ডিফল্টভাবে Azure জানে: একই VNet-এর ভেতরে ট্র্যাফিক যাবে, ইন্টারনেটে যাবে Internet Gateway দিয়ে। **Route Table** দিয়ে এই ডিফল্ট বদলানো যায় — যেমন "0.0.0.0/0 (সব ইন্টারনেট ট্র্যাফিক) যাবে Azure Firewall-এর দিকে" — এটাই hub-and-spoke-এ কেন্দ্রীয় পরিদর্শনের (inspection) উপায়। মনে রাখুন: Route Table সাবনেটে লাগানো হয়, এবং একটি সাবনেটে একটি Route Table থাকে।

৭.৬ NAT Gateway: প্রাইভেট সাবনেটের বহির্গামী পথ

প্রাইভেট সাবনেটের VM-কে প্যাচ ডাউনলোড বা বাইরের API-তে যেতে হলে বহির্গামী (outbound) ইন্টারনেট লাগে — কিন্তু inbound খোলা যাবে না। **NAT Gateway** সেই কাজটি করে: এটি সাবনেটের সাথে যুক্ত হয়, একটি স্থির পাবলিক IP দেয়, এবং শুধু বহির্গামী সংযোগ অনুমোদন করে। NAT Gateway-এর প্রতি ঘণ্টার চার্জ ও প্রক্রিয়াকৃত ডেটার চার্জ আছে — ল্যাবে খরচ-ঝুঁকি দেখুন। বিকল্প: Azure Firewall (বেশি নিয়ন্ত্রণ, বেশি খরচ) বা HTTP প্রক্সি।

৭.৭ Private Endpoint ও Private Link: PaaS-কে ব্যক্তিগত করা

Azure SQL বা Storage Account ডিফল্টভাবে পাবলিক এন্ডপয়েন্টে থাকে — ইন্টারনেট থেকে পৌঁছানো যায় (ফায়ারওয়াল নিয়ম দিয়ে সীমাবদ্ধ করা যায়, কিন্তু তবু পাবলিক)। **Private Endpoint** সেই সার্ভিসের জন্য আপনার VNet-এর ভেতরে একটি ব্যক্তিগত IP তৈরি করে — যেমন SQL সার্ভার এখন 10.0.3.10-এ পৌঁছানো যায়, এবং পাবলিক অ্যাক্সেস পুরোপুরি বন্ধ করা যায়। **Private Link** হলো এর পেছনের প্রযুক্তি। এর সাথে **Private DNS Zone** লাগে — যাতে `technova-sql.database.windows.net` নামটি ব্যক্তিগত IP-তে রেজোলভ হয়।

> **CONCEPT: Service Endpoint** বনাম **Private Endpoint**: Service Endpoint VNet-এর পরিচয় Azure সার্ভিসের ফায়ারওয়ালে যোগ করে — ট্র্যাফিক তবু Azure-এর পাবলিক নেটওয়ার্ক দিয়ে যায়। Private Endpoint সম্পূর্ণ ব্যক্তিগত IP দেয় — ট্র্যাফিক কখনো ইন্টারনেটে যায় না। নতুন নকশায় Private Endpoint-ই সুপারিশ করা হয়, বিশেষ করে সংবেদনশীল ডেটার জন্য।

```
# NSG তৈরি ও নিয়ম যোগ: শুধু 443 পোর্টে HTTPS অনুমোদন
az network nsg create --name web-nsg --resource-group technova-prod-rg --location
southeastasia
az network nsg rule create --name allow-https --nsg-name web-nsg --resource-group
technova-prod-rg \
  --priority 100 --source-address-prefixes Internet --destination-port-ranges 443 \
  --protocol Tcp --access Allow --direction Inbound
# NSG সাবনেটে লাগান
az network vnet subnet update --name web-subnet --vnet-name technova-vnet \
  --resource-group technova-prod-rg --network-security-group web-nsg
# NAT Gateway তৈরি ও app সাবনেটে যুক্ত করুন
az network public-ip create --name technova-nat-ip --resource-group technova-prod-rg --
sku Standard --allocation-method Static
az network nat gateway create --name technova-nat --resource-group technova-prod-rg --
location southeastasia --public-ip-addresses technova-nat-ip
```

৭.৮ সাধারণ ভুল ও ট্রাবলশুটিং

- **ভুল:** NSG-তে 0.0.0.0/0 থেকে সব পোর্ট খোলা রাখা — বিশেষ করে 22 (SSH) ও 3389 (RDP)। SSH-এর জন্য Azure Bastion ব্যবহার করুন (অধ্যায় ৯)।
- **ভুল:** Private Endpoint তৈরি করে Private DNS Zone যুক্ত না করা — নামটি তখন পাবলিক IP-তে রেজোলভ হয়, সংযোগ ব্যর্থ হয়।
- **ট্রাবলশুটিং:** সংযোগ ব্যর্থ হলে ধাপ: (১) NSG নিয়ম ও অগ্রাধিকার দেখুন, (২) Route Table দেখুন — ট্র্যাফিক কোথায় যাচ্ছে, (৩) `az network watcher test-connectivity` দিয়ে পথ পরীক্ষা করুন, (৪) NSG Flow Logs চালু করে দেখুন কোন নিয়ম ট্র্যাফিক ব্লক করছে।

৭.১২ সারসংক্ষেপ

NSG হলো Azure-এর ফায়ারওয়াল — অগ্রাধিকার ক্রমে মূল্যায়িত, stateful নিয়ম। ASG দিয়ে ভূমিকা অনুযায়ী নিয়ম লিখুন, IP দিয়ে নয়। Route Table দিয়ে ট্র্যাফিকের পথ নিয়ন্ত্রণ করুন। NAT Gateway প্রাইভেট সাবনেটকে নিরাপদ বহির্গামী পথ দেয়। আর PaaS সার্ভিসকে সত্যিকারের ব্যক্তিগত করতে Private Endpoint + Private DNS Zone ব্যবহার করুন — Service Endpoint-এর চেয়ে এটিই আধুনিক সুপারিশ।

৭.১৩ পরিভাষা

NSG — Network Security Group — সাবনেট বা NIC-স্তরের ফায়ারওয়াল নিয়মের সেট।

ASG — Application Security Group — ভূমিকা অনুযায়ী VM-এর গোষ্ঠী; নিয়মে IP-এর বদলে নাম ব্যবহৃত হয়।

UDR — User-Defined Route — Route Table-এ নিজের রাউট নিয়ম।

NAT Gateway — প্রাইভেট সাবনেটের জন্য বহির্গামী-শুধু ইন্টারনেট গেটওয়ে; স্থির পাবলিক IP দেয়।

Private Endpoint — PaaS সার্ভিসের জন্য VNet-এর ভেতরে ব্যক্তিগত IP — পাবলিক অ্যাক্সেস বন্ধ করা যায়।

Private Link — Private Endpoint-এর পেছনের প্রযুক্তি — ব্যক্তিগত সংযোগের ব্যবস্থা।

৭.১৪ জ্ঞান যাচাই

1. NSG stateful হওয়ার অর্থ কী? একটি উদাহরণ দিন।
2. ASG ব্যবহার করলে IP-ভিত্তিক নিয়মের চেয়ে কী সুবিধা হয়?
3. NAT Gateway কখন দরকার? এটি inbound ট্র্যাফিক অনুমোদন করে কি?
4. Service Endpoint ও Private Endpoint-এর পার্থক্য কী? কোনটি বেশি নিরাপদ এবং কেন?
5. Private Endpoint তৈরি করেও সংযোগ ব্যর্থ হলে প্রথমে কী যাচাই করবেন?

৭.১৫ অনুশীলন

একটি NSG তৈরি করুন: inbound-এ শুধু 443 (Internet থেকে) ও 22 (শুধু আপনার IP থেকে) অনুমোদন করুন, বাকি সব Deny। NSG Flow Logs চালু করার পদ্ধতি পড়ুন। একটি Storage Account-এ Private Endpoint তৈরি করে দেখুন নামটি ব্যক্তিগত IP-তে রেজোলভ হয় কি না (nslookup দিয়ে)।

অধ্যায় ৮ • PART II

Chapter 8 — Azure DNS and Load Balancing

৮.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি পারবেন — (১) Azure DNS-এ হোস্টেড জোন ও রেকর্ড পরিচালনা করতে; (২) Azure Load Balancer, Application Gateway ও Front Door-এর মধ্যে সঠিকটি বাছাই করতে; (৩) Health Probe ও TLS Termination ব্যাখ্যা করতে; (৪) পাথ-ভিত্তিক ও হোস্ট-ভিত্তিক রাউটিং সাজাতে; (৫) WAF-এর ধারণা ব্যাখ্যা করতে; এবং (৬) অসুস্থ (unhealthy) টার্গেটের সমস্যার সমাধান করতে।

৮.২ পূর্বশর্ত জ্ঞান

অধ্যায় ৬–৭: VNet, NSG। DNS (A রেকর্ড, CNAME) ও HTTP/HTTPS-এর মৌলিক ধারণা।

৮.৩ কেন এই বিষয়টি গুরুত্বপূর্ণ

ব্যবহারকারী আপনার সাইটে আসে একটি নামে — technova.example — কিন্তু সার্ভার চলে IP-তে। আর একটি সার্ভার নয়, অনেক সার্ভার — ট্র্যাফিক ভাগ করে দিতে হয়, একটি সার্ভার নষ্ট হলে অন্যটিতে পাঠাতে হয়, HTTPS শেষ করতে হয়। এই অধ্যায়ে আপনি Azure-এর ট্র্যাফিক ব্যবস্থাপনার তিনটি স্তর শিখবেন: DNS, লোড ব্যালান্সিং, এবং এজ ডেলিভারি।

৮.৪ প্রথম নীতি থেকে: নাম থেকে সার্ভার পর্যন্ত পথ

একটি অনুরোধের যাত্রা: ব্যবহারকারী ব্রাউজারে `https://app.technova.example` লেখে → DNS রেজোলিউশন নামটিকে IP-তে বদলায় → ট্র্যাফিক Azure Front Door বা Application Gateway-এ পৌঁছায় → সেটি একটি সুস্থ ব্যাকএন্ড VM বা কনটেইনারে পাঠায় → উত্তর ফেরত যায়। প্রতিটি ধাপে ব্যর্থতা হতে পারে — DNS ভুল, লোড ব্যালান্সার অসুস্থ টার্গেটে পাঠাচ্ছে, সার্টিফিকেটের মেয়াদ শেষ। এই অধ্যায়ে প্রতিটি ধাপ আলাদাভাবে শেখা হবে।

৮.৫ Azure DNS: হোস্টেড জোন ও রেকর্ড

Azure DNS হলো Microsoft-এর পরিচালিত DNS সেবা। একটি **হোস্টেড জোন** (যেমন technova.example) তৈরি করে আপনি A, AAAA, CNAME, MX, TXT রেকর্ড পরিচালনা করেন। ডোমেইন রেজিস্ট্রারের কাছে নেমসার্ভার হিসেবে Azure-এর ৪টি নেমসার্ভার দিলে Azure DNS কর্তৃত্বপূর্ণ (authoritative) হয়ে যায়। **Private DNS Zone** একই ধারণা, কিন্তু শুধু VNet-এর ভেতরে কাজ করে (অধ্যায় ৭)।

৮.৬ তিনটি লোড ব্যালান্সার: কোনটি কখন

সার্ভিস	স্তর	কী করে	কখন ব্যবহার করবেন
Azure Load Balancer	Layer 4 (TCP/UDP)	দ্রুত, সস্তা, VM-স্তরে ট্র্যাফিক ভাগ করে	অভ্যন্তরীণ বা পাবলিক TCP/UDP লোড ব্যালান্সিং
Application Gateway	Layer 7 (HTTP/HTTPS)	পাথ/হোস্ট রাউটিং, TLS শেষ করা, WAF	ওয়েব অ্যাপ — পাথ-ভিত্তিক রাউটিং দরকার হলে

Azure Front Door	Layer 7, গ্লোবাল	বিশ্বজুড়ে রাউটিং, ক্যাশিং, WAF	মাল্টি-Region অ্যাপ, গ্লোবাল ব্যবহারকারী

সহজ নিয়ম: একটি Region-এর ভেতরে ওয়েব অ্যাপের জন্য **Application Gateway**; শুধু TCP/UDP (যেমন ডেটাবেস ক্লাস্টার) বা অভ্যন্তরীণ সার্ভিসের জন্য **Load Balancer**; বিশ্বজুড়ে ব্যবহারকারী ও মাল্টি-Region-এর জন্য **Front Door**। Front Door ও Application Gateway একসাথে ব্যবহার করা যায় — Front Door গ্লোবাল রাউটিং করে, Application Gateway Region-স্তরে রাউটিং করে।

৮.৭ Health Probe ও TLS Termination

Health Probe হলো লোড ব্যালান্সারের নিয়মিত স্বাস্থ্য পরীক্ষা — যেমন প্রতি ৩০ সেকেন্ডে `/health` এন্ডপয়েন্টে HTTP GET; ২০০ উত্তর না এলে ব্যাকএন্ডকে অসুস্থ ধরা হয় এবং ট্র্যাফিক পাঠানো বন্ধ হয়। Probe-এর পথটি আসল অ্যাপের স্বাস্থ্য প্রতিফলিত করতে হবে — শুধু "সার্ভার চলছে" নয়, "ডেটাবেস সংযোগ আছে" কি না তাও যাচাই করা উচিত। **TLS Termination** মানে HTTPS সংযোগটি লোড ব্যালান্সারেই শেষ করা (সার্টিফিকেট সেখানে থাকে), আর ব্যাকএন্ডে HTTP-তে যাওয়া — এতে ব্যাকএন্ডের ওপর চাপ কমে। তবে সংবেদনশীল ডেটার জন্য **end-to-end TLS** (ব্যাকএন্ডেও HTTPS) বিবেচনা করুন।

৮.৮ WAF: ওয়েব অ্যাপ্লিকেশন ফায়ারওয়াল

Web Application Firewall (WAF) Application Gateway বা Front Door-এর সাথে যুক্ত হয়ে সাধারণ ওয়েব আক্রমণ (SQL Injection, Cross-Site Scripting, OWASP Top 10) ব্লক করে। WAF-এর দুটি মোড: **Detection** (শুধু লগ করে) ও **Prevention** (ব্লক করে)। নতুন নিয়ম প্রথমে Detection মোডে চালিয়ে দেখুন — ভুল নিয়মে বৈধ ব্যবহারকারী ব্লক হতে পারে।

```
# পাবলিক Load Balancer তৈরি (Standard SKU)
az network lb create --name technova-lb --resource-group technova-prod-rg \
  --location southeastasia --sku Standard --frontend-ip-name technova-lb-fe \
  --backend-pool-name technova-lb-be --public-ip-address technova-lb-ip
# Health Probe ও Load Balancing Rule
az network lb probe create --name http-probe --lb-name technova-lb \
  --resource-group technova-prod-rg --protocol Http --port 80 --path /health
az network lb rule create --name http-rule --lb-name technova-lb \
  --resource-group technova-prod-rg --protocol Tcp --frontend-port 80 \
  --backend-port 80 --frontend-ip-name technova-lb-fe --backend-pool-name technova-lb-be
--probe-name http-probe
```

৮.৯ সাধারণ ভুল ও ট্রাবলশুটিং

- **ভুল:** Health Probe-এর পথে এমন একটি পাতা দেওয়া যা সবসময় ২০০ দেয় (যেমন /) — অ্যাপের ডেটাবেস নষ্ট হলেও Probe সুস্থ দেখাবে। /health এন্ডপয়েন্টে নির্ভরতা যাচাই করুন।
- **ভুল:** Basic SKU Load Balancer ব্যবহার করা — এতে Availability Zone সমর্থন নেই; প্রোডাকশনে Standard SKU ব্যবহার করুন।
- **ট্রাবলশুটিং — "All backends unhealthy":** (১) VM-এ অ্যাপ চলছে কি না, (২) NSG ব্যাকএন্ড পোর্টে Load Balancer-এর Probe ট্র্যাফিক অনুমোদন করে কি না (সার্ভিস ট্যাগ `AzureLoadBalancer`), (৩) Probe-এর পথ ও পোর্ট সঠিক কি না, (৪) VM-এর ফায়ারওয়াল (iptables/Windows Firewall) পোর্ট খোলা কি না।

৮.১২ সারসংক্ষেপ

DNS নামকে IP-তে বদলায়; Load Balancer ট্র্যাফিক ভাগ করে; Application Gateway Layer 7 রাউটিং ও TLS শেষ করে; Front Door বিশ্বজুড়ে রাউটিং করে। সঠিক সরঞ্জাম বাছাই করুন: TCP/UDP-র জন্য Load Balancer, ওয়েব অ্যাপের জন্য Application Gateway, গ্লোবাল অ্যাপের জন্য Front Door। Health Probe যেন আসল স্বাস্থ্য প্রতিফলিত করে, এবং WAF নতুন নিয়মে প্রথমে Detection মোডে চালান।

৮.১৩ পরিভাষা

Azure DNS — Microsoft-পরিচালিত DNS সেবা — হোস্টেড জোন ও রেকর্ড পরিচালনা করে।

Hosted Zone — একটি ডোমেইনের DNS রেকর্ডের ধারক (যেমন technova.example)।

Azure Load Balancer — Layer 4 লোড ব্যালান্সার — TCP/UDP ট্র্যাফিক ভাগ করে।

Application Gateway — Layer 7 লোড ব্যালান্সার — HTTP/HTTPS রাউটিং, TLS, WAF।

Front Door — গ্লোবাল Layer 7 এন্ট্রি পয়েন্ট — মাল্টি-Region রাউটিং ও ক্যাশিং।

Health Probe — ব্যাকএন্ডের স্বাস্থ্য পরীক্ষার নিয়মিত অনুরোধ।

TLS Termination — HTTPS সংযোগ লোড ব্যালান্সারে শেষ করা — ব্যাকএন্ডে HTTP-তে যাওয়া।

WAF — Web Application Firewall — সাধারণ ওয়েব আক্রমণ ব্লক করে।

৮.১৪ জ্ঞান যাচাই

1. Load Balancer, Application Gateway ও Front Door-এর মধ্যে পার্থক্য কী? প্রতিটির একটি ব্যবহার-ক্ষেত্র লিখুন।
2. Health Probe-এর পথ হিসেবে “/” ব্যবহার করা কেন ঝুঁকিপূর্ণ?
3. TLS Termination কী? End-to-end TLS কখন দরকার?
4. WAF-এর Detection ও Prevention মোডের পার্থক্য কী? নতুন নিয়মে কোনটি দিয়ে শুরু করবেন?
5. “All backends unhealthy” দেখলে কোন চারটি জিনিস যাচাই করবেন?

৮.১৫ অনুশীলন

Azure Portal-এ একটি Azure DNS জোন তৈরি করে একটি A রেকর্ড যোগ করুন। একটি Load Balancer তৈরি করে Health Probe-সহ একটি নিয়ম সাজান। তারপর একটি ব্যাকএন্ড VM বন্ধ করে দেখুন ট্র্যাফিক কীভাবে আচরণ করে।

অধ্যায় ৯ • PART II

Chapter 9 — Azure Virtual Machines

৯.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি পারবেন — (১) VM সাইজ ও পরিবার (family) বাছাই করতে; (২) ইমেজ, Managed Disk ও VM লাইফসাইকেল ব্যাখ্যা করতে; (৩) cloud-init ও এক্সটেনশন দিয়ে VM কনফিগার করতে; (৪) SSH ও Azure Bastion দিয়ে নিরাপদে সংযোগ করতে; (৫) IMDS ও পরিচয় ব্যবস্থাপনা ব্যাখ্যা করতে; এবং (৬) VM সমস্যার সমাধান করতে।

৯.২ পূর্বশর্ত জ্ঞান

অধ্যায় ৬–৮: VNet, NSG, Load Balancer। Linux কমান্ড লাইন ও SSH-এর মৌলিক জ্ঞান।

৯.৩ কেন এই বিষয়টি গুরুত্বপূর্ণ

VM হলো Azure-এর সবচেয়ে মৌলিক কম্পিউট সার্ভিস — এবং সবচেয়ে বেশি নিয়ন্ত্রণ দেয়। PaaS যখন যথেষ্ট নয় (বিশেষ OS কনফিগারেশন, পুরোনো অ্যাপ, বিশেষ সফটওয়্যার), তখন VM-ই পথ। কিন্তু VM-এর সাথে আসে দায়িত্ব: OS প্যাচ, নিরাপত্তা, ব্যাকআপ — সব আপনার। এই অধ্যায়ে আপনি VM-কে পেশাদারভাবে চালানো শিখবেন।

৯.৪ প্রথম নীতি থেকে: VM কীভাবে কাজ করে

একটি Azure VM হলো Microsoft-এর হাইপারভাইজারে চলা একটি ভার্চুয়াল মেশিন — আপনার নিজের কম্পিউটারের মতো, কিন্তু ডেটা সেন্টারে। VM তৈরি করার সময় আপনি বাছাই করেন: **সাইজ** (vCPU ও RAM — যেমন Standard_B2s: ২ vCPU, ৪ GB RAM), **ইমেজ** (OS — যেমন Ubuntu 24.04 LTS), **ডিস্ক** (OS ডিস্ক + ডেটা ডিস্ক), **নেটওয়ার্ক** (কোন VNet/সাবনেট, পাবলিক IP আছে কি না), এবং **পরিচয়** (Managed Identity)। VM-এর দাম নির্ভর করে সাইজ, Region ও চলার সময়ের ওপর — বন্ধ (deallocated) VM-এর কম্পিউট চার্জ হয় না, কিন্তু ডিস্ক ও IP-এর চার্জ চলতে থাকে।

৯.৫ VM সাইজ ও পরিবার

পরিবার	বৈশিষ্ট্য	ব্যবহার
B-series (Burstable)	কম বেসলাইন, ক্রেডিট জমিয়ে হঠাৎ বেশি CPU	উন্নয়ন, ছোট ওয়েবসাইট, কম লোড
D-series (General purpose)	ভারসাম্যপূর্ণ CPU/RAM	বেশিরভাগ অ্যাপ, API সার্ভার
E-series (Memory optimized)	বেশি RAM	ডেটাবেস, ক্যাশ, ইন-মেমরি অ্যাপ
F-series (Compute optimized)	বেশি CPU, কম RAM	ব্যাচ প্রসেসিং, গণনা
N-series (GPU)	GPU সহ	AI/ML, গ্রাফিক্স

নিয়ম: সবচেয়ে ছোট সাইজ দিয়ে শুরু করুন, মনিটর করুন, তারপর বাড়ান। B-series সস্তা মনে হলেও ক্রেডিট শেষ হলে CPU থেমে যায় — প্রোডাকশনে D-series নিরাপদ। **Spot VM** অনেক সস্তা (৭০–৯০% ছাড়) কিন্তু Azure যেকোনো সময় তা ফেরত নিতে পারে — শুধু ব্যাচ কাজ বা পরীক্ষায় ব্যবহার করুন, প্রোডাকশনে নয়।

৯.৬ ইমেজ, Managed Disk ও লাইফসাইকেল

ইমেজ হলো OS-এর টেমপ্লেট — Azure Marketplace থেকে (Ubuntu, Windows Server) বা নিজের কাস্টম ইমেজ (Azure Compute Gallery)। **Managed Disk** হলো VM-এর ডিস্ক — Azure তা পরিচালনা করে (রেপ্লিকেশন, স্ন্যাপশট)। ডিস্কের ধরন: Standard HDD (সস্তা, ধীর), Standard SSD, Premium SSD (দ্রুত, প্রোডাকশন), Ultra Disk (সবচেয়ে দ্রুত, বিশেষ কাজ)। VM-এর অবস্থা: চলমান (running) → বন্ধ (stopped, তবু চার্জ হয়) → **deallocated** (চার্জ বন্ধ, IP ছেড়ে দেওয়া হয়)।

৯.৭ cloud-init ও এক্সটেনশন

VM তৈরি হওয়ার সময় স্বয়ংক্রিয়ভাবে কনফিগার করতে **cloud-init** ব্যবহার করুন — একটি YAML ফাইলে লিখুন কোন প্যাকেজ ইনস্টল হবে, কোন ব্যবহারকারী তৈরি হবে, কোন কমান্ড চলবে। এতে প্রতিটি VM একইভাবে তৈরি হয় — "হাতে কনফিগার করা" VM-এর মতো ভিন্নতা থাকে না। **VM Extension** হলো Azure-এর প্লাগইন — যেমন Custom Script Extension (স্ক্রিপ্ট চালানো), Azure Monitor Agent (মনিটরিং), Key Vault Extension (সার্টিফিকেট আনা)।

```
# cloud-init দিয়ে Ubuntu VM তৈরি (Node.js ইনস্টলসহ)
cat > cloud-init.yaml <<'EOF'
#cloud-config
package_update: true
packages:
  - nginx
  - nodejs
runcmd:
  - systemctl enable nginx
  - systemctl start nginx
EOF
az vm create --name technova-api-01 --resource-group technova-prod-rg \
  --location southeastasia --image Ubuntu2404 --size Standard_B2s \
  --vnet-name technova-vnet --subnet app-subnet --public-ip-address "" \
  --assign-identity --custom-data cloud-init.yaml --generate-ssh-keys
# Azure Bastion দিয়ে নিরাপদ সংযোগ (পাবলিক IP ছাড়া)
az network bastion create --name technova-bastion --resource-group technova-prod-rg \
  --location southeastasia --vnet-name technova-vnet --public-ip-address technova-
bastion-ip
```

> **WARNING:** VM-এ **পাবলিক IP দিয়ে SSH খোলা** সবচেয়ে সাধারণ নিরাপত্তা-ভুল — ইন্টারনেটের বটগুলো কয়েক মিনিটের মধ্যে SSH ব্রুট-ফোর্স শুরু করে। এর বদলে **Azure Bastion** ব্যবহার করুন (ব্রাউজার থেকে নিরাপদ সংযোগ, কোনো পাবলিক IP লাগে না) অথবা VPN/ExpressRoute। SSH কী-ভিত্তিক প্রমাণীকরণ ব্যবহার করুন — পাসওয়ার্ড-ভিত্তিক SSH বন্ধ রাখুন।

৯.৮ IMDS ও Managed Identity

Instance Metadata Service (IMDS) হলো VM-এর ভেতর থেকে অ্যাক্সেসযোগ্য একটি বিশেষ এন্ডপয়েন্ট (`http://169.254.169.254/metadata`) — এখান থেকে VM নিজের নাম, Zone, ট্যাগ জানতে পারে, এবং Managed Identity-র টোকেন নিতে পারে। IMDS শুধু VM-এর ভেতর থেকে পৌঁছানো যায়, এবং অনুরোধে `Metadata: true` হেডার

বাধ্যতামূলক — এটি SSRF আক্রমণ থেকে রক্ষা করে। VM-এর অ্যাপ এই টোকেন দিয়ে Key Vault বা Storage-এ অ্যাক্সেস করে — কোনো secret সংরক্ষণ ছাড়াই।

৯.৯ সাধারণ ভুল ও ট্রাবলশুটিং

- **ভুল:** VM "stopped" রেখে ভাবা চার্জ বন্ধ হয়েছে — deallocate করুন (`az vm deallocate`), নইলে কম্পিউট চার্জ চলতেই থাকবে।
- **ভুল:** OS ডিস্কে অ্যাপের ডেটা রাখা — OS ডিস্ক ছোট ও ধীর; ডেটার জন্য আলাদা ডেটা ডিস্ক ব্যবহার করুন।
- **ট্রাবলশুটিং — VM-এ ঢুকতে পারছেন না:** (১) Bastion/NSG দেখুন, (২) Boot Diagnostics-এ স্ক্রিনশট দেখুন — OS বুট হয়েছে কি না, (৩) `az vm run-command invoke` দিয়ে VM-এর ভেতরে কমান্ড চালিয়ে দেখুন, (৪) Serial Console ব্যবহার করুন।

৯.১২ সারসংক্ষেপ

VM হলো সবচেয়ে নমনীয় কম্পিউট — সাইজ, ইমেজ, ডিস্ক, নেটওয়ার্ক ও পরিচয় বাছাই করুন। প্রোডাকশনে D-series দিয়ে শুরু করুন, Spot VM শুধু ব্যাচ কাজে। cloud-init দিয়ে প্রতিটি VM একইভাবে তৈরি করুন। পাবলিক IP-তে SSH খুলবেন না — Azure Bastion ব্যবহার করুন। Managed Identity ও IMDS দিয়ে secret ছাড়া Azure রিসোর্সে অ্যাক্সেস করুন। আর মনে রাখুন: "stopped" মানেই চার্জ বন্ধ নয় — deallocate করতে হয়।

৯.১৩ পরিভাষা

VM Size — vCPU ও RAM-এর সমন্বয় — যেমন Standard_B2s, Standard_D2s_v5।

Image — OS-এর টেমপ্লেট — Marketplace বা Compute Gallery থেকে।

Managed Disk — Azure-পরিচালিত ভার্চুয়াল ডিস্ক — OS ডিস্ক ও ডেটা ডিস্ক।

cloud-init — VM প্রথম বুটে স্বয়ংক্রিয় কনফিগারেশনের YAML ব্যবস্থা।

Azure Bastion — ব্রাউজার থেকে VM-এ নিরাপদ সংযোগ — পাবলিক IP ছাড়া।

IMDS — Instance Metadata Service — VM-এর ভেতর থেকে মেটাডেটা ও পরিচয় টোকেন পাওয়ার এন্ডপয়েন্ট।

Spot VM — অনেক সস্তা কিন্তু যেকোনো সময় ফেরত নেওয়া যেতে পারে এমন VM — ব্যাচ কাজের জন্য।

৯.১৪ জ্ঞান যাচাই

1. B-series VM প্রোডাকশনে ব্যবহারের ঝুঁকি কী?
2. VM "stopped" ও "deallocated"-এর মধ্যে পার্থক্য কী? খরচের ওপর এর প্রভাব কী?
3. cloud-init কেন "হাতে কনফিগার করা" VM-এর চেয়ে ভালো?

4. Azure Bastion কী সমস্যার সমাধান করে? পাবলিক IP-তে SSH খোলা কেন বিপজ্জনক?
5. IMDS থেকে Managed Identity টোকেন নেওয়ার সুবিধা কী?

৯.১৫ অনুশীলন

cloud-init দিয়ে একটি Ubuntu VM তৈরি করুন যা Nginx ইনস্টল করে। VM-টিকে প্রাইভেট সাবনেটে রাখুন (পাবলিক IP ছাড়া) এবং Azure Bastion দিয়ে সংযোগ করুন। তারপর `az vm deallocate` চালিয়ে দেখুন চার্জ বন্ধ হয়েছে কি না (Cost Analysis-এ)।

অধ্যায় ১০ • PART II

Chapter 10 — Hybrid Connectivity

১০.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি পারবেন — (১) Site-to-Site VPN, Point-to-Site VPN ও ExpressRoute-এর মধ্যে বাছাই করতে; (২) VPN Gateway-এর কাজ ও সীমা ব্যাখ্যা করতে; (৩) হাইব্রিড DNS সমস্যার সমাধান করতে; এবং (৪) হাইব্রিড সংযোগের সমস্যার ধাপে ধাপে সমাধান করতে।

১০.২ পূর্বশর্ত জ্ঞান

অধ্যায় ৬–৭: VNet, Route Table, Gateway সাবনেট। VPN ও BGP-এর সাধারণ ধারণা থাকলে ভালো হয়।

১০.৩ কেন এই বিষয়টি গুরুত্বপূর্ণ

অনেক সংস্থার ডেটা এখনো নিজের ডেটা সেন্টারে — ডেটাবেস, পুরোনো অ্যাপ, নিয়ন্ত্রক কারণে। ক্লাউডে যাওয়ার অর্থ এই দুই জগতকে নিরাপদে সংযুক্ত করা। হাইব্রিড সংযোগ ভুল হলে হয় ডেটা ইন্টারনেট দিয়ে যায় (অনিরাপদ), নয়তো ল্যাটেন্সি এত বেশি হয় যে অ্যাপ ব্যবহারের অযোগ্য হয়ে যায়।

১০.৪ প্রথম নীতি থেকে: হাইব্রিড সংযোগের তিনটি পথ

পদ্ধতি	কী	ব্যান্ডউইথ/ল্যাটেন্সি	খরচ	কখন
Site-to-Site VPN	ইন্টারনেটের ওপর এনক্রিপ্টেড IPsec টানেল	পরিবর্তনশীল, ইন্টারনেট-নির্ভর	কম	দ্রুত সংযোগ, কম গুরুত্বপূর্ণ ট্র্যাফিক
Point-to-Site VPN	একজন ব্যবহারকারীর ডিভাইস থেকে Azure-এ VPN	পরিবর্তনশীল	কম	দূরবর্তী কর্মী, অ্যাডমিন অ্যাক্সেস
ExpressRoute	Microsoft-এর সাথে ব্যক্তিগত, নিবেদিত সংযোগ	স্থির, কম ল্যাটেন্সি, 1–100 Gbps	বেশি	প্রোডাকশন, সংবেদনশীল ডেটা, স্থির ল্যাটেন্সি

VPN Gateway হলো Azure-এর দিকে টানেলের শেষ প্রান্ত — এটি Gateway সাবনেটে থাকে (সাবনেটের নাম অবশ্যই `GatewaySubnet` হতে হবে, এবং /27 বা বড় হতে হবে)। **ExpressRoute** ইন্টারনেট ব্যবহার করে না — আপনার ডেটা সেন্টার থেকে সরাসরি Microsoft-এর নেটওয়ার্কে সংযোগ যায় (একটি সংযোগ প্রদানকারীর মাধ্যমে)। ফলে ল্যাটেন্সি স্থির ও নিরাপত্তা বেশি, কিন্তু স্থাপনে সপ্তাহ লাগতে পারে এবং মাসিক চার্জ বেশি।

১০.৫ BGP ও রাউটিং

BGP (Border Gateway Protocol) হলো দুই নেটওয়ার্কের মধ্যে রাউট তথ্য বিনিময়ের ভাষা। VPN Gateway-এ BGP চালু করলে Azure স্বয়ংক্রিয়ভাবে জানে অন-প্রিমিসেসের কোন ঠিকানা কোন দিকে, আর অন-প্রিমিসেসের রাউটার জানে Azure-এর ঠিকানা। BGP ছাড়া স্ট্যাটিক রাউট লিখতে হয় — ছোট সংযোগে চলে, বড় নেটওয়ার্কে ভুলের আশঙ্কা বাড়ে। ExpressRoute-এ BGP বাধ্যতামূলক।

১০.৬ হাইব্রিড DNS

সবচেয়ে সাধারণ হাইব্রিড সমস্যা হলো DNS: Azure-এর VM অন-প্রিমিসেসের সার্ভারের নাম রেজোলভ করতে পারে না, বা উল্টোটা। সমাধান: **Private DNS Zone** ও **DNS ফরওয়ার্ডার** — VNet-এর DNS সেটিংয়ে অন-প্রিমিসেসের DNS সার্ভার যোগ করুন, অথবা Azure-এ একটি DNS ফরওয়ার্ডার VM/সার্ভিস রাখুন যা দুই দিকের নাম রেজোলভ করে। Azure Private Resolver একটি পরিচালিত সমাধান — নিজে DNS সার্ভার চালানোর প্রয়োজন নেই।

১০.৭ সাধারণ ভুল ও ট্রাবলশুটিং

- **ভুল:** Gateway সাবনেটে অন্য রিসোর্স রাখা বা NSG লাগানো — GatewaySubnet শুধু গেটওয়ের জন্য, এবং এতে NSG লাগানো সমর্থিত নয়।
- **ভুল:** অন-প্রিমিসেস ও Azure-এর ঠিকানা স্থান ওভারল্যাপ করা — রাউটিং কাজ করবে না (অধ্যায় ৬)।
- **ট্রাবলশুটিং:** টানেল সংযুক্ত হচ্ছে না? (১) শেয়ার্ড কী দুই দিকে এক কি না, (২) অন-প্রিমিসেস ফায়ারওয়ালে UDP 500/4500 খোলা কি না, (৩) `az network vpn-connection show`-এ সংযোগের অবস্থা দেখুন, (৪) Network Watcher-এর VPN ট্রাবলশুট ব্যবহার করুন।

১০.১২ সারসংক্ষেপ

হাইব্রিড সংযোগের তিনটি পথ: Site-to-Site VPN (দ্রুত, সস্তা, ইন্টারনেট-নির্ভর), Point-to-Site VPN (ব্যক্তিগত ডিভাইসের জন্য), ExpressRoute (ব্যক্তিগত, স্থির, ব্যয়বহুল)। VPN Gateway GatewaySubnet-এ থাকে। BGP দিয়ে রাউট স্বয়ংক্রিয় করুন। আর হাইব্রিড DNS-এর পরিকল্পনা আগেই করুন — নাম রেজোলভ না হলে সংযোগ থাকলেও অ্যাপ কাজ করবে না।

১০.১৩ পরিভাষা

VPN Gateway — Azure-এর দিকে IPsec টানেলের শেষ প্রান্ত — GatewaySubnet-এ থাকে।

Site-to-Site VPN — অন-প্রিমিসেস নেটওয়ার্ক ও Azure VNet-এর মধ্যে এনক্রিপ্টেড টানেল।

Point-to-Site VPN — একজন ব্যবহারকারীর ডিভাইস থেকে Azure VNet-এ VPN সংযোগ।

ExpressRoute — ইন্টারনেট ছাড়া Microsoft নেটওয়ার্কে ব্যক্তিগত, নিবেদিত সংযোগ।

BGP — দুই নেটওয়ার্কের মধ্যে রাউট তথ্য বিনিময়ের প্রোটোকল।

GatewaySubnet — শুধু VPN/ExpressRoute গেটওয়ের জন্য সংরক্ষিত সাবনেট।

১০.১৪ জ্ঞান যাচাই

1. Site-to-Site VPN ও ExpressRoute-এর মধ্যে তিনটি পার্থক্য লিখুন।
2. GatewaySubnet-এর নাম ও সাইজ নিয়ে বিশেষ নিয়ম কী?

3. BGP কেন স্ট্যাটিক রাউটের চেয়ে ভালো? একটি উদাহরণ দিন।
4. হাইব্রিড DNS সমস্যার দুটি সমাধান লিখুন।
5. VPN টানেল সংযুক্ত না হলে কোন চারটি জিনিস যাচাই করবেন?

১০.১৫ অনুশীলন

একটি Site-to-Site VPN-এর নকশা আঁকুন: অন-প্রিমিসেস 10.1.0.0/16, Azure 10.0.0.0/16, GatewaySubnet 10.0.5.0/27, BGP চালু। কোন রাউট কোন দিকে যাবে তা লিখুন। (বাস্তব অন-প্রিমিসেস রাউটার না থাকলে শুধু নকশা ও CLI কমান্ডের খসড়া তৈরি করুন — খরচ এড়াতে গেটওয়ে তৈরি করবেন না।)

অধ্যায় ১১ • PART II

Chapter 11 — Availability and Compute Scaling

১১.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি পারবেন — (১) Availability Zone, Availability Set ও VM Scale Sets-এর পার্থক্য করতে; (২) Autoscale নিয়ম (metric, schedule, target tracking) সাজাতে; (৩) Health Probe ও স্বয়ংক্রিয় প্রতিস্থাপনের কাজ ব্যাখ্যা করতে; এবং (৪) স্কেলিং ব্যর্থতার কারণ খুঁজে বের করতে।

১১.২ পূর্বশর্ত জ্ঞান

অধ্যায় ৮–৯: Load Balancer, VM। মেট্রিক (CPU, মেমরি) ও অ্যালার্টের সাধারণ ধারণা।

১১.৩ কেন এই বিষয়টি গুরুত্বপূর্ণ

একটি সার্ভারে ট্র্যাফিক বাড়লে সাইট ধীর হয়ে যায়; সার্ভার নষ্ট হলে সাইট বন্ধ হয়ে যায়। সমাধান একটাই — একাধিক কপি, স্বয়ংক্রিয়ভাবে। এই অধ্যায়ে আপনি শিখবেন কীভাবে Azure স্বয়ংক্রিয়ভাবে VM-এর সংখ্যা বাড়ায়-কমায়, এবং নষ্ট VM-কে নিজে থেকে বদলে দেয়।

১১.৪ প্রথম নীতি থেকে: উচ্চ-উপলভ্যতার তিনটি স্তর

পদ্ধতি	কী থেকে রক্ষা করে	SLA ধারণা	কখন
Availability Set	একই ডেটা সেন্টারের ভেতরে র‍্যাক/বিদ্যুৎ ব্যর্থতা (Fault Domain ও Update Domain)	দুটি VM-এ 99.95%	পুরোনো নকশা — এখন Zone ব্যবহার করা ভালো
Availability Zone	পুরো ডেটা সেন্টারের ব্যর্থতা	দুটি Zone-এ 99.99%	প্রোডাকশন — ডিফল্ট পছন্দ
Multi-Region	পুরো Region-এর বিপর্যয়	99.99%+	অতি-গুরুত্বপূর্ণ সিস্টেম (অধ্যায় ২৩)

Fault Domain হলো একই বিদ্যুৎ/নেটওয়ার্ক ভাগ করে এমন র‍্যাকের গোষ্ঠী; **Update Domain** হলো একসাথে আপডেট/রিবুট হয় এমন গোষ্ঠী। Availability Set নিশ্চিত করে আপনার VM-গুলো আলাদা Fault Domain ও Update Domain-এ ছড়িয়ে থাকে — একটি র‍্যাক নষ্ট হলে বা একটি আপডেটে রিবুট হলে সব VM একসাথে বন্ধ হয় না। তবে আধুনিক নকশায় **Availability Zone**-ই সেরা — কারণ এটি পুরো ডেটা সেন্টারের ব্যর্থতা থেকে রক্ষা করে।

১১.৫ VM Scale Sets (VMSS): একই VM-এর অনেক কপি

Virtual Machine Scale Sets (VMSS) হলো একই কনফিগারেশনের VM-এর গোষ্ঠী — একটি ইমেজ, একটি সাইজ, একটি সংখ্যা (যেমন ২–১০টি)। লোড বাড়লে VMSS নতুন VM যোগ করে, কমলে সরিয়ে দেয়। VMSS Load Balancer-এর সাথে যুক্ত থাকে, এবং **Flexible orchestration mode**-এ প্রতিটি VM-কে আলাদাভাবে পরিচালনা করা যায় (একটি VM মুছে ফেলা, আলাদা সাইজ দেওয়া)। Uniform মোড পুরোনো — নতুন কাজে Flexible ব্যবহার করুন।

১১.৬ Autoscale: কখন বাড়বে, কখন কমবে

Autoscale নিয়ম তিন ধরনের: (১) **Metric-based** — যেমন গড় CPU ৭০%-এর বেশি ৫ মিনিট ধরে থাকলে ১টি VM যোগ করো; ৩০%-এর নিচে থাকলে ১টি কমাও। (২) **Schedule-based** — যেমন সকাল ৯টায় ৫টি VM, রাত ১২টায় ২টি (অফিস সময়ের অ্যাপের জন্য)। (৩) **Predictive** — অতীতের প্যাটার্ন দেখে আগেই স্কেল করা (সীমিত সার্ভিসে)। গুরুত্বপূর্ণ নিয়ম: **cooldown** সময় দিন (যেমন ৫ মিনিট) — নইলে স্কেল-আপ ও স্কেল-ডাউন বারবার হতে থাকে (flapping)। আর স্কেল-ডাউন সবসময় ধীরে করুন — হঠাৎ VM কমালে চলমান অনুরোধ হারিয়ে যেতে পারে।

> **CONCEPT:** স্কেলিংয়ের আগে অ্যাপকে **stateless** করুন — সেশন ডেটা VM-এর মেমরিতে রাখবেন না, Redis বা ডেটাবেসে রাখুন। নইলে নতুন VM-এ গেলে ব্যবহারকারীর সেশন হারিয়ে যাবে। এটাই “12-factor app”-এর মূল নীতি, এবং স্কেলিংয়ের পূর্বশর্ত।

```
# VM Scale Set তৈরি (2টি VM, Zone 1 ও 2-এ ছড়ানো)
az vmss create --name technova-vmss --resource-group technova-prod-rg \
  --location southeastasia --image Ubuntu2404 --vm-sku Standard_B2s \
  --instance-count 2 --zones 1 2 --vnet-name technova-vnet --subnet app-subnet \
  --lb technova-lb --assign-identity --generate-ssh-keys
# Autoscale নিয়ম: CPU 70% ছাড়ালে বাড়বে, 30%-এর নিচে কমবে
az monitor autoscale create --name technova-autoscale --resource-group technova-prod-rg \
  --resource technova-vmss --resource-type Microsoft.Compute/virtualMachineScaleSets \
  --min-count 2 --max-count 6 --count 2
az monitor autoscale rule create --autoscale-name technova-autoscale --resource-group
technova-prod-rg \
  --condition "Percentage CPU > 70 avg 5m" --scale out 1 --cooldown 5
```

১১.৭ Health Monitoring ও স্বয়ংক্রিয় প্রতিস্থাপন

VMSS-এর **Application Health Extension** বা Load Balancer Probe VM-এর স্বাস্থ্য দেখে — অসুস্থ VM শনাক্ত হলে VMSS তাকে স্বয়ংক্রিয়ভাবে বদলে দেয় (reimage/replace)। **Automatic Repairs** চালু করলে এই প্রতিস্থাপন স্বয়ংক্রিয় হয়। মনে রাখুন: VM “চলছে” মানেই অ্যাপ সুস্থ নয় — Health Extension-এ আপনার /health এন্ডপয়েন্ট যাচাই করা উচিত (অধ্যায় ৮)।

১১.৮ সাধারণ ভুল ও ট্রাবলশুটিং

- **ভুল:** সর্বনিম্ন (min) সংখ্যা ১ রাখা — একটি VM নষ্ট হলে সেবা বন্ধ; প্রোডাকশনে min অন্তত ২, দুটি Zone-এ।
- **ভুল:** Cooldown না দেওয়া — স্কেল-আপ/ডাউন বারবার হতে থাকে, বিল বাড়ে।
- **ট্রাবলশুটিং — স্কেল হচ্ছে না:** (১) সাবস্ক্রিপশনের vCPU কোটা দেখুন (`az vm list-usage`), (২) Autoscale নিয়মের মেট্রিক নাম ও থ্রেশহোল্ড যাচাই করুন, (৩) Activity Log-এ Autoscale ইভেন্ট দেখুন — নিয়মটি আসলে ট্রিগার হয়েছে কি না।

১১.১২ সারসংক্ষেপ

উচ্চ-উপলভ্যতা মানে ব্যর্থতা ধরে নিয়ে নকশা করা: Zone দিয়ে ডেটা সেন্টারের ব্যর্থতা থেকে রক্ষা পান, VMSS দিয়ে একই VM-এর অনেক কপি চালান, Autoscale দিয়ে লোড অনুযায়ী সংখ্যা বাড়ান-কমান। স্কেলিংয়ের পূর্বশর্ত stateless অ্যাপ। Cooldown দিন, min সংখ্যা অন্তত ২ রাখুন, এবং স্বাস্থ্য পরীক্ষা যেন আসল অ্যাপের স্বাস্থ্য দেখে।

১১.১৩ পরিভাষা

Availability Zone — আলাদা ডেটা সেন্টার — Zone-স্তরের ব্যর্থতা থেকে রক্ষা করে।

Availability Set — একই ডেটা সেন্টারের ভেতরে Fault Domain ও Update Domain-এ VM ছড়ানো — পুরোনো পদ্ধতি।

Fault Domain — একই বিদ্যুৎ/নেটওয়ার্ক ভাগ করে এমন র‍্যাকের গোষ্ঠী।

Update Domain — একসাথে আপডেট/রিবুট হয় এমন VM-এর গোষ্ঠী।

VMSS — Virtual Machine Scale Sets — একই কনফিগারেশনের VM-এর গোষ্ঠী, স্বয়ংক্রিয় স্কেলিংসহ।

Autoscale — মেট্রিক বা সময়সূচি অনুযায়ী VM-এর সংখ্যা স্বয়ংক্রিয়ভাবে বাড়ানো-কমানো।

Cooldown — দুটি স্কেলিং কাজের মধ্যে অপেক্ষার সময় — flapping রোধ করে।

১১.১৪ জ্ঞান যাচাই

1. Availability Set ও Availability Zone-এর মধ্যে পার্থক্য কী? নতুন নকশায় কোনটি ব্যবহার করবেন?
2. VMSS-এর Flexible ও Uniform মোডের পার্থক্য কী?
3. Autoscale-এর তিন ধরনের নিয়ম কী কী? একটি করে উদাহরণ দিন।
4. Cooldown কেন দরকার? Cooldown না থাকলে কী হয়?
5. স্কেলিংয়ের আগে অ্যাপকে stateless করা কেন জরুরি?

১১.১৫ অনুশীলন

একটি VMSS তৈরি করুন (min 2, max 4, Zone 1 ও 2)। একটি CPU-ভিত্তিক Autoscale নিয়ম যোগ করুন। তারপর একটি VM মুছে দেখুন VMSS স্বয়ংক্রিয়ভাবে তা বদলায় কি না। কাজ শেষে VMSS মুছে ফেলুন — চলমান VM-এর চার্জ বন্ধ করতে ভুলবেন না।

PART III • DEVOPS ENGINEERING MASTERY

Storage, Database & Application Services

Azure Storage, ডিস্ক ও ব্যাকআপ, রিলেশনাল ডেটাবেস, Cosmos DB ও ক্যাশিং, App Service ও Edge Delivery।

অধ্যায় ১২ • PART III

Chapter 12 — Azure Storage Engineering

১২.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি পারবেন — (১) Storage Account-এর কাঠামো ও সার্ভিসগুলো (Blob, Files, Queue, Table) ব্যাখ্যা করতে; (২) স্টোরেজ অ্যাক্সেস টিয়ার ও লাইফসাইকেল নীতি সাজাতে; (৩) রিডান্ড্যান্সি বিকল্প (LRS/ZRS/GRS/GZRS) বাছাই করতে; (৪) SAS ও Managed Identity দিয়ে নিরাপদ অ্যাক্সেস দিতে; এবং (৫) স্টোরেজ খরচ অপ্টিমাইজ করতে।

১২.২ পূর্বশর্ত জ্ঞান

অধ্যায় ৬–৭: VNet, Private Endpoint। HTTP, REST API ও অবজেক্ট স্টোরেজের সাধারণ ধারণা।

১২.৩ কেন এই বিষয়টি গুরুত্বপূর্ণ

প্রায় প্রতিটি অ্যাপ্লিকেশনের ডেটা কোথাও রাখতে হয় — ছবি, ভিডিও, ব্যাকআপ, লগ, ব্যবহারকারীর ফাইল। Azure Storage হলো সেই ভিত্তি: সস্তা, প্রায় অসীম, এবং অত্যন্ত টেকসই। কিন্তু ভুল কনফিগারেশন (যেমন পাবলিক Blob) বিশ্বের সবচেয়ে সাধারণ ডেটা লিকের কারণ। এই অধ্যায়ে আপনি Storage-কে নিরাপদ ও সাশ্রয়ীভাবে ব্যবহার করা শিখবেন।

১২.৪ প্রথম নীতি থেকে: অবজেক্ট স্টোরেজ কী

প্রচলিত ফাইল সিস্টেমে ফাইল থাকে ফোল্ডারের ভেতরে ফোল্ডারে — একটি গাছের মতো কাঠামো। **অবজেক্ট স্টোরেজ** সেই গাছ ভেঙে দেয়: প্রতিটি অবজেক্ট (ফাইল) একটি সমতল নামস্থানে (flat namespace) থাকে, একটি অনন্য নাম ও মেটাডেটাসহ। এর ফলে কোটি কোটি অবজেক্ট রাখা যায়, যেকোনো অবজেক্ট সরাসরি নাম দিয়ে পাওয়া যায়, এবং স্টোরেজ প্রায় অসীমভাবে স্কেল করে। Azure Blob Storage এই মডেলের বাস্তবায়ন।

সার্ভিস	কী রাখে	ব্যবহার
Blob Storage	ফাইল/অবজেক্ট (ছবি, ভিডিও, ব্যাকআপ)	TechNova-র ব্যবহারকারীর আপলোড, স্ট্যাটিক অ্যাসেট
Azure Files	SMB/NFS ফাইল শেয়ার	অ্যাপ সার্ভারের শেয়ার্ড ফাইল, লিফট-অ্যান্ড-শিফট
Queue Storage	বার্তা (message)	অ্যাসিঙ্ক কাজের সারি (সহজ, সস্তা)
Table Storage	NoSQL কী-ভ্যালু ডেটা	সহজ ট্যাবুলার ডেটা, লগ

সব সার্ভিস একটি **Storage Account**-এর ভেতরে থাকে — এটি একটি অনন্য নামসহ (যেমন technovastore001) একটি কনটেইনার। Storage Account-এর ধরন (StorageV2 সাধারণ উদ্দেশ্যে) ও পারফরম্যান্স স্তর (Standard বনাম Premium) বাছাই করতে হয়। নামটি বিশ্বজুড়ে অনন্য হতে হয় — কারণ এটি DNS নামের অংশ হয় (`technovastore001.blob.core.windows.net`)।

১২.৫ Blob-এর ধরন ও অ্যাক্সেস টিয়ার

Blob তিন ধরনের: **Block Blob** (সাধারণ ফাইল — ছবি, ভিডিও, ডকুমেন্ট), **Append Blob** (শুধু শেষে যোগ করা যায় — লগের জন্য), **Page Blob** (র‍্যান্ডম পড়া/লেখা — VM ডিস্কের পেছনে এটাই ব্যবহৃত হয়)। অ্যাক্সেস টিয়ার নির্ধারণ করে খরচ: **Hot** (ঘন ঘন অ্যাক্সেস — বেশি স্টোরেজ চার্জ, কম অ্যাক্সেস চার্জ), **Cool** (মাসে একবার — কম স্টোরেজ, বেশি অ্যাক্সেস), **Cold** (কদাচিৎ), **Archive** (প্রায় কখনো নয় — সবচেয়ে সস্তা, পড়তে ঘণ্টা লাগে)। **লাইফসাইকেল নীতি** দিয়ে স্বয়ংক্রিয়ভাবে পুরোনো Blob Cool/Archive-এ পাঠানো যায় — যেমন ৩০ দিন পর Cool, ৯০ দিন পর Archive।

১২.৬ রিডান্ড্যান্সি: LRS, ZRS, GRS, GZRS

বিকল্প	কপি কোথায়	সহনশীলতা	খরচ
LRS	একটি ডেটা সেন্টারে ৩ কপি	ডিস্ক/র‍্যাক ব্যর্থতা	সবচেয়ে কম
ZRS	৩টি Zone-এ ৩ কপি	পুরো Zone-এর ব্যর্থতা	মাঝারি
GRS	প্রাথমিক Region + দ্বিতীয় Region-এ ৩ কপি করে	পুরো Region-এর ব্যর্থতা	বেশি
GZRS	Zone + দ্বিতীয় Region	Zone ও Region ব্যর্থতা	সবচেয়ে বেশি

নিয়ম: প্রোডাকশন ডেটায় অন্তত ZRS ব্যবহার করুন — LRS-এ একটি ডেটা সেন্টারের বিপর্যয়ে ডেটা হারাতে পারে। GRS-এ দ্বিতীয় Region-এ পড়া (read-access) চাইলে **RA-GRS** বাছাই করুন। মনে রাখুন: রিডান্ড্যান্সি হার্ডওয়্যার ব্যর্থতা থেকে রক্ষা করে, ভুল করে মোছা থেকে নয় — তার জন্য **Blob Versioning** ও **Soft Delete** চালু করুন (অধ্যায় ১৩)।

১২.৭ নিরাপদ অ্যাক্সেস: SAS, Managed Identity ও পাবলিক অ্যাক্সেস

Storage-এ অ্যাক্সেসের তিনটি পথ: (১) **অ্যাকাউন্ট কী** — পুরো অ্যাকাউন্টের চাবি, কখনো অ্যাপে সরাসরি ব্যবহার করবেন না; (২) **Managed Identity + RBAC** — সেরা পদ্ধতি: VM বা App Service-কে “Storage Blob Data Reader” ভূমিকা দিন; (৩) **SAS (Shared Access Signature)** — সময়সীমাবদ্ধ, নির্দিষ্ট অনুমতিসহ একটি স্বাক্ষরিত URL — যেমন ব্যবহারকারীকে ১ ঘণ্টার জন্য একটি ফাইল আপলোড করতে দেওয়া। SAS-এর মেয়াদ ও অনুমতি যত সংকীর্ণ সম্ভব রাখুন। আর **“Allow Blob public access” বন্ধ রাখুন** — এটি Storage Account-স্তরে বন্ধ করা যায়, এবং Azure Policy দিয়ে জোর করা যায়।

```
# Storage Account তৈরি (ZRS, HTTPS-only, TLS 1.2, পাবলিক Blob অ্যাক্সেস বন্ধ)
az storage account create --name technovastore001 --resource-group technova-prod-rg \
  --location southeastasia --sku Standard_ZRS --kind StorageV2 \
  --https-only true --min-tls-version TLS1_2 --allow-blob-public-access false
# একটি কনটেইনার ও লাইফসাইকেল নীতি
az storage container create --name uploads --account-name technovastore001 --auth-mode
login
cat > lifecycle.json <<'EOF'
{
  "rules": [
    { "enabled": true, "name": "archive-old-uploads", "type": "Lifecycle",
      "definition": {
        "actions": { "baseBlob": { "tierToCool": { "daysAfterModificationGreaterThan":
30 }, "tierToArchive": { "daysAfterModificationGreaterThan": 90 } } },
        "filters": { "blobTypes": ["blockBlob"], "prefixMatch": ["uploads/"] } } }
```

```
  ]
}
EOF
az storage account management-policy create --account-name technovastore001 --resource-group technova-prod-rg --policy @lifecycle.json
# 1 ঘণ্টার জন্য একটি Blob-এ পড়ার SAS তৈরি করুন
az storage blob generate-sas --account-name technovastore001 --container-name uploads \
  --name report.pdf --permissions r --expiry 2026-11-09T02:00:00Z --auth-mode login --as-user --output tsv
```

১২.৮ সাধারণ ভুল ও ট্রাবলশুটিং

- **ভুল:** অ্যাকাউন্ট কী কোডে লেখা বা Git-এ দেওয়া — কী ফাঁস হলে পুরো অ্যাকাউন্ট আপস হয়। Managed Identity ব্যবহার করুন।
- **ভুল:** পাবলিক Blob অ্যাক্সেস চালু রাখা — অনেক ডেটা লিকের কারণ।
- **ট্রাবলশুটিং — "AuthorizationPermissionMismatch":** আপনার RBAC ভূমিকা ডেটা-স্তরের কি না দেখুন — "Reader" শুধু অ্যাকাউন্ট দেখতে দেয়, Blob পড়তে "Storage Blob Data Reader" লাগে (অধ্যায় ৩)।
- **ট্রাবলশুটিং — SAS কাজ করছে না:** মেয়াদ (expiry) UTC-তে কি না, অনুমতি (permissions) যথেষ্ট কি না, এবং ঘড়ির সময় ঠিক আছে কি না দেখুন।

১২.১২ সারসংক্ষেপ

Azure Storage-এর ভিত্তি হলো Storage Account — Blob (অবজেক্ট), Files, Queue, Table সার্ভিসসহ। Block Blob সাধারণ ফাইলের জন্য; Hot/Cool/Cold/Archive টিয়ার ও লাইফসাইকেল নীতি দিয়ে খরচ নিয়ন্ত্রণ করুন। প্রোডাকশনে অন্তত ZRS ব্যবহার করুন, Versioning ও Soft Delete চালু করুন। অ্যাক্সেসের জন্য Managed Identity + RBAC প্রথম পছন্দ, SAS সময়সীমাবদ্ধ ও সংকীর্ণ রাখুন, এবং পাবলিক Blob অ্যাক্সেস বন্ধ রাখুন।

১২.১৩ পরিভাষা

Storage Account — Blob, Files, Queue, Table সার্ভিসের ধারক — বিশ্বজুড়ে অনন্য নাম।

Blob Storage — অবজেক্ট স্টোরেজ — ফাইল/অবজেক্ট রাখার সার্ভিস।

Block Blob — সাধারণ ফাইল সংরক্ষণের Blob ধরন।

Access Tier — Hot/Cool/Cold/Archive — অ্যাক্সেসের ঘনত্ব অনুযায়ী খরচের স্তর।

LRS / ZRS / GRS / GZRS — রিডান্ড্যান্সি বিকল্প — কপি কোথায় রাখা হবে তা নির্ধারণ করে।

SAS — Shared Access Signature — সময়সীমাবদ্ধ, নির্দিষ্ট অনুমতিসহ স্বাক্ষরিত URL।

Soft Delete — মোছা Blob/ফাইল নির্দিষ্ট সময়ের জন্য পুনরুদ্ধারযোগ্য রাখার ব্যবস্থা।

১২.১৪ জ্ঞান যাচাই

1. Block Blob, Append Blob ও Page Blob-এর পার্থক্য কী? প্রতিটির একটি ব্যবহার লিখুন।
2. Hot, Cool ও Archive টিয়ারের মধ্যে খরচের trade-off কী? লাইফসাইকেল নীতি কীভাবে সাহায্য করে?
3. LRS ও ZRS-এর মধ্যে পার্থক্য কী? প্রোডাকশনে কোনটি ব্যবহার করবেন?
4. Storage অ্যাক্সেসে Managed Identity কেন অ্যাকাউন্ট কী-এর চেয়ে ভালো?
5. SAS-এর মেয়াদ ও অনুমতি সংকীর্ণ রাখা কেন গুরুত্বপূর্ণ?

১২.১৫ অনুশীলন

একটি Storage Account তৈরি করুন (Standard_ZRS, HTTPS-only, পাবলিক অ্যাক্সেস বন্ধ)। একটি কনটেইনার তৈরি করে একটি ফাইল আপলোড করুন, তারপর ১ ঘণ্টার SAS দিয়ে সেটি পড়ুন। লাইফসাইকেল নীতি যোগ করুন: ৩০ দিন পর Cool, ৯০ দিন পর Archive।

অধ্যায় ১৩ • PART III

Chapter 13 — Disks, Files and Backup

১৩.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি পারবেন — (১) Managed Disk-এর ধরন ও পারফরম্যান্স বাছাই করতে; (২) Azure Files দিয়ে শেয়ার্ড ফাইল ব্যবস্থা সাজাতে; (৩) স্ন্যাপশট ও Azure Backup দিয়ে ব্যাকআপ নীতি তৈরি করতে; (৪) Recovery Services Vault-এর কাজ ব্যাখ্যা করতে; এবং (৫) পুনরুদ্ধার (restore) পরীক্ষা করতে।

১৩.২ পূর্বশর্ত জ্ঞান

অধ্যায় ৯ ও ১২: VM, Storage। ফাইল সিস্টেম ও ব্যাকআপের সাধারণ ধারণা।

১৩.৩ কেন এই বিষয়টি গুরুত্বপূর্ণ

ডেটা হারানোই সবচেয়ে খারাপ বিপর্যয় — একটি মুছে যাওয়া ডেটাবেস, একটি র‍্যানসমওয়্যার আক্রমণ, একটি ভুল স্ক্রিপ্ট। ব্যাকআপ মানে শুধু “কপি রাখা” নয় — মানে “প্রয়োজনের সময় নির্দিষ্ট সময়ের মধ্যে ফিরিয়ে আনতে পারা”। এই অধ্যায়ে আপনি Azure-এর ব্যাকআপ ব্যবস্থা শিখবেন।

১৩.৪ প্রথম নীতি থেকে: ব্লক, ফাইল ও অবজেক্ট স্টোরেজ

ধরন	কী	ব্যবহার
Block Storage	কাঁচা ব্লক — OS ডিস্কের মতো	VM-এর Managed Disk
File Storage	শেয়ার্ড ফাইল সিস্টেম (SMB/NFS)	একাধিক VM-এর শেয়ার্ড ফাইল
Object Storage	HTTP API দিয়ে অবজেক্ট	Blob — ছবি, ব্যাকআপ, অ্যাসেট

একটি VM-এর OS ও ডেটা থাকে **Managed Disk**-এ — ব্লক স্টোরেজ। একাধিক VM একই ফাইল ভাগ করতে চাইলে **Azure Files** (SMB প্রোটোকল) ব্যবহার করুন — যেমন একটি কনটেন্ট ম্যানেজমেন্ট সিস্টেমের আপলোড ফোল্ডার। বড় ফাইল, ব্যাকআপ ও অ্যাসেটের জন্য Blob (অধ্যায় ১২)।

১৩.৫ Managed Disk-এর ধরন ও পারফরম্যান্স

ডিস্ক ধরন	পারফরম্যান্স	ব্যবহার
Standard HDD	কম IOPS, সস্তা	ব্যাকআপ, কম ব্যবহৃত ডেটা
Standard SSD	মাঝারি IOPS	উন্নয়ন, ছোট অ্যাপ
Premium SSD	উচ্চ IOPS ও থ্রুপুট	প্রোডাকশন ডেটাবেস, অ্যাপ
Ultra Disk	সবচেয়ে উচ্চ, কনফিগারযোগ্য IOPS	অতি-দ্রুত ডেটাবেস

Premium SSD-তে পারফরম্যান্স ডিস্কের সাইজের সাথে বাড়ে — বড় ডিস্ক মানে বেশি IOPS। তাই শুধু জায়গার জন্য নয়, পারফরম্যান্সের জন্যও বড় ডিস্ক লাগতে পারে। **Bursting** সুবিধায় ছোট ডিস্কও সাময়িকভাবে বেশি IOPS দিতে পারে — হঠাৎ লোডের জন্য উপযোগী।

১৩.৬ স্ন্যাপশট বনাম ব্যাকআপ

স্ন্যাপশট হলো একটি ডিস্কের একটি নির্দিষ্ট মুহূর্তের কপি — দ্রুত তৈরি হয়, নতুন ডিস্ক বানাতে ব্যবহার করা যায়। কিন্তু স্ন্যাপশট নিজে কোনো ব্যাকআপ নীতি নয় — এটি একই Region-এ থাকে, এবং ভুলে মুছে ফেলা যায়। **Azure Backup** হলো পরিচালিত ব্যাকআপ সেবা — নীতি অনুযায়ী (যেমন প্রতিদিন রাত ২টায়) স্বয়ংক্রিয়ভাবে ব্যাকআপ নেয়, **Recovery Services Vault**-এ সংরক্ষণ করে, এবং নির্দিষ্ট সময় (retention) পর্যন্ত রাখে। Vault-এ **Soft Delete** ও **immutable vault** চালু করলে আক্রমণকারীও ব্যাকআপ মুছতে পারে না — র‍্যানসমওয়্যার থেকে রক্ষার মূল উপায়।

১৩.৭ ব্যাকআপ নীতি: RPO, RTO ও 3-2-1 নিয়ম

RPO (Recovery Point Objective) মানে কতটুকু ডেটা হারানো মেনে নেওয়া যায় — যেমন RPO ১ ঘণ্টা মানে সর্বোচ্চ ১ ঘণ্টার ডেটা হারানো যাবে, তাই প্রতি ঘণ্টায় ব্যাকআপ দরকার। **RTO (Recovery Time Objective)** মানে কত সময়ের মধ্যে সেবা ফিরিয়ে আনতে হবে। **3-2-1 নিয়ম**: ডেটার ৩টি কপি, ২টি ভিন্ন মাধ্যমে, ১টি ভিন্ন স্থানে (যেমন অন্য Region)। Azure Backup-এ **Cross-Region Restore** চালু করলে দ্বিতীয় Region থেকে পুনরুদ্ধার করা যায়।

```
# একটি ডেটা ডিস্ক তৈরি ও VM-এ যুক্ত করুন
az disk create --name technova-data-disk --resource-group technova-prod-rg \
  --location southeastasia --size-gb 128 --sku Premium_LRS
az vm disk attach --name technova-api-01 --resource-group technova-prod-rg \
  --disk technova-data-disk --lun 0
# ডিস্কের স্ন্যাপশট নিন
az snapshot create --name technova-data-snap-20261109 --resource-group technova-prod-rg \
  --location southeastasia --source technova-data-disk
# Recovery Services Vault তৈরি ও VM ব্যাকআপ চালু করুন
az backup vault create --name technova-rsv --resource-group technova-prod-rg --location
southeastasia
az backup protection enable-for-vm --resource-group technova-prod-rg --vault-name
technova-rsv \
  --vm technova-api-01 --policy-name DefaultPolicy
```

১৩.৮ সাধারণ ভুল ও ট্রাবলশুটিং

- **ভুল:** ব্যাকআপ নেওয়া কিন্তু কখনো restore পরীক্ষা না করা — ব্যর্থ ব্যাকআপ সবচেয়ে বিপজ্জনক, কারণ আপনি ভাবেন আপনি নিরাপদ। প্রতি মাসে একটি test restore করুন।
- **ভুল:** ব্যাকআপ ও প্রোডাকশন ডেটা একই অ্যাকাউন্ট/সাবস্ক্রিপশনে রাখা — আক্রমণকারী দুটোই মুছে দিতে পারে। Vault-এ Soft Delete ও immutable সেটিং চালু করুন।
- **ট্রাবলশুটিং — ব্যাকআপ ব্যর্থ:** VM Agent চলছে কি না দেখুন, VSS (Windows) বা fsfreeze (Linux) সমস্যা আছে কি না, এবং Vault-এর নীতিতে পর্যাপ্ত retention আছে কি না।

১৩.১২ সারসংক্ষেপ

ব্লক স্টোরেজ VM-এর ডিস্কের জন্য, ফাইল স্টোরেজ শেয়ার্ড ফাইলের জন্য, অবজেক্ট স্টোরেজ বড় ফাইলের জন্য। Premium SSD প্রোডাকশনের ডিফল্ট; পারফরম্যান্স ডিস্কের সাইজের সাথে বাড়ে। স্ন্যাপশট দ্রুত কপি, কিন্তু ব্যাকআপ নীতি নয় — Azure Backup + Recovery Services Vault দিয়ে নীতিভিত্তিক ব্যাকআপ নিন, Soft Delete ও immutable vault চালু করুন, এবং নিয়মিত restore পরীক্ষা করুন। RPO/RTO ঠিক করুন, 3-2-1 নিয়ম মানুন।

১৩.১৩ পরিভাষা

Managed Disk — Azure-পরিচালিত VM ডিস্ক — Standard HDD/SSD, Premium SSD, Ultra Disk।

Snapshot — ডিস্কের একটি নির্দিষ্ট মুহূর্তের কপি — নতুন ডিস্ক তৈরিতে ব্যবহৃত হয়।

Azure Backup — পরিচালিত ব্যাকআপ সেবা — নীতি অনুযায়ী স্বয়ংক্রিয় ব্যাকআপ নেয়।

Recovery Services Vault — ব্যাকআপ ডেটা সংরক্ষণের ধারক — Soft Delete ও immutable সেটিংসহ।

RPO — Recovery Point Objective — সর্বোচ্চ কতটুকু ডেটা হারানো মেনে নেওয়া যায়।

RTO — Recovery Time Objective — কত সময়ের মধ্যে সেবা ফিরিয়ে আনতে হবে।

১৩.১৪ জ্ঞান যাচাই

1. Managed Disk-এর চারটি ধরনের পার্থক্য কী? প্রোডাকশন ডেটাবেসে কোনটি ব্যবহার করবেন?
2. স্ন্যাপশট ও Azure Backup-এর পার্থক্য কী? কেন স্ন্যাপশট একা যথেষ্ট নয়?
3. RPO ও RTO-এর পার্থক্য কী? RPO ১ ঘণ্টা হলে ব্যাকআপ কত ঘন ঘন নিতে হবে?
4. 3-2-1 নিয়ম কী? Azure-এ এটি কীভাবে প্রয়োগ করবেন?
5. Immutable vault কেন র‍্যানসমওয়্যার থেকে রক্ষা করে?

১৩.১৫ অনুশীলন

একটি ডেটা ডিস্ক তৈরি করে VM-এ যুক্ত করুন, তার স্ন্যাপশট নিন, স্ন্যাপশট থেকে একটি নতুন ডিস্ক তৈরি করুন। একটি Recovery Services Vault তৈরি করে একটি VM-এর ব্যাকআপ চালু করুন এবং একটি test restore-এর পরিকল্পনা লিখুন।

অধ্যায় ১৪ • PART III

Chapter 14 — Azure Relational Database Engineering

১৪.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি পারবেন — (১) Azure SQL Database, SQL Managed Instance ও Azure Database for PostgreSQL-এর মধ্যে বাছাই করতে; (২) উচ্চ-উপলভ্যতা (Zone-redundant, Failover Group) সাজাতে; (৩) ব্যাকআপ ও Point-in-Time Recovery ব্যাখ্যা করতে; (৪) নেটওয়ার্ক ও পরিচয় দিয়ে ডেটাবেস সুরক্ষিত করতে; এবং (৫) পারফরম্যান্স পর্যবেক্ষণ ও সমস্যার সমাধান করতে।

১৪.২ পূর্বশর্ত জ্ঞান

অধ্যায় ৭ ও ১২–১৩: Private Endpoint, Storage, Backup। SQL ও রিলেশনাল ডেটাবেসের মৌলিক জ্ঞান।

১৪.৩ কেন এই বিষয়টি গুরুত্বপূর্ণ

ডেটাবেস হলো অ্যাপ্লিকেশনের হৃদয় — এটি বন্ধ হলে সব বন্ধ। নিজে ডেটাবেস সার্ভার চালানো মানে প্যাচ, ব্যাকআপ, রেপ্লিকেশন, ফেইলওভার — সব নিজে করা। পরিচালিত (managed) ডেটাবেস এই ভার Azure-কে দেয়, কিন্তু সঠিক সার্ভিস বাছাই, নেটওয়ার্কিং ও ব্যাকআপের দায়িত্ব আপনারই থাকে।

১৪.৪ প্রথম নীতি থেকে: পরিচালিত ডেটাবেস কী দেয়

একটি পরিচালিত ডেটাবেস সার্ভিসে Azure দেয়: OS প্যাচ, ডেটাবেস ইঞ্জিন প্যাচ, স্বয়ংক্রিয় ব্যাকআপ, উচ্চ-উপলভ্যতা (রেপ্লিকা), মনিটরিং ও স্কেলিং। আপনি দেন: স্কিমা, ইনডেক্স, কোয়েরি, অ্যাক্সেস নিয়ন্ত্রণ, এবং ব্যাকআপ নীতি যাচাই। এই ভাগাভাগি বোঝা গুরুত্বপূর্ণ — “Azure দেখবে” ভেবে ব্যাকআপ যাচাই না করলে একদিন দেখবেন retention শেষ হয়ে গেছে।

সার্ভিস	কী	কখন
Azure SQL Database	সম্পূর্ণ পরিচালিত SQL Server — single database বা elastic pool	নতুন অ্যাপ, SaaS, সহজ পরিচালনা
SQL Managed Instance	প্রায় সম্পূর্ণ SQL Server — SQL Agent, CLR, cross-database কোয়েরিসহ	অন-প্রিমিসেস SQL Server থেকে মাইগ্রেশন
Azure Database for PostgreSQL	পরিচালিত PostgreSQL — Flexible Server সুপারিশ করা হয়	PostgreSQL-ভিত্তিক অ্যাপ (TechNova-র Prisma)

TechNova PostgreSQL ব্যবহার করে (Prisma-র সাথে) — তাই **Azure Database for PostgreSQL — Flexible Server** আমাদের প্রধান পছন্দ। Flexible Server-এ আপনি Zone-redundant উচ্চ-উপলভ্যতা, স্বয়ংক্রিয় ব্যাকআপ (৭–৩৫ দিন retention), এবং পঠন-রেপ্লিকা (read replica) পান।

১৪.৫ উচ্চ-উপলভ্যতা ও রেপ্লিকা

Zone-redundant high availability মানে প্রাথমিক সার্ভারের একটি standby কপি অন্য Zone-এ থাকে — প্রাথমিক নষ্ট হলে স্বয়ংক্রিয়ভাবে standby সক্রিয় হয় (সাধারণত ৬০–১২০ সেকেন্ডে)। **Read replica** হলো পড়ার জন্য আলাদা কপি — রিপোর্টিং বা পড়া-বেশি কাজে মূল সার্ভারের চাপ কমায়। **Failover Group** (Azure SQL) Region-স্তরে ফেইলওভার দেয় — পুরো Region বন্ধ হলে অন্য Region-এর সার্ভার সক্রিয় হয়। মনে রাখুন: রেপ্লিকা হার্ডওয়্যার ব্যর্থতা থেকে রক্ষা করে, ভুল DELETE কোয়েরি থেকে নয় — তার জন্য Point-in-Time Recovery দরকার।

১৪.৬ ব্যাকআপ ও Point-in-Time Recovery (PITR)

পরিচালিত ডেটাবেস স্বয়ংক্রিয়ভাবে সম্পূর্ণ, পার্থক্যমূলক ও লগ ব্যাকআপ নেয়। **PITR** দিয়ে আপনি যেকোনো নির্দিষ্ট সময়ে (যেমন ভুল DELETE-এর ৫ মিনিট আগে) ডেটাবেস ফিরিয়ে আনতে পারেন — retention সময়ের মধ্যে। **Long-term retention (LTR)** দিয়ে সাপ্তাহিক/মাসিক ব্যাকআপ বছর ধরে রাখা যায় (নিয়ন্ত্রক প্রয়োজনে)। নিয়ম: retention সময় যাচাই করুন, এবং বছরে অন্তত একবার PITR পরীক্ষা করুন।

১৪.৭ নেটওয়ার্ক ও পরিচয় দিয়ে সুরক্ষা

ডেটাবেসের নিরাপত্তার তিনটি স্তর: (১) **নেটওয়ার্ক:** Public access বন্ধ করে Private Endpoint দিয়ে VNet-এর ভেতর থেকে অ্যাক্সেস — এটাই ডিফল্ট হওয়া উচিত। (২) **প্রমাণীকরণ:** PostgreSQL-এ Microsoft Entra প্রমাণীকরণ চালু করুন — পাসওয়ার্ডের বদলে Managed Identity দিয়ে সংযোগ। (৩) **এনক্রিপশন:** Transparent Data Encryption (TDE) ডিফল্টভাবে চালু থাকে; সংবেদনশীল কলামের জন্য **Always Encrypted** বা কলাম-স্তরের এনক্রিপশন বিবেচনা করুন। সংযোগ স্ট্রিং কখনো কোডে লিখবেন না — Key Vault থেকে নিন (অধ্যায় ২২)।

```
# PostgreSQL Flexible Server তৈরি (Zone-redundant, Private Endpoint-সহ)
az postgres flexible-server create --name technova-pg --resource-group technova-prod-rg \
  --location southeastasia --version 16 --tier Burstable --sku-name Standard_B2ms \
  --storage-size 128 --storage-type Premium_LRS --vnet technova-vnet --subnet data-subnet
\
  --private-dns-zone technova-pg.private.postgres.database.azure.com \
  --high-availability ZoneRedundant --zone 1 --standby-zone 2 \
  --backup-retention 14 --geo-redundant-backup Disabled --yes
# ডেটাবেস তৈরি
az postgres flexible-server db create --server-name technova-pg --resource-group
technova-prod-rg --database-name technova_app
```

১৪.৮ সাধারণ ভুল ও ট্রাবলশুটিং

- **ভুল:** ডেটাবেস পাবলিক রেখে ফায়ারওয়ালে 0.0.0.0/0 খোলা — সবচেয়ে বিপজ্জনক ভুল। Private Endpoint ব্যবহার করুন।
- **ভুল:** সংযোগ স্ট্রিংয়ে পাসওয়ার্ড সরাসরি দেওয়া — Managed Identity + Key Vault ব্যবহার করুন।
- **ট্রাবলশুটিং — সংযোগ ব্যর্থ:** (১) Private DNS Zone যুক্ত আছে কি না, (২) NSG 5432 পোর্ট অনুমোদন করে কি না, (৩) PostgreSQL-এ SSL বাধ্যতামূলক কি না (ডিফল্টভাবে হ্যাঁ — সংযোগ স্ট্রিংয়ে sslmode=require দিন), (৪) অ্যাপের সংযোগ পুল শেষ হয়ে গেছে কি না — PgBouncer বিবেচনা করুন।
- **ট্রাবলশুটিং — ধীর কোয়েরি:** Query Store / pg_stat_statements দিয়ে ধীর কোয়েরি খুঁজুন, ইনডেক্স যাচাই করুন, এবং `EXPLAIN ANALYZE` দিয়ে পরিকল্পনা দেখুন।

১৪.১২ সারসংক্ষেপ

পরিচালিত ডেটাবেস Azure-কে দেয় প্যাচ, ব্যাকআপ ও উচ্চ-উপলভ্যতা — কিন্তু নকশা, নেটওয়ার্কিং ও যাচাই আপনার দায়িত্ব। নতুন অ্যাপে Azure SQL Database, মাইগ্রেশনে SQL Managed Instance, PostgreSQL-এ Flexible Server। Zone-redundant HA দিয়ে Zone-স্তরের ব্যর্থতা, PITR দিয়ে ভুল কোয়েরি থেকে পুনরুদ্ধার করুন। ডেটাবেস সবসময় Private Endpoint-এর পেছনে রাখুন, Entra/Managed Identity দিয়ে প্রমাণীকরণ করুন, সংযোগ স্ট্রিং Key Vault-এ রাখুন।

১৪.১৩ পরিভাষা

Azure SQL Database — সম্পূর্ণ পরিচালিত SQL Server সার্ভিস।

SQL Managed Instance — অন-প্রিমিসেস SQL Server-এর প্রায় সম্পূর্ণ পরিচালিত সংস্করণ।

Flexible Server — Azure Database for PostgreSQL-এর সুপারিশকৃত ডিপ্লয়মেন্ট মডেল।

Zone-redundant HA — Standby কপি অন্য Zone-এ — স্বয়ংক্রিয় ফেইলওভার।

Read Replica — পড়ার জন্য আলাদা কপি — মূল সার্ভারের চাপ কমায়।

PITR — Point-in-Time Recovery — নির্দিষ্ট সময়ে ডেটাবেস ফিরিয়ে আনা।

TDE — Transparent Data Encryption — ডিস্কে ডেটার এনক্রিপশন।

১৪.১৪ জ্ঞান যাচাই

1. Azure SQL Database ও SQL Managed Instance-এর মধ্যে পার্থক্য কী? কখন কোনটি?
2. Zone-redundant HA কী থেকে রক্ষা করে, আর কী থেকে করে না?
3. PITR কী? ভুল DELETE কোয়েরির পর কীভাবে পুনরুদ্ধার করবেন?
4. ডেটাবেসের তিনটি নিরাপত্তা স্তর কী কী?
5. Read Replica কখন ব্যবহার করবেন? এটি কি ব্যাকআপের বিকল্প?

১৪.১৫ অনুশীলন

একটি PostgreSQL Flexible Server তৈরি করুন (Burstable, Zone-redundant, Private Endpoint)। একটি ডেটাবেস ও টেবিল তৈরি করে ডেটা ঢোকান, তারপর PITR দিয়ে ১০ মিনিট আগের অবস্থায় ফিরিয়ে আনার পরীক্ষা করুন (test পরিবেশে)।

অধ্যায় ১৫ • PART III

Chapter 15 — Cosmos DB and Caching

১৫.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি পারবেন — (১) Cosmos DB-এর পার্টিশন কী ও কনসিস্টেন্সি স্তর ব্যাখ্যা করতে; (২) Request Unit (RU) দিয়ে ক্ষমতা ও খরচ হিসাব করতে; (৩) Change Feed-এর ব্যবহার ব্যাখ্যা করতে; (৪) Azure Managed Redis দিয়ে ক্যাশিং সাজাতে; এবং (৫) ক্যাশ ইনভ্যালিডেশন কৌশল বাছাই করতে।

১৫.২ পূর্বশর্ত জ্ঞান

অধ্যায় ১৪: রিলেশনাল ডেটাবেস। NoSQL ও কী-ভ্যালু ধারণার সাধারণ জ্ঞান।

১৫.৩ কেন এই বিষয়টি গুরুত্বপূর্ণ

সব ডেটা টেবিলে বসে না — ব্যবহারকারীর সেশন, পণ্যের ক্যাটালগ, IoT টেলিমেট্রি, চ্যাট বার্তা — এসবের জন্য NoSQL ভালো। আর প্রতিটি অনুরোধ ডেটাবেসে গেলে সিস্টেম ধীর হয়ে যায় — ক্যাশিং সেই চাপ কমায়। এই অধ্যায়ে আপনি Azure-এর NoSQL ও ক্যাশিং সার্ভিস শিখবেন।

১৫.৪ প্রথম নীতি থেকে: Cosmos DB কীভাবে স্কেল করে

Azure Cosmos DB হলো বিশ্বজুড়ে বিতরণ করা (globally distributed) NoSQL ডেটাবেস — একই ডেটা একাধিক Region-এ লেখা ও পড়া যায়, একক-অঙ্কের মিলিসেকেন্ড ল্যাটেন্সিসহ। এর মূল ধারণা **পার্টিশন কী (Partition Key)**: প্রতিটি আইটেম একটি পার্টিশন কী-এর মান অনুযায়ী একটি **লজিক্যাল পার্টিশন**-এ যায়; লজিক্যাল পার্টিশনগুলো **ফিজিক্যাল পার্টিশনে** ভাগ হয়। ভালো পার্টিশন কী (যেমন userId) মানে লোড সমানভাবে ছড়ায়; খারাপ পার্টিশন কী (যেমন country — যেখানে একটি দেশে ৯০% ব্যবহারকারী) মানে একটি পার্টিশন গরম (hot partition) হয়ে যায়, বাকিগুলো অলস থাকে।

Cosmos DB-এর **কনসিস্টেন্সি স্তর** পাঁচটি: Strong (সবচেয়ে শক্তিশালী, সবচেয়ে ধীর), Bounded Staleness, Session (একই সেশনে নিজের লেখা দেখা যায় — ডিফল্ট ও সুপারিশকৃত), Consistent Prefix, Eventual (সবচেয়ে দ্রুত, সবচেয়ে দুর্বল)। বেশিরভাগ ওয়েব অ্যাপের জন্য **Session** সঠিক ভারসাম্য দেয়।

১৫.৫ Request Unit (RU): ক্ষমতা ও খরচের একক

Request Unit (RU) হলো Cosmos DB-এর ক্ষমতার একক — একটি ছোট আইটেম পড়তে ১ RU, একটি জটিল কোয়েরিতে শত শত RU লাগতে পারে। আপনি RU/s প্রভিশন করেন (যেমন ৪০০ RU/s) — এর বেশি চাইলে **429 (Too Many Requests)** আসে। খরচ RU/s-এর ওপর নির্ভর করে — তাই কোয়েরি অপ্টিমাইজ করা সরাসরি খরচ কমায়। **Autoscale** মোডে RU/s লোড অনুযায়ী ১০%–১০০% ওঠানামা করে — অনিয়মিত লোডের জন্য ভালো। **Serverless** মোডে শুধু ব্যবহৃত RU-এর জন্য অর্থ দেন — কম ট্র্যাফিকের অ্যাপের জন্য উপযোগী।

১৫.৬ Change Feed ও ইনডেক্সিং

Change Feed হলো Cosmos DB-তে পরিবর্তনের ধারা — নতুন বা বদলানো আইটেমের তালিকা ক্রমানুসারে পাওয়া যায়। এটি দিয়ে করা যায়: ক্যাশ আপডেট, সার্চ ইনডেক্স আপডেট, অন্য সিস্টেমে ডেটা পাঠানো (ইভেন্ট-চালিত আর্কিটেকচার)। Cosmos DB ডিফল্টভাবে সব প্রপার্টি ইনডেক্স করে — দ্রুত কোয়েরির জন্য ভালো, কিন্তু লেখার খরচ বাড়ে। শুধু প্রয়োজনীয় পথ ইনডেক্স করে খরচ কমানো যায়।

১৫.৭ Azure Managed Redis: ক্যাশিং

Azure Managed Redis (পূর্বে Azure Cache for Redis) হলো ইন-মেমরি ডেটা স্টোর — সেশন, ক্যাশ, লিডারবোর্ড, রেট লিমিটিংয়ের জন্য। Redis-এ ডেটা RAM-এ থাকে — পড়া মাইক্রোসেকেন্ডে হয়, ডেটাবেসের মিলিসেকেন্ডের তুলনায় ১০০ গুণ দ্রুত। ক্যাশিং কৌশল: **Cache-aside** (অ্যাপ প্রথমে ক্যাশ দেখে, না পেলে ডেটাবেস থেকে এনে ক্যাশে রাখে — সবচেয়ে সাধারণ), **Write-through** (লেখা ক্যাশ ও ডেটাবেস দুটোতেই যায়)।

> **WARNING:** ক্যাশ **কখনোই একমাত্র সত্যের উৎস (source of truth) নয়** — ক্যাশ মুছে গেলে ডেটা ডেটাবেস থেকে ফিরিয়ে আনা যায়। ক্যাশে সংবেদনশীল ডেটা (পাসওয়ার্ড, টোকেন) রাখবেন না, এবং TTL (Time to Live) সবসময় সেট করুন — নইলে পুরোনো ডেটা চিরকাল থেকে যাবে। **Cache stampede** থেকে বাঁচতে জনপ্রিয় কী-এর মেয়াদ একসাথে শেষ হতে দেবেন না (jitter যোগ করুন)।

```
# Cosmos DB অ্যাকাউন্ট তৈরি (Session consistency, Southeast Asia)
az cosmosdb create --name technova-cosmos --resource-group technova-prod-rg \
  --locations regionName=southeastasia failoverPriority=0 --default-consistency-level
Session
# ডেটাবেস ও কনটেইনার (পার্টিশন কী /userId, Autoscale 4000 RU/s)
az cosmosdb sql database create --account-name technova-cosmos --resource-group technova-
prod-rg --name technova_app
az cosmosdb sql container create --account-name technova-cosmos --resource-group
technova-prod-rg \
  --database-name technova_app --name sessions --partition-key-path /userId --max-
throughput 4000
# Azure Managed Redis তৈরি (Basic tier — উন্নয়নের জন্য)
az redis create --name technova-redis --resource-group technova-prod-rg \
  --location southeastasia --sku Basic --vm-size c0 --enable-non-ssl-port false
```

১৫.৮ সাধারণ ভুল ও ট্রাবলশুটিং

- **ভুল:** খারাপ পার্টিশন কী বাছাই — hot partition তৈরি হয়, ৪২৯ ত্রুটি আসে, খরচ বাড়ে। পার্টিশন কী বাছাইয়ের আগে অ্যাক্সেস প্যাটার্ন লিখুন।
- **ভুল:** ক্যাশে TTL না দেওয়া — পুরোনো ডেটা চিরকাল থাকে, ব্যবহারকারী ভুল তথ্য দেখে।
- **ট্রাবলশুটিং — ৪২৯ ত্রুটি:** RU/s বাড়ান, কোয়েরি অপ্টিমাইজ করুন (পার্টিশন কী দিয়ে ফিল্টার করুন), অথবা Autoscale চালু করুন। SDK-তে retry নীতি আছে কি না দেখুন।

১৫.১২ সারসংক্ষেপ

Cosmos DB বিশ্বজুড়ে বিতরণ করা NoSQL — পার্টিশন কী দিয়ে স্কেল করে, কনসিস্টেন্সি স্তর দিয়ে গতি বনাম সঠিকতার ভারসাম্য দেয় (Session ডিফল্ট)। RU/s হলো ক্ষমতা ও খরচের একক — কোয়েরি অপ্টিমাইজ করা মানে খরচ কমানো। Redis দিয়ে ক্যাশিং করুন — Cache-aside কৌশলে, TTL-সহ, এবং ক্যাশকে কখনোই একমাত্র সত্যের উৎস বানাবেন না।

১৫.১৩ পরিভাষা

Cosmos DB — বিশ্বজুড়ে বিতরণ করা NoSQL ডেটাবেস — একক-অঙ্কের মিলিসেকেন্ড ল্যাটেন্সি।

Partition Key — আইটেম কোন লজিক্যাল পার্টিশনে যাবে তা নির্ধারণ করে।

Hot Partition — অতিরিক্ত লোড পাওয়া পার্টিশন — খারাপ পার্টিশন কী-এর লক্ষণ।

Request Unit (RU) — Cosmos DB-এর ক্ষমতার একক — পড়া/লেখা/কোয়েরির খরচ মাপে।

Consistency Level — বিতরণ করা ডেটায় কতটুকু সঠিকতা নিশ্চিত করা হবে — Strong থেকে Eventual।

Change Feed — Cosmos DB-তে পরিবর্তনের ক্রমানুসারী ধারা।

Cache-aside — অ্যাপ প্রথমে ক্যাশ দেখে, না পেলে ডেটাবেস থেকে এনে ক্যাশে রাখে।

TTL — Time to Live — ক্যাশ আইটেমের মেয়াদ।

১৫.১৪ জ্ঞান যাচাই

1. ভালো ও খারাপ পার্টিশন কী-এর উদাহরণ দিন। Hot partition কীভাবে তৈরি হয়?
2. Session consistency কেন বেশিরভাগ ওয়েব অ্যাপের জন্য সঠিক পছন্দ?
3. RU/s কী? একটি কোয়েরিতে বেশি RU লাগলে খরচের ওপর কী প্রভাব পড়ে?
4. Cache-aside ও Write-through কৌশলের পার্থক্য কী?
5. ক্যাশকে একমাত্র সত্যের উৎস বানানো কেন বিপজ্জনক?

১৫.১৫ অনুশীলন

একটি Cosmos DB কনটেইনার তৈরি করুন (পার্টিশন কী /userId)। কয়েকটি আইটেম ঢুকিয়ে পার্টিশন কী দিয়ে ও ছাড়া কোয়েরি চালিয়ে RU খরচের পার্থক্য দেখুন (Portal-এর Data Explorer-এ)। একটি Redis ক্যাশ তৈরি করে Node.js থেকে একটি মান লিখে TTL-সহ পড়ুন।

অধ্যায় ১৬ • PART III

Chapter 16 — Application Hosting and Edge Delivery

১৬.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি পারবেন — (১) App Service-এ Node.js অ্যাপ ডিপ্লয় করতে; (২) Deployment Slot দিয়ে নিরাপদ রিলিজ করতে; (৩) Static Web Apps দিয়ে ফ্রন্টএন্ড হোস্ট করতে; (৪) Front Door ও CDN দিয়ে বিশ্বজুড়ে ডেলিভারি সাজাতে; এবং (৫) কাস্টম ডোমেইন ও HTTPS কনফিগার করতে।

১৬.২ পূর্বশর্ত জ্ঞান

অধ্যায় ৮ ও ১৪: DNS, Load Balancing, ডেটাবেস। Node.js অ্যাপ চালানোর অভিজ্ঞতা।

১৬.৩ কেন এই বিষয়টি গুরুত্বপূর্ণ

আপনার অ্যাপ তৈরি — এখন তা ব্যবহারকারীর কাছে পৌঁছে দিতে হবে: দ্রুত, নিরাপদে, HTTPS-সহ। Azure-এ এর জন্য PaaS হোস্টিং (App Service), স্ট্যাটিক হোস্টিং (Static Web Apps) ও এজ ডেলিভারি (Front Door/CDN) আছে। এই অধ্যায়ে আপনি TechNova-র MERN অ্যাপকে ইন্টারনেটে আনার পথ শিখবেন।

১৬.৪ প্রথম নীতি থেকে: PaaS হোস্টিং কী দেয়

Azure App Service হলো ওয়েব অ্যাপের জন্য PaaS — আপনি কোড দেন, Azure OS, রানটাইম, প্যাচ, স্কেলিং ও লোড ব্যালান্সিং দেখে। App Service Plan নির্ধারণ করে ক্ষমতা (যেমন B1, P1v3) — একই Plan-এ একাধিক অ্যাপ চলতে পারে। **Static Web Apps** হলো ফ্রন্টএন্ডের (React/Next.js স্ট্যাটিক অংশ) জন্য সার্ভারলেস হোস্টিং — GitHub থেকে স্বয়ংক্রিয় ডিপ্লয়, বিনামূল্যে SSL, গ্লোবাল CDN অন্তর্ভুক্ত।

১৬.৫ Deployment Slot: নিরাপদ রিলিজের মূল সরঞ্জাম

Deployment Slot হলো একই অ্যাপের একটি অতিরিক্ত পরিবেশ — যেমন production (app) ও staging (app/staging)। নতুন সংস্করণ প্রথমে staging স্লটে ডিপ্লয় করুন, সেখানে পরীক্ষা করুন, তারপর **swap** করুন — ট্র্যাফিক মুহূর্তের মধ্যে নতুন সংস্করণে চলে যায়, ডাউনটাইম ছাড়া। সমস্যা হলে আবার swap করে ফিরে যান — এটাই **rollback**। Slot-এ **slot setting** চিহ্নিত করা যায় — যেমন ডেটাবেস সংযোগ স্ট্রিং staging-এ আলাদা থাকে, swap-এর সময় বদলায় না।

১৬.৬ Front Door ও CDN: ব্যবহারকারীর কাছে দ্রুত পৌঁছানো

Azure Front Door হলো গ্লোবাল এন্ট্রি পয়েন্ট — ব্যবহারকারীর নিকটতম Edge থেকে অনুরোধ নেয়, দ্রুততম ব্যাকএন্ডে পাঠায়, ক্যাশ করে, WAF দিয়ে সুরক্ষা দেয়। **CDN (Content Delivery Network)** স্ট্যাটিক অ্যাসেট (ছবি, CSS, JS) Edge-এ ক্যাশ করে — ব্যবহারকারী ঢাকায় থাকলে সিঙ্গাপুরের সার্ভার থেকে নয়, কাছের Edge থেকে ফাইল পায়। মনে রাখুন: Azure-এর ক্লাসিক CDN প্রোফাইল (Microsoft CDN) এখন Front Door-এর অংশ হিসেবে সুপারিশ করা হয় — নতুন নকশায় Front Door ব্যবহার করুন।

১৬.৭ কাস্টম ডোমেইন ও HTTPS

কাস্টম ডোমেইন (app.technova.example) যোগ করতে DNS-এ CNAME/TXT রেকর্ড দিতে হয় (মালিকানা প্রমাণের জন্য), তারপর App Service-এ ডোমেইন বাঁধতে হয়। HTTPS-এর জন্য **App Service Managed Certificate** (বিনামূল্যে, স্বয়ংক্রিয় নবায়ন) অথবা Key Vault-এর সার্টিফিকেট ব্যবহার করুন। **HTTPS Only** চালু করুন — HTTP অনুরোধ স্বয়ংক্রিয়ভাবে HTTPS-এ যাবে।

```
# App Service Plan ও Web App তৈরি (Linux, Node.js 20)
az appservice plan create --name technova-plan --resource-group technova-prod-rg --
location southeastasia --sku B1 --is-linux
az webapp create --name technova-api --resource-group technova-prod-rg --plan technova-
plan --runtime "NODE:20-lts"
# Staging slot তৈরি ও সেটিং চিহ্নিত করুন
az webapp deployment slot create --name technova-api --resource-group technova-prod-rg --
slot staging
az webapp config appsettings set --name technova-api --resource-group technova-prod-rg --
slot staging \
  --settings NODE_ENV=staging --slot-settings DATABASE_URL="CHANGE-ME-STAGING-URL"
# ZIP দিয়ে ডিপ্লয় করুন
az webapp deploy --name technova-api --resource-group technova-prod-rg --src-path ./
api.zip --type zip
# Slot swap (staging → production)
az webapp deployment slot swap --name technova-api --resource-group technova-prod-rg --
slot staging --target-slot production
```

১৬.৮ সাধারণ ভুল ও ট্রাবলশুটিং

- **ভুল:** Slot setting চিহ্নিত না করা — swap-এর সময় staging-এর ডেটাবেস সংযোগ production-এ চলে যায়, production অ্যাপ staging ডেটাবেসে লেখে।
- **ভুল:** Free/Shared Plan-এ প্রোডাকশন চালানো — এতে Always On নেই, অ্যাপ ঘুমিয়ে যায় (cold start)। প্রোডাকশনে অন্তত Basic (B1) ব্যবহার করুন।
- **ট্রাবলশুটিং — অ্যাপ ৫০০ ত্রুটি দিচ্ছে:** Log Stream (`az webapp log tail`) দেখুন, Application Settings-এ পরিবেশ ভেরিয়েবল ঠিক আছে কি না দেখুন, এবং Kudu কনসোলে (https://<app>.scm.azurewebsites.net) ফাইল ও প্রক্রিয়া যাচাই করুন।

১৬.১২ সারসংক্ষেপ

App Service PaaS হোস্টিং দেয় — কোড দিন, Azure বাকিটা দেখবে। Deployment Slot দিয়ে swap-ভিত্তিক নিরাপদ রিলিজ করুন, এবং slot setting চিহ্নিত করতে ভুলবেন না। Static Web Apps ফ্রন্টএন্ডের জন্য সেরা। Front Door দিয়ে গ্লোবাল ডেলিভারি, ক্যাশিং ও WAF পান। কাস্টম ডোমেইনে Managed Certificate ও HTTPS Only ব্যবহার করুন।

১৬.১৩ পরিভাষা

App Service — ওয়েব অ্যাপের জন্য PaaS হোস্টিং — Node.js, Python, .NET ইত্যাদি।

App Service Plan — App Service-এর ক্ষমতার একক — SKU দিয়ে CPU/RAM নির্ধারিত হয়।

Deployment Slot — একই অ্যাপের অতিরিক্ত পরিবেশ — staging, swap ও rollback-এর জন্য।

Slot Setting — Swap-এর সময় স্লটের সাথে থেকে যাওয়া সেটিং — যেমন ডেটাবেস সংযোগ।

Static Web Apps — ফ্রন্টএন্ডের জন্য সার্ভারলেস হোস্টিং — GitHub থেকে স্বয়ংক্রিয় ডিপ্লয়।

Front Door — গ্লোবাল এন্ট্রি পয়েন্ট — রাউটিং, ক্যাশিং, WAF।

CDN — Content Delivery Network — স্ট্যাটিক অ্যাসেট Edge-এ ক্যাশ করে।

১৬.১৪ জ্ঞান যাচাই

1. App Service Plan ও Web App-এর মধ্যে পার্থক্য কী?
2. Deployment Slot কীভাবে ডাউনটাইম ছাড়া রিলিজ দেয়? Rollback কীভাবে কাজ করে?
3. Slot setting চিহ্নিত না করলে কী বিপদ হতে পারে?
4. Static Web Apps কখন App Service-এর চেয়ে ভালো পছন্দ?
5. Front Door ও Application Gateway-এর মধ্যে পার্থক্য কী?

১৬.১৫ অনুশীলন

একটি App Service Web App তৈরি করুন (B1, Node.js 20)। একটি staging slot তৈরি করুন, একটি ছোট Node.js অ্যাপ ডিপ্লয় করুন, তারপর swap করে দেখুন। HTTPS Only চালু করুন এবং Log Stream দিয়ে লগ দেখুন।

PART IV • DEVOPS ENGINEERING MASTERY

Containers, Serverless & Delivery

Azure কনটেইনার ইঞ্জিনিয়ারিং — ACR, Container Apps ও AKS — Serverless ও মেসেজিং, Azure DevOps ও CI/CD, Azure-এ Infrastructure as Code।

অধ্যায় ১৭ • PART IV

Chapter 17 — Azure Container Engineering

১৭.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি পারবেন — (১) Azure Container Registry (ACR)-এ ইমেজ সংরক্ষণ ও স্ক্যান করতে; (২) Azure Container Apps ও AKS-এর মধ্যে বাছাই করতে; (৩) AKS-এর নোড পুল, নেটওয়ার্কিং ও ইনগ্রেস ব্যাখ্যা করতে; (৪) Workload Identity দিয়ে পড থেকে Azure রিসোর্সে অ্যাক্সেস দিতে; এবং (৫) কনটেইনার আপগ্রেড ও খরচ পরিচালনা করতে।

১৭.২ পূর্বশর্ত জ্ঞান

Volume 6 (Docker) ও Volume 7 (Kubernetes)-এর জ্ঞান। কনটেইনার ইমেজ, পড, ডিপ্লয়মেন্ট ও সার্ভিসের ধারণা।

১৭.৩ কেন এই বিষয়টি গুরুত্বপূর্ণ

Docker ও Kubernetes শিখেছেন — এখন সেগুলো Azure-এ চালানোর পালা। Azure-এ কনটেইনার চালানোর তিনটি পথ আছে: সহজতম (Container Apps), সম্পূর্ণ নিয়ন্ত্রণ (AKS), এবং মাঝামাঝি (Container Instances)। ভুল পথ বাছাই করলে হয় অযথা পরিচালনা-ভার, নয়তো প্রয়োজনের সময় নিয়ন্ত্রণের অভাব।

১৭.৪ প্রথম নীতি থেকে: তিনটি কনটেইনার পথ

সার্ভিস	কী	কখন
Azure Container Registry (ACR)	প্রাইভেট Docker রেজিস্ট্রি — ইমেজ সংরক্ষণ, স্ক্যান, ভূ-প্রতিলিপি	সব কনটেইনার কাজের ভিত্তি
Azure Container Apps (ACA)	সার্ভারলেস কনটেইনার — KEDA অটোস্কেল, Dapr, Ingress অন্তর্ভুক্ত	মাইক্রোসার্ভিস, API, ব্যাকগ্রাউন্ড কাজ — কম পরিচালনা
Azure Kubernetes Service (AKS)	পরিচালিত Kubernetes — কন্ট্রোল প্লেন Azure দেখে	জটিল অর্কেস্ট্রেশন, পূর্ণ Kubernetes নিয়ন্ত্রণ
Container Instances (ACI)	একক কনটেইনার, সার্ভার ছাড়া	এককালীন কাজ, পরীক্ষা

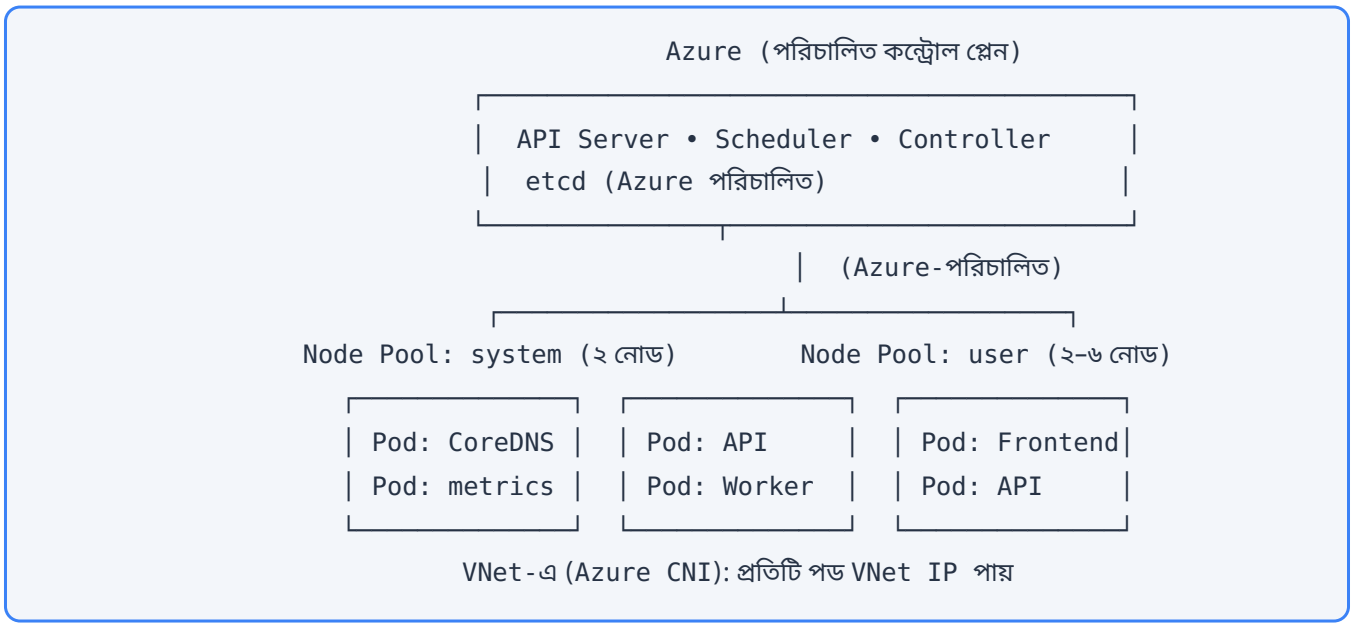
নিয়ম: শুরুতে **Container Apps** দিয়ে দেখুন — বেশিরভাগ MERN অ্যাপের জন্য এটাই যথেষ্ট। যখন Kubernetes-এর বিশেষ সুবিধা (কাস্টম অপারেটর, জটিল নেটওয়ার্ক পলিসি, মাল্টি-টেন্যান্ট) দরকার হবে, তখন **AKS**-এ যান। AKS-এর কন্ট্রোল প্লেন পরিচালিত হলেও নোড পুল, আপগ্রেড ও নেটওয়ার্কিং আপনার দায়িত্ব — এটি “সার্ভারলেস” নয়।

১৭.৫ ACR: ইমেজের নিরাপদ ঘর

ACR-এ ইমেজ সংরক্ষণ করুন — Docker Hub-এর পাবলিক ইমেজের ওপর নির্ভর করবেন না। ACR-এর সুবিধা: **ভূ-প্রতিলিপি (geo-replication)** — ইমেজ একাধিক Region-এ থাকে, টান দ্রুত হয়; **ইমেজ স্ক্যান** — Microsoft Defender দিয়ে দুর্বলতা খোঁজা; **ACR Tasks** — Git কমিটে স্বয়ংক্রিয়ভাবে ইমেজ বানানো; **Private Endpoint** — রেজিস্ট্রি শুধু

VNet থেকে অ্যাক্সেসযোগ্য। AKS থেকে ACR-এ টান দিতে **ACR-সংযুক্তি (attach-acr)** ব্যবহার করুন — এতে AKS-এর kubelet পরিচয়কে ACR-এ AcrPull ভূমিকা দেওয়া হয়, কোনো secret লাগে না।

১৭.৬ AKS আর্কিটেকচার



নোড পুল হলো একই কনফিগারেশনের নোডের গোষ্ঠী। **System নোড পুল**-এ ক্লাস্টারের নিজস্ব পড (CoreDNS, metrics) চলে; **User নোড পুল**-এ আপনার অ্যাপ চলে। নোড পুল আলাদা রাখলে অ্যাপের লোডে সিস্টেম পড প্রভাবিত হয় না। **Azure CNI** নেটওয়ার্কিংয়ে প্রতিটি পড VNet-এর IP পায় — VNet-এর অন্য রিসোর্সের সাথে সরাসরি যোগাযোগ করা যায়, এবং NSG দিয়ে পড-স্তরে ফিল্টার করা যায়।

১৭.৭ Workload Identity ও নিরাপত্তা

Workload Identity হলো পড থেকে Azure রিসোর্সে অ্যাক্সেসের আধুনিক পদ্ধতি: একটি Kubernetes ServiceAccount-কে একটি Entra ID Managed Identity-র সাথে যুক্ত করা হয় — পড OIDC টোকেন দিয়ে Azure-এর টোকেন নেয়, কোনো secret ছাড়াই। পুরোনো পদ্ধতি (পডে secret হিসেবে connection string রাখা) এড়িয়ে চলুন। অন্যান্য নিরাপত্তা: **Azure Policy for Kubernetes** দিয়ে "শুধু ACR-এর ইমেজ চলবে" নিয়ম, **Defender for Containers** দিয়ে রানটাইম সুরক্ষা, এবং **Secret Store CSI Driver** দিয়ে Key Vault-এর secret পডে মাউন্ট করা।

```
# ACR তৈরি ও ইমেজ পুশ করুন
az acr create --name technovaacr --resource-group technova-prod-rg --location
southeastasia --sku Standard --admin-enabled false
az acr build --registry technovaacr --image technova-api:v1 ./api
# AKS ক্লাস্টার তৈরি (2টি সিস্টেম নোড, 2টি ইউজার নোড, ACR সংযুক্ত)
az aks create --name technova-aks --resource-group technova-prod-rg --location
southeastasia \
  --node-count 2 --node-vm-size Standard_B2s --zones 1 2 --network-plugin azure \
  --attach-acr technovaacr --enable-managed-identity --enable-workload-identity --
generate-ssh-keys
# kubectl কনফিগার করুন
az aks get-credentials --name technova-aks --resource-group technova-prod-rg
kubectl get nodes
```

১৭.৮ সাধারণ ভুল ও ট্রাবলশুটিং

- **ভুল:** AKS-এ একটি মাত্র নোড পুলে সিস্টেম ও অ্যাপ পড একসাথে রাখা — অ্যাপের লোডে CoreDNS প্রভাবিত হয়।
- **ভুল:** ACR-এ admin user চালু রাখা — এতে একটি স্থির পাসওয়ার্ড থাকে; বদলে Managed Identity/Workload Identity ব্যবহার করুন।
- **ট্রাবলশুটিং — পড ImagePullBackOff:** AKS-এর ACR-এ AcrPull ভূমিকা আছে কি না দেখুন (`az role assignment list`), ইমেজের ট্যাগ সঠিক কি না, এবং ACR-এ Private Endpoint থাকলে নেটওয়ার্ক পথ যাচাই করুন।

১৭.১২ সারসংক্ষেপ

ACR হলো ইমেজের নিরাপদ ঘর — geo-replication, স্ক্যান, Private Endpointসহ। Container Apps বেশিরভাগ মাইক্রোসার্ভিসের জন্য যথেষ্ট; AKS তখনই নিন যখন পূর্ণ Kubernetes নিয়ন্ত্রণ দরকার। AKS-এ system ও user নোড পুল আলাদা রাখুন, Azure CNI দিয়ে পডকে VNet IP দিন, এবং Workload Identity দিয়ে পড থেকে secret ছাড়া Azure রিসোর্সে অ্যাক্সেস করুন।

১৭.১৩ পরিভাষা

ACR — Azure Container Registry — প্রাইভেট Docker রেজিস্ট্রি।

Container Apps — সার্ভারলেস কনটেইনার প্ল্যাটফর্ম — অটোস্কেল ও Ingress অন্তর্ভুক্ত।

AKS — Azure Kubernetes Service — পরিচালিত Kubernetes।

Node Pool — একই কনফিগারেশনের AKS নোডের গোষ্ঠী — system ও user পুল আলাদা।

Azure CNI — AKS নেটওয়ার্কিং — প্রতিটি পড VNet IP পায়।

Workload Identity — পড থেকে OIDC টোকেন দিয়ে Azure-এ প্রমাণীকরণ — secret ছাড়া।

১৭.১৪ জ্ঞান যাচাই

1. Container Apps ও AKS-এর মধ্যে পার্থক্য কী? কখন কোনটি বাছাই করবেন?
2. ACR-এ ইমেজ স্ক্যান ও geo-replication কেন গুরুত্বপূর্ণ?
3. System ও User নোড পুল আলাদা রাখা কেন জরুরি?
4. Workload Identity কী সমস্যার সমাধান করে?
5. পড ImagePullBackOff হলে কোন তিনটি জিনিস যাচাই করবেন?

১৭.১৫ অনুশীলন

একটি ACR তৈরি করে একটি Node.js অ্যাপের ইমেজ বানিয়ে পুশ করুন। একটি AKS ক্লাস্টার তৈরি করে (test, ছোট সাইজ) ইমেজটি ডিপ্লয় করুন। কাজ শেষে ক্লাস্টার মুছে ফেলুন — AKS নোডের চার্জ বন্ধ করতে ভুলবেন না।

অধ্যায় ১৮ • PART IV

Chapter 18 — Serverless and Messaging

১৮.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি পারবেন — (১) Azure Functions-এর ট্রিগার ও বাইন্ডিং ব্যাখ্যা করতে; (২) API Management দিয়ে API প্রকাশ করতে; (৩) Service Bus ও Event Grid-এর মধ্যে বাছাই করতে; (৪) পুনঃচেষ্টা, Dead-Letter Queue ও Idempotency সাজাতে; এবং (৫) সার্ভারলেস খরচ ও নিরাপত্তা পরিচালনা করতে।

১৮.২ পূর্বশর্ত জ্ঞান

অধ্যায় ১৪–১৫: ডেটাবেস, ক্যাশিং। ইভেন্ট-চালিত নকশার সাধারণ ধারণা।

১৮.৩ কেন এই বিষয়টি গুরুত্বপূর্ণ

সব কাজ সবসময় চলমান সার্ভারে করার দরকার নেই — একটি ফাইল আপলোড হলে থাম্বনেইল বানানো, একটি অর্ডার এলে ইমেইল পাঠানো, একটি বার্তা এলে তা প্রসেস করা — এসব কাজ ঘটনা (event) ঘটলে হয়, তারপর থেমে যায়। সার্ভারলেস মানে সার্ভার নেই তা নয় — মানে সার্ভার পরিচালনার ভার আপনার নয়।

১৮.৪ প্রথম নীতি থেকে: Function কীভাবে চলে

Azure Functions হলো ইভেন্ট-চালিত কোড — একটি **ট্রিগার** (যেমন HTTP অনুরোধ, Blob আপলোড, Queue বার্তা, টাইমার) ফাংশনটি চালায়, ফাংশন কাজ করে, তারপর থেমে যায়। **বাইন্ডিং** দিয়ে ফাংশন ইনপুট নেয় (যেমন Blob-এর বিষয়বস্তু) ও আউটপুট দেয় (যেমন Cosmos DB-তে লেখা) — সংযোগের কোড লিখতে হয় না। হোস্টিং পরিকল্পনা: **Consumption** (শুধু চলার সময়ের জন্য অর্থ — প্রথম ১০ লাখ এক্সিকিউশন বিনামূল্যে), **Premium** (কোল্ড স্টার্ট নেই, VNet সংযোগ), **Dedicated (App Service Plan)**।

> **CONCEPT: কোল্ড স্টার্ট (Cold Start):** Consumption পরিকল্পনায় অনেকক্ষণ না চলা ফাংশন প্রথম অনুরোধে দেরিতে সাড়া দেয় — কারণ রানটাইম চালু হতে সময় লাগে। ব্যবহারকারী-মুখী API-তে এটি সমস্যা হতে পারে — তখন Premium পরিকল্পনা (pre-warmed instance) ব্যবহার করুন।

১৮.৫ Service Bus বনাম Event Grid

বিষয়	Service Bus	Event Grid
মডেল	বার্তা সারি/বিষয় — গ্রাহক টেনে নেয় (pull)	ইভেন্ট রাউটিং — প্রকাশক পাঠায়, গ্রাহক পায় (push)
গ্যারান্টি	FIFO, ডুপ্লিকেট শনাক্তকরণ, লেনদেন	অন্তত একবার ডেলিভারি (at-least-once)
ব্যবহার	অর্ডার প্রসেসিং, কাজের সারি	Blob আপলোড হলে প্রতিক্রিয়া, রিসোর্স পরিবর্তনের বিজ্ঞপ্তি
ধারণক্ষমতা	বড় বার্তা, দীর্ঘ ধারণ	ছোট ইভেন্ট, দ্রুত

সহজ নিয়ম: **কাজের সারি** দরকার হলে Service Bus (অর্ডার প্রসেস করা, ইমেইল পাঠানো); **ঘটনার বিজ্ঞপ্তি** দরকার হলে Event Grid (ফাইল আপলোড হলে থাম্বনেইল বানানো)। **API Management (APIM)** হলো API-এর সামনের দরজা — রেট লিমিটিং, প্রমাণীকরণ, ক্যাশিং, সংস্করণ ব্যবস্থাপনা, ডেভেলপার পোর্টাল।

১৮.৬ পুনঃচেষ্টা, Dead-Letter ও Idempotency

বিতরণ করা সিস্টেমে ব্যর্থতা স্বাভাবিক — নেটওয়ার্ক ছিঁড়বে, ডেটাবেস ব্যস্ত থাকবে। তাই প্রতিটি বার্তা প্রসেসিংয়ে তিনটি জিনিস দরকার: (১) **পুনঃচেষ্টা (Retry)** — ব্যর্থ হলে কিছুক্ষণ পর আবার চেষ্টা, exponential backoff-সহ (১ সেকেন্ড, ২ সেকেন্ড, ৪ সেকেন্ড...); (২) **Dead-Letter Queue (DLQ)** — নির্দিষ্টবার ব্যর্থ হলে বার্তাটি আলাদা সারিতে যায়, যাতে মূল সারি আটকে না যায় এবং পরে তদন্ত করা যায়; (৩) **Idempotency** — একই বার্তা দুবার প্রসেস হলেও ফল একবারের মতো হওয়া — যেমন অর্ডার ID দিয়ে ডুপ্লিকেট পরীক্ষা করা। Event Grid at-least-once ডেলিভারি দেয় — অর্থাৎ একই ইভেন্ট দুবার আসতে পারে, তাই idempotency বাধ্যতামূলক।

```
# একটি Service Bus নেমস্পেস ও সারি তৈরি করুন
az servicebus namespace create --name technova-sb --resource-group technova-prod-rg --
location southeastasia --sku Standard
az servicebus queue create --name orders --namespace-name technova-sb --resource-group
technova-prod-rg \
  --max-delivery-count 3 --dead-lettering-on-message-expiration true
# একটি Function App তৈরি করুন (Consumption, Node.js 20)
az functionapp create --name technova-order-fn --resource-group technova-prod-rg \
  --consumption-plan-location southeastasia --runtime node --runtime-version 20 \
  --functions-version 4 --storage-account technovastore001 --os-type linux
```

১৮.৭ সাধারণ ভুল ও ট্রাবলশুটিং

- **ভুল:** Function-এ দীর্ঘ কাজ (৫ মিনিটের বেশি) করা — Consumption পরিকল্পনায় সময়সীমা আছে; দীর্ঘ কাজ Durable Functions বা Container Apps-এ নিন।
- **ভুল:** Idempotency ছাড়া ইভেন্ট প্রসেস করা — একই অর্ডার দুবার চার্জ হতে পারে।
- **ট্রাবলশুটিং — Function চলছে না:** Application Insights-এ লগ দেখুন, ট্রিগারের সংযোগ স্ট্রিং (Key Vault-এ আছে কি না) যাচাই করুন, এবং Function App-এর “Diagnose and solve problems” ব্যবহার করুন।

১৮.১২ সারসংক্ষেপ

Azure Functions ইভেন্ট-চালিত কোড — ট্রিগার চালায়, বাইন্ডিং সংযোগ দেয়। Consumption পরিকল্পনা সস্তা কিন্তু কোল্ড স্টার্ট আছে; ব্যবহারকারী-মুখী API-তে Premium বিবেচনা করুন। কাজের সারির জন্য Service Bus, ঘটনার বিজ্ঞপ্তির জন্য Event Grid। প্রতিটি বার্তা প্রসেসিংয়ে retry, Dead-Letter Queue ও idempotency — এই তিনটি বাধ্যতামূলক।

১৮.১৩ পরিভাষা

Azure Functions — ইভেন্ট-চালিত সার্ভারলেস কোড — ট্রিগার ও বাইন্ডিংসহ।

Trigger — ফাংশন চালু করে এমন ঘটনা — HTTP, Queue, Blob, Timer।

Binding — ফাংশনের ইনপুট/আউটপুট সংযোগ — কোড ছাড়াই ডেটা আসা-যাওয়া।

Service Bus — বার্তা সারি ও বিষয় — নির্ভরযোগ্য বার্তা প্রসেসিং।

Event Grid — ইভেন্ট রাউটিং সেবা — ঘটনা ঘটলে গ্রাহকদের পাঠায়।

Dead-Letter Queue — নির্দিষ্টবার ব্যর্থ বার্তার আলাদা সারি — তদন্তের জন্য।

Idempotency — একই কাজ দুবার করলেও ফল একবারের মতো হওয়ার বৈশিষ্ট্য।

Cold Start — অনেকক্ষণ না চলা ফাংশনের প্রথম অনুরোধে দেরি।

১৮.১৪ জ্ঞান যাচাই

1. Service Bus ও Event Grid-এর মধ্যে পার্থক্য কী? প্রতিটির একটি ব্যবহার লিখুন।
2. Consumption ও Premium পরিকল্পনার পার্থক্য কী? কোল্ড স্টার্ট কখন সমস্যা?
3. Dead-Letter Queue কেন দরকার? এটি ছাড়া কী সমস্যা হয়?
4. Idempotency কী এবং at-least-once ডেলিভারির সাথে এর সম্পর্ক কী?
5. API Management কী সমস্যার সমাধান করে?

১৮.১৫ অনুশীলন

একটি Service Bus সারি তৈরি করুন (max delivery count 3, DLQ চালু)। একটি Function তৈরি করুন যা সারি থেকে বার্তা পড়ে Application Insights-এ লগ লেখে। একটি বার্তা পাঠিয়ে দেখুন তা প্রসেস হয় কি না, তারপর ইচ্ছাকৃতভাবে ব্যর্থ করে DLQ-তে যাওয়া পর্যবেক্ষণ করুন।

অধ্যায় ১৯ • PART IV

Chapter 19 — Azure DevOps and CI/CD

১৯.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি পারবেন — (১) Azure Pipelines-এ YAML পাইপলাইন লিখতে; (২) Environment ও Approval দিয়ে নিরাপদ রিলিজ সাজাতে; (৩) GitHub Actions থেকে Workload Identity Federation দিয়ে Azure-এ ডিপ্লয় করতে; (৪) আর্টিফ্যাক্ট ও ভেরিয়েবল গ্রুপ পরিচালনা করতে; এবং (৫) রোলব্যাক কৌশল বাছাই করতে।

১৯.২ পূর্বশর্ত জ্ঞান

Volume 5 (CI/CD) ও অধ্যায় ৩–৪: RBAC, Managed Identity, CLI। YAML ও Git-এর জ্ঞান।

১৯.৩ কেন এই বিষয়টি গুরুত্বপূর্ণ

কোড লেখা অর্ধেক কাজ — বাকি অর্ধেক হলো তা নিরাপদে, বারবার, একইভাবে প্রোডাকশনে পৌঁছে দেওয়া। Azure-এ CI/CD-এর দুটি প্রধান পথ: Azure DevOps (Pipelines, Repos, Artifacts, Boards) ও GitHub Actions। এই অধ্যায়ে আপনি দুটোই শিখবেন — এবং সবচেয়ে গুরুত্বপূর্ণভাবে, secret ছাড়া নিরাপদ প্রমাণীকরণ।

১৯.৪ প্রথম নীতি থেকে: পাইপলাইনের পর্যায়

একটি CI/CD পাইপলাইনের পর্যায়: **Build** (কোড কম্পাইল, টেস্ট, আর্টিফ্যাক্ট তৈরি) → **Deploy to staging** (staging-এ ডিপ্লয়, স্বয়ংক্রিয় পরীক্ষা) → **Approval** (মানুষের অনুমোদন) → **Deploy to production** (নীল-সবুজ বা ক্যানারি কৌশলে) → **Verify** (স্বাস্থ্য পরীক্ষা, মনিটরিং)। Azure Pipelines-এ এই পর্যায়গুলো **Stage**, **Job** ও **Step**-এ সাজানো হয় — YAML ফাইলে (`azure-pipelines.yml`)।

১৯.৫ Azure Pipelines: YAML পাইপলাইন

```
# azure-pipelines.yml — TechNova API: Build + Deploy to staging
trigger:
  - main
pool:
  vmImage: ubuntu-latest
stages:
  - stage: Build
    jobs:
      - job: BuildJob
        steps:
          - task: NodeTool@0
            inputs: { versionSpec: "20.x" }
          - script: npm ci && npm test
            displayName: "Install and test"
          - task: ArchiveFiles@2
            inputs:
              rootFolderOrFile: "$(System.DefaultWorkingDirectory)"
              includeRootFolder: false
              archiveFile: "$(Build.ArtifactStagingDirectory)/api.zip"
```

```
          - publish: $(Build.ArtifactStagingDirectory)/api.zip
            artifact: api
  - stage: DeployStaging
    dependsOn: Build
    jobs:
      - deployment: DeployWeb
        environment: staging
        strategy:
          runOnce:
            deploy:
              steps:
                - task: AzureWebApp@1
                  inputs:
                    azureSubscription: "technova-service-connection"
                    appName: "technova-api"
                    slotName: "staging"
                    package: "$(Pipeline.Workspace)/api/api.zip"
```

Environment (staging, production) হলো ডিপ্লয়মেন্টের লক্ষ্য — এতে **Approval** (অনুমোদনকারী) ও **Check** (যেমন ব্যবসায়িক সময়ে ডিপ্লয় নিষেধ) লাগানো যায়। **Variable Group** দিয়ে পরিবেশ ভেরিয়েবল পরিচালনা করুন — secret ভেরিয়েবল Key Vault-এর সাথে যুক্ত করুন, পাইপলাইনে সরাসরি লিখবেন না।

১৯.৬ Workload Identity Federation: secret ছাড়া CI/CD

GitHub Actions থেকে Azure-এ ডিপ্লয় করার পুরোনো পদ্ধতি ছিল Service Principal-এর client secret GitHub Secret-এ রাখা — secret ফাঁস হলে Azure আপস হয়। **Workload Identity Federation** এই সমস্যার সমাধান করে: GitHub Actions একটি স্বল্পমেয়াদি OIDC টোকেন দেয়, Azure তা যাচাই করে Entra ID টোকেন দেয় — কোনো দীর্ঘমেয়াদি secret সংরক্ষণ করতে হয় না। Azure-এ একটি **Federated Credential** তৈরি করুন (GitHub রিপোজিটরি, ব্রাঞ্চ ও পরিবেশ নির্দিষ্ট করে), তারপর GitHub Actions-এ `azure/login@v2` ব্যবহার করুন।

```
# .github/workflows/deploy.yml — GitHub Actions থেকে Azure-এ ডিপ্লয় (OIDC, secret ছাড়া)
name: Deploy API
on:
  push: { branches: [main] }
permissions:
  id-token: write
  contents: read
jobs:
  deploy:
    runs-on: ubuntu-latest
    environment: production
    steps:
      - uses: actions/checkout@v4
      - uses: azure/login@v2
        with:
          client-id: ${{ secrets.AZURE_CLIENT_ID }}
          tenant-id: ${{ secrets.AZURE_TENANT_ID }}
          subscription-id: ${{ secrets.AZURE_SUBSCRIPTION_ID }}
      - run: az webapp deploy --name technova-api --resource-group technova-prod-rg --
src-path ./api.zip --type zip
```

১৯.৭ রিলিজ কৌশল ও রোলব্যাক

কৌশল	কী	ঝুঁকি
Blue-Green	নতুন সংস্করণ আলাদা পরিবেশে, তারপর ট্র্যাফিক বদল	খরচ বেশি (দুটি পরিবেশ)
Canary	প্রথমে ৫–১০% ব্যবহারকারী, তারপর ধীরে ধীরে সবাই	জটিল মনিটরিং দরকার
Rolling	একটি করে সার্ভার বদলানো	মাঝখানে দুটি সংস্করণ চলে
Slot Swap	App Service slot বদলানো (অধ্যায় ১৬)	ডেটাবেস মাইগ্রেশনে সতর্কতা

ডেটাবেস মাইগ্রেশনের নিয়ম: **expand-migrate-contract** — প্রথমে নতুন কলাম যোগ করুন (পুরোনো কোড চলে), তারপর কোড ডিপ্লয় করুন, তারপর পুরোনো কলাম সরান। কখনোই কোড ও স্কিমা একসাথে ভাঙবেন না।

১৯.৮ সাধারণ ভুল ও ট্রাবলশুটিং

- **ভুল:** Secret পাইপলাইন YAML-এ লেখা — YAML Git-এ থাকে, secret ফাঁস হয়। Key Vault বা Variable Group ব্যবহার করুন।
- **ভুল:** Production-এ Approval ছাড়া স্বয়ংক্রিয় ডিপ্লয় — একটি ভুল কমিট সরাসরি ব্যবহারকারীর কাছে যায়।
- **ট্রাবলশুটিং — OIDC লগইন ব্যর্থ:** Federated Credential-এ রিপোজিটরি/ব্রাঞ্চ/পরিবেশের নাম ঠিক আছে কি না, এবং `permissions: id-token: write` আছে কি না দেখুন।

১৯.১২ সারসংক্ষেপ

CI/CD মানে Build → Deploy to staging → Approval → Deploy to production → Verify। Azure Pipelines-এ YAML-এ Stage/Job/Step সাজান, Environment-এ Approval লাগান। GitHub Actions থেকে Workload Identity Federation দিয়ে secret ছাড়া Azure-এ প্রমাণীকরণ করুন। রিলিজে Blue-Green, Canary বা Slot Swap ব্যবহার করুন, এবং ডেটাবেস মাইগ্রেশনে expand-migrate-contract নিয়ম মানুন।

১৯.১৩ পরিভাষা

Azure Pipelines — Azure DevOps-এর CI/CD সেবা — YAML পাইপলাইন।

Stage / Job / Step — পাইপলাইনের স্তর — পর্যায়, কাজ, ধাপ।

Environment — ডিপ্লয়মেন্টের লক্ষ্য পরিবেশ — Approval ও Check লাগানো যায়।

Variable Group — পরিবেশ ভেরিয়েবলের গোষ্ঠী — Key Vault-এর সাথে যুক্ত করা যায়।

Workload Identity Federation — OIDC টোকেন দিয়ে secret ছাড়া বাইরের প্ল্যাটফর্ম থেকে Azure-এ প্রমাণীকরণ।

Blue-Green — নতুন সংস্করণ আলাদা পরিবেশে তৈরি করে ট্র্যাফিক বদলানোর কৌশল।

Canary — প্রথমে অল্প ব্যবহারকারীকে নতুন সংস্করণ দেওয়ার কৌশল।

১৯.১৪ জ্ঞান যাচাই

1. Azure Pipelines-এর Stage, Job ও Step-এর মধ্যে সম্পর্ক কী?
2. Environment-এ Approval কেন দরকার? একটি উদাহরণ দিন।
3. Workload Identity Federation কীভাবে client secret-এর সমস্যার সমাধান করে?
4. Blue-Green ও Canary কৌশলের পার্থক্য কী? কখন কোনটি?
5. ডেটাবেস মাইগ্রেশনে expand-migrate-contract নিয়ম কী?

১৯.১৫ অনুশীলন

একটি YAML পাইপলাইন লিখুন: Build (npm test) → Deploy to staging → Approval → Deploy to production। GitHub Actions-এ একটি OIDC Federated Credential তৈরি করে একটি test ডিপ্লয় চালান।

অধ্যায় ২০ • PART IV

Chapter 20 — Infrastructure as Code on Azure

২০.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি পারবেন — (১) ARM টেমপ্লেট, Bicep ও Terraform-এর মধ্যে বাছাই করতে; (২) Bicep-এ মডিউল লিখতে; (৩) Terraform-এ Azure Provider ও State পরিচালনা করতে; (৪) Drift শনাক্ত করতে; এবং (৫) IaC-এর জন্য CI/CD সাজাতে।

২০.২ পূর্বশর্ত জ্ঞান

অধ্যায় ৫ (ARM) ও Volume 8 (Terraform)। JSON/YAML-এর জ্ঞান।

২০.৩ কেন এই বিষয়টি গুরুত্বপূর্ণ

হাতে ক্লিক করে তৈরি করা অবকাঠামো পুনরাবৃত্তি করা যায় না, পরীক্ষা করা যায় না, Git-এ রাখা যায় না। Infrastructure as Code (IaC) মানে অবকাঠামোকে কোড হিসেবে লেখা — পর্যালোচনা, সংস্করণ, স্বয়ংক্রিয় ডিপ্লয়। Azure-এ তিনটি প্রধান সরঞ্জাম: ARM টেমপ্লেট, Bicep ও Terraform।

২০.৪ প্রথম নীতি থেকে: তিনটি সরঞ্জামের তুলনা

বিষয়	ARM টেমপ্লেট	Bicep	Terraform
ভাষা	JSON	Bicep (ARM-এ কম্পাইল হয়)	HCL
প্রদানকারী	শুধু Azure	শুধু Azure	বহু ক্লাউড ও সার্ভিস
State	Azure-এ (deployment history)	Azure-এ	নিজের State ফাইল (Blob-এ)
শেখা	কঠিন (JSON বড় হয়)	সহজ	মাঝারি
কখন	Azure-নির্দিষ্ট, গভীর একীকরণ	Azure-এ ডিফল্ট পছন্দ	মাল্টি-ক্লাউড বা বিদ্যমান Terraform দক্ষতা

নিয়ম: শুধু Azure-এ কাজ করলে **Bicep** দিয়ে শুরু করুন — এটি Microsoft-এর সুপারিশকৃত ভাষা, ARM-এর সব সুবিধা পায়, কিন্তু JSON-এর চেয়ে অনেক ছোট ও পাঠযোগ্য। মাল্টি-ক্লাউড বা Terraform-দক্ষ দলে **Terraform** ব্যবহার করুন। ARM JSON সরাসরি লেখা এখন আর সুপারিশ করা হয় না — Bicep-এ লিখুন, প্রয়োজনে JSON দেখুন।

২০.৫ Bicep: মডিউল দিয়ে পুনঃব্যবহার

```
// main.bicep — TechNova VNet ও Storage, মডিউল দিয়ে
param location string = 'southeastasia'
param env string = 'prod'

module vnet './modules/vnet.bicep' = {
  name: 'vnetDeploy'
  params: {
```

```
    location: location
    addressPrefix: '10.0.0.0/16'
  }
}

resource storage 'Microsoft.Storage/storageAccounts@2023-01-01' = {
  name: 'technovastore001'
  location: location
  sku: { name: 'Standard_ZRS' }
  kind: 'StorageV2'
  properties: {
    supportsHttpsTrafficOnly: true
    minimumTlsVersion: 'TLS1_2'
    allowBlobPublicAccess: false
  }
  tags: { Env: env }
}

output storageId string = storage.id
```

Bicep-এ **মডিউল** হলো পুনঃব্যবহারযোগ্য ফাইল — VNet মডিউল একবার লিখে dev, staging, prod-এ ব্যবহার করুন। **প্যারামিটার** দিয়ে পরিবেশভেদে মান বদলান। ডিপ্লয় করার আগে `az deployment group what-if` দিয়ে দেখুন কী বদলাবে — এটি “plan”-এর Azure সংস্করণ।

২০.৬ Terraform on Azure: Provider, State ও Drift

Terraform-এ Azure Provider (`hashicorp/azurerm`) ব্যবহার করুন। **State** সংরক্ষণ করুন Azure Blob Storage-এ (azurerm backend) — State-এ সংবেদনশীল মান থাকতে পারে, তাই Blob-এ এনক্রিপশন ও ব্যক্তিগত অ্যাক্সেস চালু করুন, এবং **State Locking**-এর জন্য Blob lease ব্যবহার করুন। **Drift** মানে হাতে বদলানো রিসোর্স — `terraform plan` দিয়ে শনাক্ত করুন, এবং হাতে বদলানো বন্ধ করুন: সব পরিবর্তন কোড দিয়ে করুন।

```
# Terraform — Azure Provider ও Remote State (Blob)
terraform {
  required_providers {
    azurerm = {
      source  = "hashicorp/azurerm"
      version = "~> 4.0"
    }
  }
  backend "azurerm" {
    resource_group_name  = "technova-tfstate-rg"
    storage_account_name = "technovatfstate001"
    container_name       = "tfstate"
    key                  = "prod.terraform.tfstate"
  }
}

provider "azurerm" {
  features {}
}

resource "azurerm_resource_group" "prod" {
```

```
  name     = "technova-prod-rg"
  location = "southeastasia"
  tags     = { Env = "prod" }
}
```

২০.৭ সাধারণ ভুল ও ট্রাবলশুটিং

- **ভুল:** Terraform State Git-এ রাখা — State-এ secret থাকতে পারে, এবং একসাথে দুজন apply করলে State নষ্ট হয়। Blob backend + locking ব্যবহার করুন।
- **ভুল:** `what-if` / `plan` ছাড়া apply করা — কী বদলাবে না জেনে ডিপ্লয় করবেন না।
- **ট্রাবলশুটিং — Bicep ডিপ্লয় ব্যর্থ:** `az deployment group show`-এ ত্রুটি দেখুন; সাধারণ কারণ: নামের অনন্যতা (Storage Account নাম বিশ্বজুড়ে অনন্য হতে হয়), কোটা, বা নীতি (Policy) ব্লক।

২০.১২ সারসংক্ষেপ

শুধু Azure-এ কাজ করলে Bicep ডিফল্ট পছন্দ — ছোট, পাঠযোগ্য, ARM-এর সব সুবিধাসহ। মাল্টি-ক্লাউডে Terraform। মডিউল দিয়ে পুনঃব্যবহার করুন, প্যারামিটার দিয়ে পরিবেশ ভেদ করুন, what-if/plan দিয়ে আগে দেখুন কী বদলাবে। Terraform State Blob-এ রাখুন, locking চালু করুন, এবং হাতে বদলানো বন্ধ করে Drift প্রতিরোধ করুন।

২০.১৩ পরিভাষা

Bicep — ARM টেমপ্লেট লেখার সহজ ভাষা — কম্পাইল হয়ে ARM JSON হয়।

Module (Bicep) — পুনঃব্যবহারযোগ্য Bicep ফাইল — অন্য ফাইল থেকে ব্যবহার করা যায়।

what-if — ডিপ্লয়ের আগে কী বদলাবে তা দেখার Azure কমান্ড।

Terraform State — Terraform-এর পরিচালিত রিসোর্সের অবস্থার ফাইল।

Drift — কোডের বাইরে হাতে বদলানো অবকাঠামো — plan দিয়ে শনাক্ত করা যায়।

State Locking — একসাথে দুজন apply করা থেকে State রক্ষার ব্যবস্থা।

২০.১৪ জ্ঞান যাচাই

1. Bicep ও ARM টেমপ্লেটের সম্পর্ক কী? Bicep কেন সুপারিশ করা হয়?
2. Terraform ও Bicep-এর মধ্যে কখন কোনটি বাছাই করবেন?
3. what-if/plan ছাড়া apply করা কেন বিপজ্জনক?
4. Terraform State Blob-এ রাখা কেন জরুরি? State Locking কী?
5. Drift কী এবং কীভাবে প্রতিরোধ করবেন?

২০.১৫ অনুশীলন

একটি Bicep মডিউল লিখুন যা একটি VNet ও তিনটি সাবনেট তৈরি করে। `az deployment group what-if` দিয়ে দেখুন কী তৈরি হবে, তারপর ডিপ্লয় করুন। Terraform দিয়ে একই Resource Group তৈরি করে State Blob-এ সংরক্ষণ করুন।

PART V • DEVOPS ENGINEERING MASTERY

Production Azure Engineering

Observability ও Monitoring, Azure সিকিউরিটি ইঞ্জিনিয়ারিং, নির্ভরযোগ্যতা ও Disaster Recovery, প্রোডাকশন আর্কিটেকচার ও FinOps।

অধ্যায় ২১ • PART V

Chapter 21 — Observability and Monitoring

২১.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি পারবেন — (১) Azure Monitor, Log Analytics ও Application Insights-এর ভূমিকা ব্যাখ্যা করতে; (২) KQL দিয়ে লগ অনুসন্ধান করতে; (৩) মেট্রিক অ্যালার্ট ও অ্যাকশন গ্রুপ সাজাতে; (৪) Distributed Tracing দিয়ে অনুরোধের পথ দেখতে; এবং (৫) ঘটনা তদন্তের ধাপ অনুসরণ করতে।

২১.২ পূর্বশর্ত জ্ঞান

Volume 10 (Observability) — মেট্রিক, লগ, ট্রেস, SLI/SLO ধারণা।

২১.৩ কেন এই বিষয়টি গুরুত্বপূর্ণ

সিস্টেম বন্ধ হলে প্রথম প্রশ্ন: “কী হয়েছে?” — উত্তর দিতে হলে আগে থেকে ডেটা সংগ্রহ করতে হয়। Azure Monitor সেই ডেটার কেন্দ্র: মেট্রিক, লগ, ট্রেস — সব এক জায়গায়। এই অধ্যায়ে আপনি Azure-এর পর্যবেক্ষণ ব্যবস্থা শিখবেন।

২১.৪ প্রথম নীতি থেকে: Azure Monitor-এর তিনটি স্তর

স্তর	কী	উদাহরণ
Metrics	সংখ্যাগত সময়-সিরিজ ডেটা	CPU ৭৫%, অনুরোধ/সেকেন্ড ১২০
Logs	বিস্তারিত ঘটনার রেকর্ড	অ্যাপ লগ, NSG Flow Log, Audit Log
Traces	একটি অনুরোধের বিভিন্ন সার্ভিসে যাত্রা	Frontend → API → ডেটাবেস

Log Analytics Workspace হলো লগের কেন্দ্রীয় সংগ্রহস্থল — সব সার্ভিসের লগ এখানে আসে, এবং **KQL (Kusto Query Language)** দিয়ে অনুসন্ধান করা যায়। **Application Insights** হলো অ্যাপ্লিকেশনের জন্য APM — অনুরোধ, নির্ভরতা, ব্যতিক্রম (exception) ও পারফরম্যান্স ট্র্যাক করে। **Diagnostic Settings** দিয়ে প্রতিটি সার্ভিসের লগ ও মেট্রিক Log Analytics-এ পাঠান — ডিফল্টভাবে অনেক সার্ভিসের লগ কোথাও যায় না!

২১.৫ KQL: লগ অনুসন্ধানের ভাষা

```
// গত 1 ঘণ্টায় 500 ত্রুটি দেওয়া অনুরোধ
requests
| where timestamp > ago(1h)
| where resultCode startswith "5"
| summarize count() by operation_Name, resultCode
| order by count_ desc

// ধীরতম 10টি নির্ভরতা (ডেটাবেস কল)
dependencies
| where timestamp > ago(1h)
| summarize avg(duration), count() by target
```

```
| order by avg_duration desc
| take 10
```

KQL-এর মূল ধারণা: টেবিল থেকে শুরু করুন (`requests`, `dependencies`, `traces`), `| where` দিয়ে ফিল্টার করুন, `| summarize` দিয়ে গোষ্ঠীবদ্ধ করুন, `| order by` দিয়ে সাজান। প্রতিদিনের তদন্তে এই তিনটি অপারেটরই ৮০% কাজ করে।

২১.৬ অ্যালার্ট: কখন, কীভাবে, কাকে

অ্যালার্টের নিয়ম: **কার্যকর (actionable)** হতে হবে — অ্যালার্ট পেলে ইঞ্জিনিয়ার জানবে কী করতে হবে। "CPU ৮০%" কার্যকর নয়; "API-এর ৫xx ত্রুটি ৫ মিনিটে ৫%-এর বেশি" কার্যকর। **অ্যাকশন গ্রুপ** নির্ধারণ করে কে বিজ্ঞপ্তি পাবে (ইমেইল, SMS, Teams, Webhook, Logic App)। **অ্যালার্ট নিয়ম** তিন ধরনের: মেট্রিক অ্যালার্ট (থ্রেশহোল্ড), লগ অ্যালার্ট (KQL কোয়েরি), Activity Log অ্যালার্ট (যেমন "কেউ VM মুছেছে")।

```
# একটি মেট্রিক অ্যালার্ট তৈরি করুন: 5xx ত্রুটি 5%-এর বেশি হলে
az monitor metrics alert create --name high-5xx-errors --resource-group technova-prod-rg
\
  --scopes "/subscriptions/00000000-0000-0000-0000-000000000000/resourceGroups/technova-
prod-rg/providers/Microsoft.Web/sites/technova-api" \
  --condition "avg Http5xx > 5" --window-size 5m --evaluation-frequency 1m --severity 2 \
  --action "/subscriptions/00000000-0000-0000-0000-000000000000/resourceGroups/technova-
prod-rg/providers/microsoft.insights/actionGroups/technova-oncall"
```

WARNING: Log Analytics-এ লগ ইনজেশনের খরচ ডেটার পরিমাণের ওপর নির্ভর করে — সব লগ সবসময় পাঠাবেন না। **Basic Logs** পরিকল্পনা (সস্তা, কম অনুসন্ধান সুবিধা) ও **ডেটা সংগ্রহ নিয়ম (DCR)** দিয়ে শুধু প্রয়োজনীয় লগ পাঠান। খরচের অ্যালার্ট লাগান — একটি ভুল কনফিগারেশন একদিনে হাজার ডলারের লগ তৈরি করতে পারে।

২১.৭ সাধারণ ভুল ও ট্রাবলশুটিং

- **ভুল:** Diagnostic Settings চালু না করা — ঘটনার পর দেখা যায় লগই নেই। প্রতিটি প্রোডাকশন সার্ভিসে Diagnostic Settings বাধ্যতামূলক করুন।
- **ভুল:** অ্যালার্টে শুধু থ্রেশহোল্ড দেওয়া, অ্যাকশন গ্রুপ না দেওয়া — অ্যালার্ট আসে কিন্তু কেউ জানে না।
- **ট্রাবলশুটিং — লগ দেখা যাচ্ছে না:** (১) Diagnostic Settings-এ গন্তব্য (Log Analytics Workspace) সঠিক কি না, (২) লগ আসতে কয়েক মিনিট লাগে — অপেক্ষা করুন, (৩) Workspace-এর Region ও অনুমতি যাচাই করুন।

২১.১২ সারসংক্ষেপ

Azure Monitor মেট্রিক, লগ ও ট্রেসের কেন্দ্র। Log Analytics Workspace-এ লগ জমা হয়, KQL দিয়ে অনুসন্ধান করা যায়। Application Insights অ্যাপের APM দেয়। Diagnostic Settings প্রতিটি সার্ভিসে চালু করুন — নইলে ঘটনার সময় লগ থাকবে না। অ্যালার্ট কার্যকর হতে হবে, এবং অ্যাকশন গ্রুপ ছাড়া অ্যালার্ট অর্থহীন। আর লগের খরচ নিয়ন্ত্রণ করুন — DCR ও Basic Logs ব্যবহার করুন।

২১.১৩ পরিভাষা

Azure Monitor — Azure-এর পর্যবেক্ষণ প্ল্যাটফর্ম — মেট্রিক, লগ, অ্যালার্ট।

Log Analytics Workspace — লগের কেন্দ্রীয় সংগ্রহস্থল — KQL দিয়ে অনুসন্ধান করা যায়।

KQL — Kusto Query Language — লগ অনুসন্ধানের ভাষা।

Application Insights — অ্যাপ্লিকেশনের APM — অনুরোধ, নির্ভরতা, ব্যতিক্রম ট্র্যাক করে।

Diagnostic Settings — সার্ভিসের লগ ও মেট্রিক Log Analytics-এ পাঠানোর সেটিং।

Action Group — অ্যালার্ট পেলে কে কীভাবে বিজ্ঞপ্তি পাবে — তা নির্ধারণ করে।

DCR — Data Collection Rule — কোন লগ সংগ্রহ করা হবে তা নিয়ন্ত্রণ করে।

২১.১৪ জ্ঞান যাচাই

1. Metrics, Logs ও Traces-এর পার্থক্য কী? প্রতিটির একটি উদাহরণ দিন।
2. Diagnostic Settings কেন প্রতিটি প্রোডাকশন সার্ভিসে বাধ্যতামূলক?
3. KQL-এ where, summarize ও order by-এর কাজ কী?
4. কার্যকর অ্যালার্ট কাকে বলে? “CPU ৮০%” কেন কার্যকর নয়?
5. Log Analytics-এর খরচ কীভাবে নিয়ন্ত্রণ করবেন?

২১.১৫ অনুশীলন

একটি Log Analytics Workspace তৈরি করুন। একটি App Service-এর Diagnostic Settings চালু করে লগ পাঠান। KQL দিয়ে গত ১ ঘণ্টার অনুরোধ দেখুন এবং একটি 5xx অ্যালার্ট নিয়ম তৈরি করুন।

অধ্যায় ২২ • PART V

Chapter 22 — Azure Security Engineering

২২.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি পারবেন — (১) Key Vault দিয়ে secret, কী ও সার্টিফিকেট পরিচালনা করতে; (২) Defender for Cloud ও Microsoft Sentinel-এর ভূমিকা ব্যাখ্যা করতে; (৩) নেটওয়ার্ক, ডেটা ও পরিচয় — তিন স্তরে নিরাপত্তা সাজাতে; (৪) নিরাপত্তা অনুসন্ধানের (finding) জবাব দিতে; এবং (৫) CI/CD ও সাপ্লাই চেইন সুরক্ষিত করতে।

২২.২ পূর্বশর্ত জ্ঞান

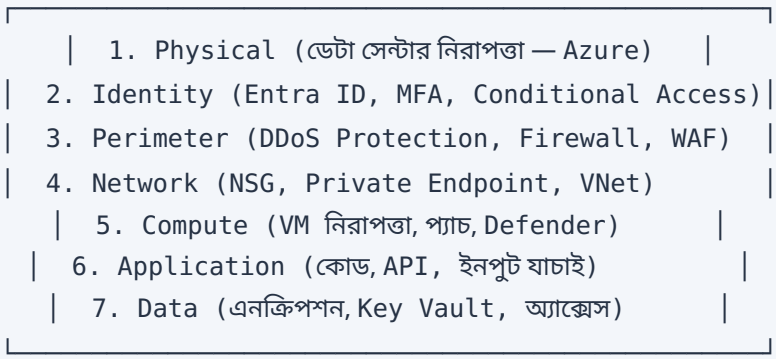
অধ্যায় ৩, ৭, ২১: Entra ID, NSG, Azure Monitor। এনক্রিপশন ও secret ব্যবস্থাপনার সাধারণ ধারণা।

২২.৩ কেন এই বিষয়টি গুরুত্বপূর্ণ

নিরাপত্তা কোনো একক পণ্য নয় — এটি স্তরে স্তরে সাজানো একটি ব্যবস্থা: পরিচয়, নেটওয়ার্ক, ডেটা, অ্যাপ, পর্যবেক্ষণ। একটি স্তর ভেঙে গেলেও অন্য স্তর রক্ষা করে — একে বলে **defense in depth**। এই অধ্যায়ে আপনি Azure-এর নিরাপত্তা ব্যবস্থা একত্রে দেখবেন।

২২.৪ প্রথম নীতি থেকে: Defense in Depth

Defense in Depth — বাইরে থেকে ভেতরে:



প্রতিটি স্তর ভাঙলেও পরের স্তর রক্ষা করে — একটি স্তরের ওপর নির্ভর করা যাবে না।

২২.৫ Key Vault: গোপন তথ্যের নিরাপদ ঘর

Azure Key Vault হলো secret, কী ও সার্টিফিকেট সংরক্ষণের সেবা — পাসওয়ার্ড, সংযোগ স্ট্রিং, API কী, TLS সার্টিফিকেট। Key Vault-এর তিনটি বস্তু: **Secret** (যেকোনো গোপন মান), **Key** (এনক্রিপশন কী), **Certificate** (TLS সার্টিফিকেট — স্বয়ংক্রিয় নবায়নসহ)। অ্যাক্সেস দিন Managed Identity দিয়ে — অ্যাপ Key Vault থেকে secret পড়ে, কোডে secret থাকে না। **Soft Delete** ও **Purge Protection** চালু করুন — ভুলে মুছে ফেলা secret ফিরিয়ে আনা যায়, আক্রমণকারী স্থায়ীভাবে মুছতে পারে না।

২২.৬ Defender for Cloud ও Microsoft Sentinel

Microsoft Defender for Cloud হলো Azure-এর নিরাপত্তা ব্যবস্থাপনা — এটি আপনার পরিবেশ স্ক্যান করে সুপারিশ দেয় (যেমন "এই Storage Account-এ পাবলিক অ্যাক্সেস খোলা"), **Secure Score** দেখায়, এবং সার্ভার/কনটেইনার/ডেটাবেসের জন্য হুমকি সুরক্ষা (Defender প্ল্যান) দেয়। **Microsoft Sentinel** হলো SIEM — সব লগ এক জায়গায় এনে আক্রমণের প্যাটার্ন শনাক্ত করে, স্বয়ংক্রিয় প্রতিক্রিয়া (playbook) চালায়। Defender "কী ভুল আছে" বলে; Sentinel "কী আক্রমণ হচ্ছে" বলে।

২২.৭ এনক্রিপশন ও ডেটা সুরক্ষা

Azure-এ ডেটা দুই অবস্থায় এনক্রিপ্ট করা হয়: **at rest** (ডিস্কে — Storage, Disk, SQL-এ TDE ডিফল্টভাবে চালু) ও **in transit** (নেটওয়ার্কে — TLS)। কী পরিচালনার দুটি মডেল: **Microsoft-managed keys** (সহজ, ডিফল্ট) ও **Customer-managed keys (CMK)** (Key Vault-এ আপনার কী — বেশি নিয়ন্ত্রণ, বেশি দায়িত্ব)। সংবেদনশীল ডেটার জন্য CMK বিবেচনা করুন। **Microsoft Purview** দিয়ে সংবেদনশীল ডেটা (যেমন জাতীয় পরিচয় নম্বর) শনাক্ত ও শ্রেণিবদ্ধ করা যায়।

২২.৮ CI/CD ও সাপ্লাই চেইন নিরাপত্তা

সাপ্লাই চেইন আক্রমণ মানে আপনার কোডে নয়, আপনার নির্ভরতায় (dependency) আক্রমণ — একটি আপস হওয়া npm প্যাকেজ আপনার অ্যাপে ঢুকে যায়। প্রতিরোধ: (১) নির্ভরতা স্ক্যান করুন (Dependabot, Defender for DevOps); (২) ইমেজ স্ক্যান করুন (ACR, Defender for Containers); (৩) পাইপলাইনে শুধু বিশ্বস্ত টাস্ক ব্যবহার করুন; (৪) আর্টিফ্যাক্টে স্বাক্ষর করুন; (৫) প্রোডাকশন ডিপ্লয়ে Approval বাধ্যতামূলক করুন (অধ্যায় ১৯)।

```
# Key Vault তৈরি (Soft Delete ও Purge Protection-সহ)
az keyvault create --name technova-kv --resource-group technova-prod-rg --location
southeastasia \
  --enable-soft-delete true --enable-purge-protection true --retention-days 90
# একটি Secret রাখুন ও Managed Identity-কে পড়ার অনুমতি দিন
az keyvault secret set --vault-name technova-kv --name db-password --value "CHANGE-ME-
STRONG-PASSWORD"
az role assignment create --assignee-object-id <APP_MANAGED_IDENTITY_OBJECT_ID> --
assignee-principal-type ServicePrincipal \
  --role "Key Vault Secrets User" --scope "/subscriptions/
00000000-0000-0000-0000-000000000000/resourceGroups/technova-prod-rg/providers/
Microsoft.KeyVault/vaults/technova-kv"
```

২২.৯ সাধারণ ভুল ও ট্রাবলশুটিং

- **ভুল:** Key Vault-এ Soft Delete বন্ধ রাখা — একটি ভুল কমান্ডে secret চিরতরে হারিয়ে যায়।
- **ভুল:** Defender-এর সুপারিশ উপেক্ষা করা — Secure Score কম থাকলে নিয়মিত পর্যালোচনা করুন।
- **ট্রাবলশুটিং — অ্যাপ Key Vault থেকে secret পড়তে পারছে না:** (১) Managed Identity-র ভূমিকা (Key Vault Secrets User) আছে কি না, (২) Key Vault-এর নেটওয়ার্ক নিয়ম (ফায়ারওয়াল/Private Endpoint) অ্যাপের VNet অনুমোদন করে কি না, (৩) secret-এর নাম ও সংস্করণ সঠিক কি না।

২২.১২ সারসংক্ষেপ

নিরাপত্তা মানে স্তরে স্তরে সুরক্ষা — পরিচয়, পেরিমিটার, নেটওয়ার্ক, কম্পিউট, অ্যাপ, ডেটা। Key Vault-এ secret রাখুন, Soft Delete ও Purge Protection চালু করুন, Managed Identity দিয়ে অ্যাক্সেস দিন। Defender for Cloud "কী ভুল আছে" বলে, Sentinel "কী আক্রমণ হচ্ছে" বলে — দুটোই ব্যবহার করুন। আর সাপ্লাই চেইন সুরক্ষিত করুন — নির্ভরতা ও ইমেজ স্ক্যান, স্বাক্ষর, Approval।

২২.১৩ পরিভাষা

Key Vault — Secret, কী ও সার্টিফিকেট সংরক্ষণের সেবা।

Defender for Cloud — Azure-এর নিরাপত্তা ব্যবস্থাপনা — সুপারিশ, Secure Score, হুমকি সুরক্ষা।

Microsoft Sentinel — SIEM — সব লগ এক জায়গায় এনে আক্রমণ শনাক্ত ও প্রতিক্রিয়া।

Secure Score — পরিবেশের নিরাপত্তা অবস্থার সংখ্যাগত পরিমাপ।

CMK — Customer-Managed Key — Key Vault-এ আপনার নিজের এনক্রিপশন কী।

Defense in Depth — স্তরে স্তরে নিরাপত্তা — একটি স্তর ভাঙলেও পরেরটি রক্ষা করে।

SIEM — Security Information and Event Management — নিরাপত্তা লগের কেন্দ্রীয় বিশ্লেষণ।

২২.১৪ জ্ঞান যাচাই

1. Defense in Depth কী? তিনটি স্তরের উদাহরণ দিন।
2. Key Vault-এ Secret, Key ও Certificate-এর পার্থক্য কী?
3. Defender for Cloud ও Microsoft Sentinel-এর মধ্যে পার্থক্য কী?
4. Soft Delete ও Purge Protection কেন গুরুত্বপূর্ণ?
5. সাপ্লাই চেইন আক্রমণ কী? তিনটি প্রতিরোধের উপায় লিখুন।

২২.১৫ অনুশীলন

একটি Key Vault তৈরি করুন (Soft Delete ও Purge Protection-সহ)। একটি secret রাখুন, একটি Managed Identity-কে পড়ার ভূমিকা দিন, এবং একটি test অ্যাপ থেকে secret পড়ুন। Defender for Cloud খুলে Secure Score দেখুন এবং একটি সুপারিশ বাস্তবায়ন করুন।

অধ্যায় ২৩ • PART V

Chapter 23 — Reliability and Disaster Recovery

২৩.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি পারবেন — (১) RPO ও RTO দিয়ে পুনরুদ্ধার লক্ষ্য নির্ধারণ করতে; (২) Azure Backup ও Site Recovery দিয়ে DR পরিকল্পনা করতে; (৩) Cross-Region পুনরুদ্ধারের নকশা করতে; (৪) ঘটনা প্রতিক্রিয়া (incident response) পরিচালনা করতে; এবং (৫) রানবুক ও পোস্টমর্টেম লিখতে।

২৩.২ পূর্বশর্ত জ্ঞান

অধ্যায় ১১, ১৩–১৪: উচ্চ-উপলভ্যতা, ব্যাকআপ, ডেটাবেস। RPO/RTO ধারণা।

২৩.৩ কেন এই বিষয়টি গুরুত্বপূর্ণ

উচ্চ-উপলভ্যতা মানে একটি Zone নষ্ট হলেও সেবা চলবে। Disaster Recovery মানে পুরো Region নষ্ট হলেও সেবা ফিরিয়ে আনা যাবে। দুটো আলাদা জিনিস — এবং দুটোরই পরিকল্পনা আগেই করতে হয়, বিপর্যয়ের সময় নয়।

২৩.৪ প্রথম নীতি থেকে: পুনরুদ্ধারের চারটি কৌশল

কৌশল	RTO	RPO	খরচ	কখন
Backup and Restore	ঘণ্টা–দিন	ঘণ্টা	সবচেয়ে কম	কম গুরুত্বপূর্ণ সিস্টেম
Pilot Light	ঘণ্টা	মিনিট	মাঝারি	গুরুত্বপূর্ণ, কিন্তু বাজেট সীমিত
Warm Standby	মিনিট–ঘণ্টা	সেকেন্ড–মিনিট	বেশি	গুরুত্বপূর্ণ সিস্টেম
Multi-Site Active-Active	সেকেন্ড	প্রায় শূন্য	সবচেয়ে বেশি	অতি-গুরুত্বপূর্ণ, বিশ্বজুড়ে

কৌশল বাছাইয়ের আগে ব্যবসার সাথে RPO/RTO ঠিক করুন — “যত দ্রুত সম্ভব” কোনো লক্ষ্য নয়। একটি ই-কমার্স সাইটের RTO ১ ঘণ্টা হলে Warm Standby দরকার; একটি অভ্যন্তরীণ রিপোর্টিং টুলের RTO ১ দিন হলে Backup and Restore যথেষ্ট। খরচ ও লক্ষ্যের ভারসাম্য এখানেই।

২৩.৫ Azure Site Recovery (ASR)

Azure Site Recovery হলো VM-এর প্রতিলিপি (replication) সেবা — VM-এর ডিস্ক ক্রমাগত অন্য Region-এ প্রতিলিপি হয়। বিপর্যয় হলে **failover** চালিয়ে অন্য Region-এ VM চালু করা যায়। **Recovery Plan** দিয়ে ফেইলওভারের ক্রম নির্ধারণ করুন — প্রথমে ডেটাবেস, তারপর API, তারপর ফ্রন্টএন্ড। **Test Failover** দিয়ে উৎপাদন ব্যাহত না করে DR পরীক্ষা করুন — বছরে অন্তত দুবার।

২৩.৬ ঘটনা প্রতিক্রিয়া (Incident Response)

ঘটনার জীবনচক্র: **শনাক্তকরণ** (অ্যালার্ট, ব্যবহারকারীর অভিযোগ) → **ঘোষণা** (ঘটনা ঘোষণা, দায়িত্বপ্রাপ্ত নির্ধারণ) → **নিয়ন্ত্রণ** (সেবা ফিরিয়ে আনা — প্রায়ই অস্থায়ী সমাধান) → **মূল কারণ নির্ণয়** (Root Cause Analysis) → **পোস্টমর্টেম**

(দোষারোপ নয়, শিক্ষা — কী হয়েছিল, কেন হয়েছিল, কী বদলাবে)। পোস্টমর্টেমে "কে ভুল করেছে" নয়, "কোন ব্যবস্থা ভুল হতে দিয়েছে" — এই প্রশ্ন করুন।

২৩.৭ নির্ভরযোগ্যতা পরীক্ষা

Chaos Engineering মানে ইচ্ছাকৃতভাবে ব্যর্থতা তৈরি করে দেখা সিস্টেম কীভাবে সাড়া দেয় — যেমন একটি Zone-এর VM বন্ধ করে দেখা ট্র্যাফিক অন্য Zone-এ যায় কি না। **Azure Chaos Studio** দিয়ে নিয়ন্ত্রিতভাবে পরীক্ষা চালানো যায়। নিয়ম: প্রথমে test পরিবেশে, তারপর production-এ কম ঝুঁকির পরীক্ষা, সবসময় rollback পরিকল্পনাসহ।

২৩.৮ সাধারণ ভুল ও ট্রাবলশুটিং

- **ভুল:** DR পরিকল্পনা লিখে কখনো পরীক্ষা না করা — পরীক্ষা না করা পরিকল্পনা পরিকল্পনা নয়, আশা মাত্র।
- **ভুল:** ব্যাকআপ ও DR একই Region-এ রাখা — Region-স্তরের বিপর্যয়ে দুটোই হারায়।
- **ট্রাবলশুটিং — ফেইলওভার ব্যর্থ:** Recovery Plan-এর ক্রম যাচাই করুন, লক্ষ্য Region-এ কোটা আছে কি না দেখুন, এবং নেটওয়ার্ক (DNS, Private Endpoint) লক্ষ্য Region-এ কাজ করে কি না পরীক্ষা করুন।

২৩.১২ সারসংক্ষেপ

Disaster Recovery মানে Region-স্তরের বিপর্যয় থেকে ফিরে আসা — RPO/RTO ঠিক করে কৌশল বাছাই করুন (Backup and Restore থেকে Active-Active)। Azure Site Recovery দিয়ে VM প্রতিলিপি করুন, Recovery Plan দিয়ে ক্রম নির্ধারণ করুন, Test Failover দিয়ে নিয়মিত পরীক্ষা করুন। ঘটনা ঘটলে শনাক্ত → ঘোষণা → নিয়ন্ত্রণ → মূল কারণ → পোস্টমর্টেম — এবং পোস্টমর্টেমে ব্যবস্থাকে দোষ দিন, মানুষকে নয়।

২৩.১৩ পরিভাষা

RPO — Recovery Point Objective — সর্বোচ্চ কতটুকু ডেটা হারানো যাবে।

RTO — Recovery Time Objective — কত সময়ের মধ্যে সেবা ফিরবে।

Azure Site Recovery — VM প্রতিলিপি ও ফেইলওভারের সেবা — অন্য Region-এ পুনরুদ্ধার।

Recovery Plan — ফেইলওভারের ক্রম ও ধাপের পরিকল্পনা।

Failover — প্রাথমিক স্থান থেকে দ্বিতীয় স্থানে সেবা সরানো।

Test Failover — উৎপাদন ব্যাহত না করে DR পরীক্ষা।

Postmortem — ঘটনার পরের বিশ্লেষণ — কী হয়েছিল, কেন, কী বদলাবে।

Chaos Engineering — ইচ্ছাকৃত ব্যর্থতা দিয়ে সিস্টেমের সহনশীলতা পরীক্ষা।

২৩.১৪ জ্ঞান যাচাই

1. RPO ও RTO-এর পার্থক্য কী? একটি উদাহরণ দিয়ে বোঝান।
2. Backup and Restore ও Warm Standby কৌশলের মধ্যে পার্থক্য কী?
3. Recovery Plan কেন দরকার? একটি উদাহরণ দিন।
4. Test Failover কেন গুরুত্বপূর্ণ?
5. পোস্টমর্টেমে “কে ভুল করেছে” জিজ্ঞাসা করা কেন ভুল?

২৩.১৫ অনুশীলন

TechNova-র জন্য একটি DR পরিকল্পনা লিখুন: RPO ১৫ মিনিট, RTO ২ ঘণ্টা — কোন কৌশল, কোন সার্ভিস, ফেইলওভারের ক্রম, এবং বছরে কতবার পরীক্ষা। একটি test VM-এর জন্য Azure Site Recovery চালু করে Test Failover চালান।

অধ্যায় ২৪ • PART V

Chapter 24 — Production Architecture and FinOps

২৪.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি পারবেন — (১) TechNova-র সম্পূর্ণ প্রোডাকশন আর্কিটেকচার নকশা করতে; (২) এন্টারপ্রাইজ ল্যান্ডিং জোনের উপাদান ব্যাখ্যা করতে; (৩) মাল্টি-Region নকশার trade-off বিশ্লেষণ করতে; (৪) FinOps দিয়ে খরচ অপ্টিমাইজ করতে; (৫) Architecture Decision Record (ADR) লিখতে; এবং (৬) ক্যাপস্টোনের জন্য সম্পূর্ণ পরিকল্পনা তৈরি করতে।

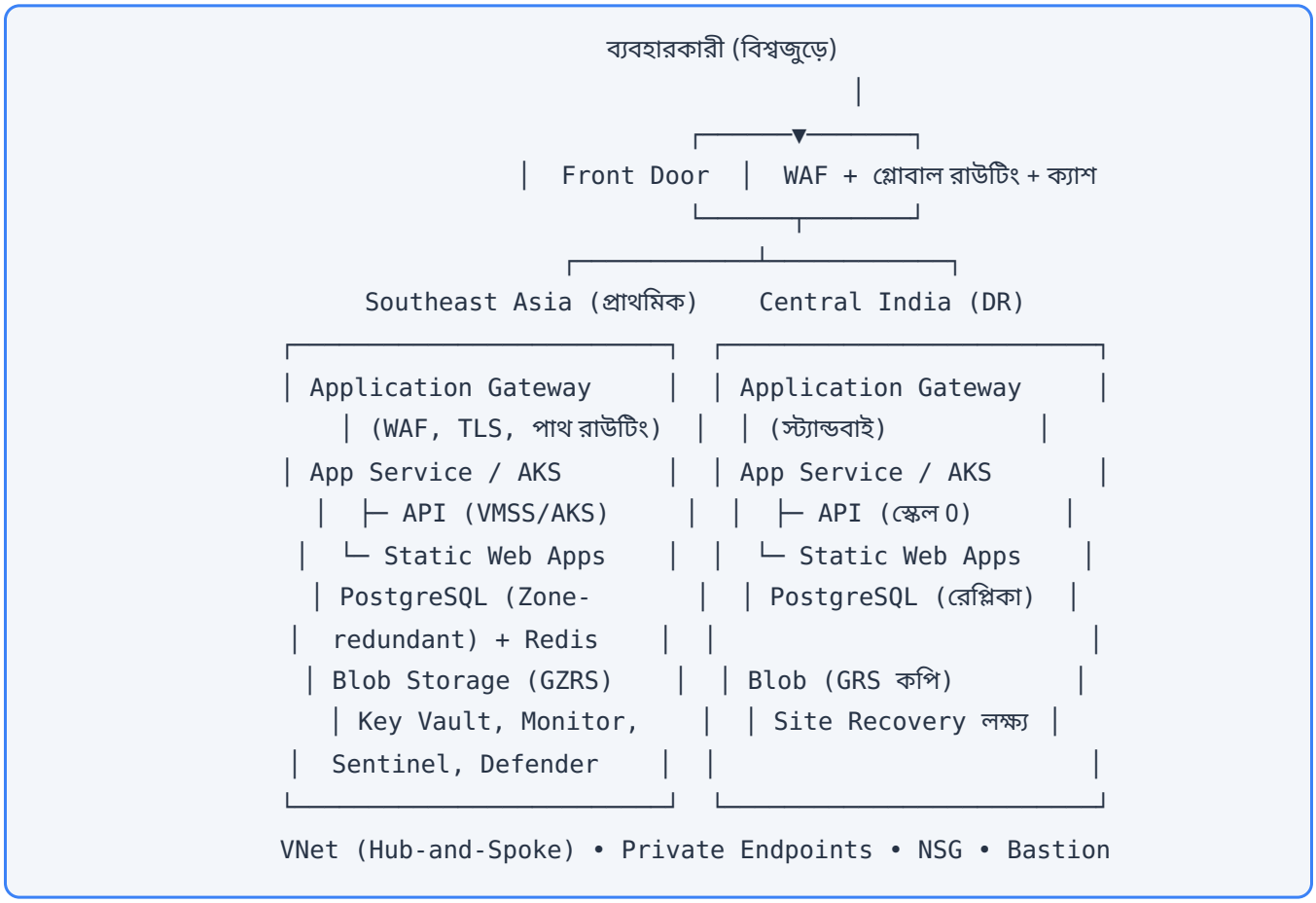
২৪.২ পূর্বশর্ত জ্ঞান

অধ্যায় ১–২৩-এর সব ধারণা। এই অধ্যায়টি পুরো বইয়ের সমন্বয় — আগের অধ্যায়গুলো না পড়ে এটি পড়বেন না।

২৪.৩ কেন এই বিষয়টি গুরুত্বপূর্ণ

এই বইয়ের ২৩ অধ্যায়ে আপনি টুকরো টুকরো শিখেছেন — এখন সেগুলো একত্র করে একটি সম্পূর্ণ প্রোডাকশন প্ল্যাটফর্ম বানানোর সময়। এই অধ্যায়ে TechNova-র সম্পূর্ণ আর্কিটেকচার, খরচ ব্যবস্থাপনা (FinOps) ও পরিচালন প্রস্তুতি একসাথে দেখা হবে — এবং ক্যাপস্টোনের পথ দেখানো হবে।

২৪.৪ TechNova প্রোডাকশন আর্কিটেকচার: সম্পূর্ণ ছবি



এই আর্কিটেকচারের মূল সিদ্ধান্তগুলো: (১) **Front Door** গ্লোবাল এন্ট্রি — একটি Region বন্ধ হলে ট্র্যাফিক অন্য Region-এ যায়। (২) **প্রতিটি স্তর প্রাইভেট** — ডেটাবেস ও Storage শুধু Private Endpoint দিয়ে অ্যাক্সেসযোগ্য। (৩) **পরিচয়-ভিত্তিক অ্যাক্সেস** — Managed Identity সর্বত্র, কোনো secret কোডে নেই। (৪) **পর্যবেক্ষণ কেন্দ্রীয়** — সব লগ Log Analytics-এ, অ্যালার্ট অ্যাকশন গ্রুপে। (৫) **DR পরিকল্পিত** — Site Recovery + GRS ব্যাকআপ, বছরে দুবার পরীক্ষা।

২৪.৫ এন্টারপ্রাইজ ল্যান্ডিং জোন: বড় সংস্থার ভিত্তি

বড় সংস্থায় ল্যান্ডিং জোনের উপাদান: **Management Group কাঠামো** (platform, landing zones, sandbox, decommissioned), **কেন্দ্রীয় নেটওয়ার্কিং** (hub VNet — Firewall, VPN/ExpressRoute Gateway, কেন্দ্রীয় DNS), **কেন্দ্রীয় পরিচয়** (Entra ID, PIM, Conditional Access), **কেন্দ্রীয় লগিং** (Log Analytics, Sentinel), **নীতি** (Azure Policy Initiative), **খরচ ব্যবস্থাপনা** (Management Group-স্তরে বাজেট)। Microsoft-এর Cloud Adoption Framework (CAF) এই নকশার নির্দেশিকা দেয় — ছোট দল এর সরলীকৃত সংস্করণ দিয়ে শুরু করতে পারে।

২৪.৬ FinOps: খরচকে প্রকৌশল হিসেবে দেখা

FinOps মানে খরচ ব্যবস্থাপনাকে অর্থ বিভাগের কাজ নয়, প্রকৌশলের অংশ হিসেবে দেখা। তিনটি পর্যায়: **Inform** (খরচ দেখা — ট্যাগ, Cost Analysis, বাজেট), **Optimize** (খরচ কমানো — সঠিক সাইজ, অপ্রয়োজনীয় রিসোর্স বন্ধ, Reserved Instance/Savings Plan, Spot VM), **Operate** (নিয়মিত পর্যালোচনা — সাপ্তাহিক খরচ পর্যালোচনা, অস্বাভাবিকতা অ্যালার্ট)। ব্যবহারিক উপায়: (১) প্রতিটি রিসোর্সে ট্যাগ (Env, CostCenter, Owner) বাধ্যতামূলক করুন; (২) dev/test পরিবেশ রাতে ও সপ্তাহান্তে বন্ধ করুন (অটো-শাটডাউন); (৩) স্থির লোডে **Reserved Instance** (১/৩ বছর, ৩০–৭০% ছাড়) বা **Savings Plan** ব্যবহার করুন; (৪) অপ্রয়োজনীয় পাবলিক IP, ডিস্ক ও স্ন্যাপশট নিয়মিত পরিষ্কার করুন।

২৪.৭ Architecture Decision Record (ADR)

প্রতিটি গুরুত্বপূর্ণ সিদ্ধান্ত একটি **ADR**-এ লিখে রাখুন — প্রসঙ্গ, বিবেচিত বিকল্প, সিদ্ধান্ত, পরিণতি। উদাহরণ: “আমরা AKS-এর বদলে Container Apps বেছে নিয়েছি, কারণ দলে Kubernetes দক্ষতা কম এবং অ্যাপের জটিলতা মাঝারি — পরিণতি: কম পরিচালনা-ভার, কিন্তু কাস্টম অপারেটর ব্যবহার করা যাবে না।” ছয় মাস পর কেউ যখন জিজ্ঞাসা করবে “কেন এভাবে করা?”, ADR-ই উত্তর দেবে।

```
# ADR-001: কনটেইনার প্ল্যাটফর্ম হিসেবে Azure Container Apps

## অবস্থা
গৃহীত — 9 নভেম্বর 2026

## প্রসঙ্গ
TechNova-র MERN অ্যাপ মাইক্রোসার্ভিসে ভাগ করা হচ্ছে। দলে Kubernetes-এর গভীর দক্ষতা নেই।

## বিবেচিত বিকল্প
1. AKS — পূর্ণ নিয়ন্ত্রণ, বেশি পরিচালনা-ভার
2. Azure Container Apps — সার্ভারলেস, KEDA অটোস্কেল, কম পরিচালনা
3. App Service — সহজ, কিন্তু কনটেইনার-কেন্দ্রিক নয়

## সিদ্ধান্ত
Azure Container Apps বেছে নেওয়া হয়েছে।

## পরিণতি
- ভালো: কম পরিচালনা-ভার, দ্রুত ডেলিভারি, খরচ লোড-অনুযায়ী
```

```
- খারাপ: কাস্টম Kubernetes অপারেটর ব্যবহার করা যাবে না
- পুনর্বিবেচনা: দলে Kubernetes দক্ষতা বাড়লে বা জটিলতা বাড়লে ADR-টি পুনর্বিবেচনা করা হবে
```

২৪.৮ পরিচালন প্রস্তুতি (Operational Readiness)

প্রোডাকশনে যাওয়ার আগে পরীক্ষা করুন: (১) **রানবুক** আছে কি — প্রতিটি সাধারণ ঘটনার জন্য ধাপে ধাপে নির্দেশ? (২) **অ্যালার্ট** পরীক্ষা করা হয়েছে কি — অ্যালার্ট আসলে কেউ সাড়া দেয়? (৩) **ব্যাকআপ restore** পরীক্ষা করা হয়েছে কি? (৪) **DR পরীক্ষা** হয়েছে কি? (৫) **খরচের অ্যালার্ট** লাগানো আছে কি? (৬) **নিরাপত্তা পর্যালোচনা** হয়েছে কি — Defender Secure Score কত? এই তালিকার কোনো একটি “না” হলে প্রোডাকশনে যাওয়া উচিত নয়।

২৪.৯ ক্যাপস্টোনের পথ

এই বইয়ের ক্যাপস্টোন (ল্যাব ওয়ার্কবুকে) আপনাকে TechNova-র সম্পূর্ণ প্রোডাকশন প্ল্যাটফর্ম বানাতে বলবে — এই অধ্যায়ের আর্কিটেকচারই তার নকশা। শুরু করুন ছোট থেকে: প্রথমে VNet ও পরিচয়, তারপর স্টোরেজ ও ডেটাবেস, তারপর অ্যাপ ডিপ্লয়, তারপর CI/CD, তারপর মনিটরিং ও নিরাপত্তা, শেষে DR ও রানবুক। প্রতিটি ধাপে খরচ যাচাই করুন এবং কাজ শেষে test রিসোর্স মুছে ফেলুন।

২৪.১২ সারসংক্ষেপ

প্রোডাকশন আর্কিটেকচার মানে সব স্তরের সমন্বয় — Front Door থেকে ডেটাবেস পর্যন্ত, প্রতিটি স্তর ব্যক্তিগত, পরিচয়-ভিত্তিক, পর্যবেক্ষিত ও পুনরুদ্ধারযোগ্য। ল্যান্ডিং জোন বড় সংস্থার ভিত্তি দেয়। FinOps মানে খরচকে প্রকৌশলের অংশ হিসেবে দেখা — Inform, Optimize, Operate। প্রতিটি গুরুত্বপূর্ণ সিদ্ধান্ত ADR-এ লিখুন। আর প্রোডাকশনে যাওয়ার আগে পরিচালন প্রস্তুতির তালিকা যাচাই করুন — রানবুক, অ্যালার্ট, ব্যাকআপ, DR, খরচ, নিরাপত্তা।

২৪.১৩ পরিভাষা

FinOps — খরচ ব্যবস্থাপনার প্রকৌশল-কেন্দ্রিক পদ্ধতি — Inform, Optimize, Operate।

Landing Zone — Azure-এর প্রস্তুতকৃত ভিত্তি — গভর্ন্যান্স, নেটওয়ার্ক, পরিচয়, লগিংসহ।

ADR — Architecture Decision Record — সিদ্ধান্ত, বিকল্প ও পরিণতির লিখিত রেকর্ড।

Reserved Instance — ১/৩ বছরের প্রতিশ্রুতিতে ছাড়কৃত মূল্য।

Savings Plan — ঘণ্টাভিত্তিক ব্যয়ের প্রতিশ্রুতিতে ছাড়।

Operational Readiness — প্রোডাকশনে যাওয়ার আগে পরীক্ষার তালিকা — রানবুক, অ্যালার্ট, ব্যাকআপ, DR।

২৪.১৪ জ্ঞান যাচাই

1. Front Door কেন এই আর্কিটেকচারের গ্লোবাল এন্ট্রি পয়েন্ট? একটি Region বন্ধ হলে কী হয়?

2. FinOps-এর তিনটি পর্যায় কী কী? প্রতিটির একটি উদাহরণ দিন।
3. ADR কেন লেখা হয়? একটি ভালো ADR-এ কী কী থাকে?
4. Reserved Instance কখন ব্যবহার করবেন, কখন নয়?
5. প্রোডাকশনে যাওয়ার আগে পরিচালন প্রস্তুতির ছয়টি পরীক্ষা কী কী?

২৪.১৫ অনুশীলন

TechNova-র জন্য একটি এক পাতার ADR লিখুন: “ডেটাবেস হিসেবে PostgreSQL Flexible Server” — প্রসঙ্গ, বিকল্প (Azure SQL, Cosmos DB), সিদ্ধান্ত, পরিণতি। তারপর একটি FinOps পরিকল্পনা লিখুন: ট্যাগিং নিয়ম, dev পরিবেশের অটো-শাটডাউন, এবং মাসিক খরচ পর্যালোচনার প্রক্রিয়া।